

Bitácora mercedev (Vol. I): El nacimiento del Boilerplate

Bitacora | Vol. 1

mercedev.es — 2026-04-25

Cómo se construyó una infraestructura web híbrida con seguridad, automatización y rendimiento perfectos — desde cero.

¿Qué es esto?

El **Merci Boilerplate** es una plantilla de infraestructura web reutilizable creada por `mercedev.es`. Combina un núcleo estático ultrarrápido con WordPress/WooCommerce aislados, gobernado todo por un ecosistema de scripts de automatización denominado **Sistema Merci**. El objetivo final: que cualquier proyecto web futuro pueda arrancar con seguridad, rendimiento y buenas prácticas ya integradas desde el minuto uno.

La filosofía que guía cada decisión es el **DevSecOps** y el principio **Shift-Left** — es decir, detectar y resolver problemas de seguridad, calidad y rendimiento lo antes posible, antes de que lleguen a producción.

Fase 1 — Los cimientos (12 de abril de 2026)

Todo empieza con la pregunta correcta: ¿cómo se organiza un proyecto para que sea auditable, seguro y reproducible desde el día uno?

Se diseña la **estructura de carpetas** del repositorio: `public/` para el sitio web, `docs/` para documentación técnica, `scripts/merci/` para las herramientas de automatización, `laboratorio/` para experimentos y `.assets-raw/` para archivos multimedia sin optimizar (excluidos de Git para mantener el repositorio ligero).

El primer artefacto importante es `merci-audit.py`, un script de auditoría escrito en Python puro — sin dependencias externas — que analiza el código en busca de secretos expuestos, errores de sintaxis, malas prácticas de JavaScript y fallos de SEO técnico. Este script se conecta al hook `pre-commit` de Git, lo que

significa que **ningún código pasa al repositorio sin pasar antes por esta revisión automática.**

 **Hito DevSecOps:** Primer escudo de seguridad activo. El auditor bloquea commits con errores (`ERROR`) y advierte sobre mejoras (`WARN`), diferenciando entre lo crítico y lo recomendable.

Fase 2 — El núcleo estático y el SEO técnico (14 de abril de 2026)

Con la infraestructura lista, se construye el primer documento HTML público siguiendo estándares de **semántica HTML5**, **WAI-ARIA** (accesibilidad) y **JSON-LD** (datos estructurados para buscadores).

Se añaden `robots.txt` y `sitemap.xml` para la indexación en buscadores, y se crea `merci-sitemap.py`, un script que actualiza automáticamente la fecha de modificación del sitemap cada vez que se cambia contenido en `public/`. Este script se integra en el hook de `pre-commit` para que nunca haya desincronización entre el código y los metadatos.

También se añade un **linter de acrónimos** al auditor: si un término técnico en inglés aparece sin su expansión (ej. «CSP» sin «Content Security Policy»), el sistema emite un aviso. El objetivo es que la documentación sea comprensible para cualquier nivel.

 **Hito:** El primer `index.html` supera la auditoría sin errores ni advertencias. El proyecto nace ya en verde.

Fase 3 — Sistema de diseño SASS y optimizador de imágenes (15–16 de abril de 2026)

El sistema de estilos

Se construye una arquitectura de estilos modular siguiendo la metodología **SASS 7-1** y la nomenclatura **BEM** (Block, Element, Modifier). Esto significa que todos los estilos están organizados en módulos reutilizables, y los componentes HTML tienen nombres de clase predecibles y estructurados.

Para compilar SASS sin instalar Node.js globalmente (en línea con la filosofía de «cero dependencias externas»), se crea `merci-styles.py` : un script que descarga automáticamente el compilador **Dart Sass** como binario independiente y lo ejecuta en local. Después, se añade `merci-watcher.py` , que vigila los archivos SASS y recompila en tiempo real mientras se trabaja.

El optimizador de imágenes

Se crea `merci-optimizer.py` , que procesa imágenes desde `.assets-raw/` y genera versiones optimizadas en formato **WebP** con múltiples resoluciones para dispositivos distintos. Incluye una corrección importante: si la imagen tiene fondo transparente (como un logo PNG), se preserva correctamente el canal alpha al convertir a WebP.

La arquitectura de información

Se define la tipología de contenidos: una **Biblioteca** para documentación técnica atemporal, y **Art de Côté** para artículos divulgativos con la estructura de tres átomos: Desafío → Maniobra → Aprendizaje (el mismo formato de esta bitácora).

 **Hito DevSecOps:** Se añaden pruebas unitarias (`unittest`) para los scripts del Sistema Merci usando `unittest.mock` . El código de automatización se prueba antes de usarse — TDD aplicado a las propias herramientas.

Pivote estratégico — De web personal a Boilerplate (16 de abril de 2026)

A mitad del desarrollo se toma una decisión importante: el valor real de lo construido no es una web personal concreta, sino la **infraestructura reutilizable** en sí misma. Se pivota formalmente el proyecto de `mercedev.es` a **Merci Boilerplate**.

Se actualiza el `README.md` y el `index.html` para que reflejen este nuevo propósito: quien clone el repositorio encontrará una plantilla lista para usar, con toda la arquitectura DevSecOps ya integrada.

Fase 4 — Integración de WordPress con aislamiento total (15–16 de abril de 2026)

El problema

Añadir WordPress a un proyecto estático de alto rendimiento es un reto. WordPress, por defecto, inyecta decenas de scripts, estilos en línea y dependencias que degradan el rendimiento. La solución no es evitar WordPress, sino **aislarlo**.

La arquitectura de aislamiento

Se diseña un sistema donde:

- El **núcleo estático** vive en `public/` y es servido directamente por **Nginx** (sin pasar por PHP).
- **WordPress** reside en un directorio hermano completamente separado (`/var/www/wordpress/`).
- Nginx actúa como **proxy inverso**: las rutas que empiezan por `/blog` se redirigen a WordPress; el resto se sirven como archivos estáticos planos.

- Un **enlace simbólico** (`ln -s`) conecta el Child Theme del repositorio Git con el directorio de temas de WordPress, de modo que `git pull` actualiza el diseño automáticamente.

El Child Theme ultraligero

Se crea el **Merci Theme**, un tema hijo de WordPress con solo tres archivos. El `functions.php` actúa como «escudo de rendimiento»: desencola los scripts y estilos por defecto de WordPress (emojis, bloques Gutenberg, scripts globales), bloquea endpoints de seguridad obsoletos (XML-RPC, versión de WP, etc.) y encola únicamente el CSS del núcleo estático.

El `index.php` implementa el bucle de WordPress con semántica HTML5 y clases BEM, reutilizando los mismos componentes visuales que el núcleo estático.

WooCommerce en modo catálogo

Se integra WooCommerce pero en **modo catálogo** (sin carrito AJAX ni scripts de pago), lo que elimina el script `wc-cart-fragments` — responsable de una petición HTTP innecesaria en cada carga de página que saltaba por encima de todas las capas de caché.

 **Hito DevSecOps:** `wp-config.php` se configura con claves criptográficas oficiales y permisos `chmod 600` (solo lectura para el propietario). El CMS se instala con el principio de mínimo privilegio desde el primer momento.

Fase 5 — Quality Assurance y Hardening (15–16 de abril de 2026)

Content Security Policy (CSP)

Se implementa una **Política de Seguridad de Contenidos** estricta mediante cabecera HTTP: `default-src 'self'` restringe todos los recursos al dominio

propio, bloqueando cualquier script externo o inyección de código (vectores de ataque XSS).

Se valida que la CSP no bloquea ningún recurso legítimo del sitio estático antes de activarla en producción.

Auditoría PHP para WordPress

El auditor `merci-audit.py` se amplía con una función que detecta funciones PHP peligrosas (`eval()` , `exec()` , `shell_exec()`) que son vectores comunes de ejecución de código remoto (RCE). Si aparecen en el código, se emite una advertencia para revisión manual.

Automatización de commits

Se crea `merci-commit.py` : un script que extrae automáticamente la última entrada de la bitácora y la convierte en el mensaje del commit de Git. Esto garantiza que el código y su justificación siempre viajan juntos en un commit atómico. Incluye salvaguardas: avisa si no hay cambios reales en Git, o si la bitácora no ha sido actualizada.

Checklist de Hardening

Se genera el documento `docs/checklist-hardening.md` con todas las medidas de seguridad implementadas: directivas CSP, hooks de bloqueo de WordPress, política de permisos del servidor y reglas del auditor. Este documento sirve como guía de validación para futuros despliegues.

 **Hito:** La auditoría integral con `merci-audit.py --strict-json-ld` sobre todo el repositorio devuelve **cero errores y cero advertencias**. La Fase 5 se cierra formalmente.

Fase 6 — Despliegue a producción y 100/100 en PageSpeed (17–23 de abril de 2026)

Preparativos pre-despliegue

Se redacta el **Deployment Playbook** (`docs/deployment-playbook.md`), un manual operativo paso a paso para desplegar el proyecto en un servidor real. Se parte de cero: desde la compra del dominio y la configuración DNS, hasta el enrutamiento final de Nginx.

Se realiza una **auditoría externa** del repositorio (mediante GitHub Copilot) que valida la arquitectura híbrida, la seguridad y el aislamiento DevSecOps con la máxima calificación. Se fijan las versiones de dependencias Python con `==` en lugar de `>=` para garantizar reproducibilidad absoluta.

Los primeros obstáculos en producción

El despliegue real descubre varios problemas que no eran visibles en local:

1. **Latencia de 290ms** por haber elegido un datacenter en Asia. Se destruye y recrea el servidor en Europa.
2. **Error DNS NXDOMAIN** al emitir el certificado SSL porque el subdominio `www` no tenía registro A. Se emite el certificado solo para el dominio raíz.
3. **Mixed Content**: WordPress devolvía URLs `http://` porque la función `is_ssl()` de PHP no detectaba correctamente el protocolo detrás del proxy Varnish. Se resuelve usando `home_url()` de WordPress (que lee la configuración de la base de datos) en lugar de variables de servidor crudas.
4. **Puerto 8080 inyectado por Varnish**: el proxy interno añadía el puerto interno a las URLs, generando rutas inválidas. Se corrige con `preg_replace` para extraer la raíz del dominio limpia.
5. **WooCommerce en modo «Coming Soon»**: las versiones modernas de WooCommerce activan una pantalla de mantenimiento por defecto en la base de datos que ignoraba completamente el Child Theme.

Cada problema se documenta en la bitácora con su causa técnica exacta y la lección aprendida.

CloudPanel y el Deployment Playbook real

Se adopta **CloudPanel** como panel de administración del servidor. Esto introduce una capa de abstracción (plantillas `{{root}}` en el VHost de Nginx) que hay que respetar: no se pueden modificar variables del panel directamente o las actualizaciones del sistema romperían la configuración. Se aprende a separar qué se configura por la interfaz visual y qué se inyecta directamente en el bloque VHost.

Depuración CSP avanzada en WooCommerce

WooCommerce inyecta varios scripts en línea que violan la CSP estricta (`script-src 'self'`):

- **Speculation Rules**: un script JSON que WordPress añade al footer para precargar páginas. Se bloquea en ambos hooks (`wp_head` y `wp_footer`).
- **Filtros SVG de Gutenberg**: bloques de filtros visuales que se inyectan tras el `<body>` . Se elimina la acción correspondiente.
- **wc_javascript_is_active** : un script mínimo de WooCommerce que detecta si JavaScript está activo. Las versiones modernas lo inyectan a nivel de renderizado de bloques (Gutenberg), esquivando los `remove_action` tradicionales.

Para el último script, se descubre una **condición de carrera (Race Condition)**: el `remove_action` se ejecutaba antes de que WooCommerce registrara su hook porque no estaba envuelto en el hook `init` . Al encapsularlo en `init` , el orden de ejecución queda garantizado.

Cuando incluso eso falla (WooCommerce usa prioridad 0), se aplica la solución definitiva: **whitelist criptográfico SHA-256**. En lugar de eliminar el script, se calcula su hash exacto y se autoriza solo ese hash específico en la cabecera CSP de Nginx. Si un atacante modifica un solo carácter del script, el hash no coincide y el navegador lo bloquea.

 **Hito DevSecOps**: Esta es la técnica más sofisticada del proyecto.

Permite convivir con código legado de terceros que no se puede eliminar, sin relajar ni un milímetro la postura de seguridad.

El orquestador total

Se crea `merci-total.py` : un script maestro que ejecuta secuencialmente toda la cadena de herramientas antes de cada pase a producción:

```
merci-optimizer → merci-styles → merci-sitemap → merci-audit (SAST) →  
merci-linkcheck (DAST)
```

Implementa un patrón **Fail-Fast**: si cualquier paso falla, la ejecución se detiene. Es el equivalente local de un pipeline de CI/CD.

La puntuación perfecta

Después de todos estos ajustes — el escudo de rendimiento en `functions.php` , las cabeceras de seguridad en Nginx, la depuración extrema de scripts en línea y las micro-correcciones de Core Web Vitals (como especificar dimensiones exactas en el logo para evitar el Cumulative Layout Shift) —, se realiza la auditoría final en **Google PageSpeed Insights**:

| Categoría | Puntuación | |---|---| | Rendimiento (LCP, INP, CLS) |  100/100 | |
Accesibilidad (WAI-ARIA) |  100/100 | | Mejores Prácticas (CSP, WebP, HTTPS) |
 100/100 | | SEO Técnico (JSON-LD, canónicas) |  100/100 |

Esto se valida tanto en la ruta estática como en la dinámica `/blog/tienda/` de WooCommerce. Conseguir 100/100 en un ecosistema WooCommerce es atípico.

Se complementa con una **auditoría manual de accesibilidad**: navegación completa solo con teclado, verificación de trampas de foco, landmarks semánticos y estados de foco visibles. La automatización tiene límites; la empatía con el usuario final no.

 **Hito final del Volumen I:** La Fase 6 se cierra. La construcción de la infraestructura está completa y validada empíricamente.

El Registro Diario (La Caja Negra)

2026-04-23 — Milestone: Cierre definitivo de Fase 6 y validación 100/100

Contexto: Tras una persistente batalla de depuración contra los scripts en línea residuales de WooCommerce, la estrategia final de utilizar un *whitelist* criptográfico (Hash SHA-256) en la cabecera CSP de Nginx fue implementada. Se requería una auditoría final para certificar la erradicación de errores en consola y el cierre de la fase de despliegue.

Hecho: - Se ha validado una puntuación perfecta (100/100) en todas las categorías de Google PageSpeed Insights para la ruta dinámica `/blog/tienda/`. - Se ha confirmado la ausencia total de errores de CSP o JavaScript en la consola del navegador. - Se ha actualizado la fecha de revisión del `checklist-hardening.md` para reflejar la finalización de la Fase 6.

Detalle técnico: La combinación de un "escudo de rendimiento" en `functions.php` (desencolado de scripts, bloqueo de hooks) y un "escudo de infraestructura" en Nginx (CSP con hash) ha demostrado ser la arquitectura definitiva para domar un CMS sin sacrificar seguridad ni velocidad.

Motivo / criterio: La consecución de este hito valida empíricamente la tesis del proyecto: es posible construir una web híbrida que cumpla con los más altos estándares de la industria. Con esto, se da por concluida la etapa de construcción de infraestructura.

Siguiente paso o deuda: Iniciar la Fase 7: Automatización y Clasificación.

2026-04-23 — Fix: Whitelist Criptográfico (CSP Hash) para script en línea residual

Contexto: El script inline `wc_javascript_is_active` de WooCommerce seguía ejecutándose, evadiendo los `remove_action` en `functions.php`. Se diagnosticó que las versiones modernas de WooCommerce inyectan este código

directamente a nivel de renderizado de bloques (Gutenberg), haciéndolo invulnerable a los hooks tradicionales de PHP.

Hecho: - Se optó por una solución de infraestructura en lugar de parchear PHP. - Se inyectó el hash exacto del script (`'sha256-eHL/Izx7K/qWL0kdBXXnHwsLSHvG0Jn/THLHydUZdog='`) en la directiva `script-src` de la cabecera CSP en Nginx. - Se actualizó el checklist de Hardening documentando la práctica de whitelisting criptográfico.

Detalle técnico: En DevSecOps avanzado, cuando un script en línea benigno (y estático) no puede ser erradicado del código legado, la solución no es relajar la seguridad permitiendo `'unsafe-inline'`. La directiva CSP permite autorizar la ejecución exclusiva de una cadena de texto concreta mediante su firma criptográfica SHA-256. Si un atacante modifica un solo carácter del script, el hash cambiará y el navegador lo bloqueará instantáneamente.

Motivo / criterio: Seguridad sin compromisos funcionales. Esta maniobra sella la consola del navegador a 0 errores, mantiene el escudo XSS al 100% de eficacia y nos libera de seguir luchando contra el monolito de bloques de WordPress/ WooCommerce.

Siguiente paso o deuda: Validar la consola en blanco y dar el salto definitivo a la Fase 7.

2026-04-23 — Fix: Sincronización del ciclo de vida de hooks (Race Condition)

Contexto: El último script inline de WooCommerce (`wc_javascript_is_active`) seguía apareciendo en el reporte de PageSpeed a pesar de haber declarado su `remove_action` correspondiente en `functions.php`.

Hecho: - Se refactorizaron las purgas de scripts inline (`wc_javascript_is_active`, *Speculation Rules*, y *Filtros SVG*) encapsulándolas dentro de la función `merci_purgar_inyecciones_inline`. - Se ancló dicha función globalmente al hook `init`.

Detalle técnico: Ocurrió una condición de carrera (Race Condition) en el orden de carga del ciclo de vida de WordPress. Colocar `remove_action` suelto en el

archivo provocaba que la orden de borrado se ejecutara en un momento donde el plugin a veces aún no había consolidado el hook, o el borrado era ignorado por la inusual prioridad `0` que WooCommerce utiliza para disparar antes del ciclo normal. Encapsular la purga dentro de `init` asegura que la orden se ejecute cuando todo el *core* y los plugins ya están cargados en memoria.

Motivo / criterio: Conocimiento profundo del ciclo de ejecución (Lifecycle) del framework. Cuando las órdenes de anulación de código fallan silenciosamente, el problema casi siempre reside en el "cuándo" y no en el "qué". Envolver purgas de seguridad en hooks consolidados es la práctica definitiva de blindaje contra código de terceros.

Siguiente paso o deuda: Validar la desaparición del hash en la consola e iniciar la Fase 7.

2026-04-23 — Fix: Aniquilación del último script inline de WooCommerce (CSP)

Contexto: Tras la purga de *Speculation Rules* y filtros SVG, PageSpeed seguía detectando una única violación de la Política de Seguridad de Contenido (CSP) por un script en línea no identificado (hash `sha256-eHL...`).

Hecho: - Se identificó la acción `wc_javascript_is_active` inyectada en el `wp_head` con prioridad 0. - Se implementó `remove_action('wp_head', 'wc_javascript_is_active', 0)` en `functions.php`.

Detalle técnico: WooCommerce inyecta un minúsculo script `<script>document.body.className = ...</script>` al inicio de la cabecera para cambiar la clase `woocommerce-no-js` a `woocommerce-js`. Al no estar en un archivo `.js` externo, este bloque chocaba frontalmente con la directiva `script-src 'self'`. Al estar el sitio en Modo Catálogo y con los scripts de carrito desencolados, esta verificación de estado es código muerto.

Motivo / criterio: Limpieza extrema y Zero Tolerance. Un solo script bloqueado es una advertencia en consola y una mancha en el reporte de rendimiento/seguridad.

Localizar el *hook* exacto y neutralizarlo desde el backend (PHP) es la única vía para conciliar un CMS pesado con una arquitectura DevSecOps limpia y sin errores.

Siguiente paso o deuda: Validar la consola del navegador limpia (0 errores) y cerrar definitivamente la Fase 6.

2026-04-23 — Fix: Erradicación definitiva de scripts en línea residuales (WP 6.x)

Contexto: Aunque se purgó el grueso de scripts de WooCommerce, PageSpeed Insights reportó un 92/100 en Mejores Prácticas debido a dos bloques `<script>` en línea restantes que violaban la Política de Seguridad de Contenido (CSP): *Speculation Rules* y un script anónimo (filtros SVG de Gutenberg).

Hecho: - Se amplió el bloqueo de `wp_print_speculation_rules` al hook `wp_footer`. - Se eliminó la acción `wp_global_styles_render_svg_filters` inyectada por el motor de bloques de WordPress en `wp_body_open` y `wp_footer`.

Detalle técnico: WordPress 6.x y las versiones recientes de WooCommerce son sumamente obstinados inyectando código en línea. Las *Speculation Rules* intentan ejecutarse en el pie de página si son bloqueadas en la cabecera, y los filtros SVG (duotone) se inyectan directamente tras abrir el cuerpo del documento. Al estar bajo una CSP estricta (`script-src 'self'`), el navegador los interceptaba con éxito, marcando la violación en consola.

Motivo / criterio: Tolerancia cero frente a la deuda técnica. Ignorar un 92/100 asumiéndolo como "suficientemente bueno" es el primer paso hacia la degradación estructural de un proyecto. Extirpar este código basura residual demuestra control absoluto sobre el motor de renderizado dinámico (CMS) y sella la perfección de la auditoría.

Siguiente paso o deuda: Validar la puntuación perfecta final (100/100) en PageSpeed e iniciar la Fase 7.

2026-04-23 — Fix: Depuración estricta de scripts dinámicos y CSP en WooCommerce

Contexto: La auditoría de PageSpeed Insights para la ruta `/blog/tienda/` reportó violaciones de la Política de Seguridad de Contenido (CSP), un `TypeError` en `order-attribution.min.js` y la carga innecesaria de jQuery.

Hecho: - Se amplió la función `merci_limpiar_scripts_wc` en `functions.php` para desencolar `wc-order-attribution`, `wc-add-to-cart`, `woocommerce` y desregistrar `jquery` en el frontend. - Se eliminó la acción `wp_print_speculation_rules` para evitar la inyección de JSON/JS en línea por parte de WordPress.

Detalle técnico: WooCommerce inyecta variables de configuración como scripts en línea (`<script>...</script>`). Al tener una cabecera HTTP CSP estricta (`script-src 'self'`), el navegador bloqueaba estos bloques en línea. Al cargar los scripts externos de WooCommerce, estos intentaban leer las variables bloqueadas, resultando en `undefined` y desencadenando el `TypeError`.

Motivo / criterio: Resiliencia arquitectónica (Shift-Left). Frente al dilema de debilitar la seguridad de la CSP permitiendo `'unsafe-inline'` o eliminar los scripts conflictivos, se optó por lo segundo. Dado que la tienda opera en "Modo Catálogo", los scripts de atribución de pedidos y carritos AJAX son peso muerto. Erradicarlos protege la puntuación de rendimiento, elimina la dependencia de jQuery y preserva la máxima postura de seguridad contra XSS.

Siguiente paso o deuda: Validar la resolución de los errores en la consola del navegador y cerrar la fase de auditoría dinámica.

2026-04-23 — Fix: Resolución de micro-métricas de Core Web Vitals (CLS y Render-Blocking)

Contexto: Un análisis exhaustivo de PageSpeed Insights alertó sobre un leve Cumulative Layout Shift (CLS de 0.022), recursos que bloquean el renderizado (`main.css`) y discrepancias en el tamaño del logotipo renderizado.

Hecho: - Se corrigieron los atributos HTML del logotipo en todas las vistas, pasando de `width="150" height="auto"` a valores absolutos exactos

(`width="263" height="65"`). - Se desestimó explícitamente la advertencia sobre el CSS bloqueante (`main.css`).

Detalle técnico: El atributo `height="auto"` es inválido en HTML5 y provoca que el navegador no reserve espacio vertical previo a la carga de la imagen, causando el micro-salto (CLS). Al aplicar las dimensiones exactas reportadas por el DOM, el salto desaparece. Respecto al CSS bloqueante, al pesar solo 1.7 KiB y resolver en ~150ms, su externalización es preferible frente a inyectar CSS en línea, preservando la limpieza del HTML y la arquitectura SASS.

Motivo / criterio: Pragmatismo frente a la automatización. No todas las advertencias de PageSpeed requieren reescribir la infraestructura. Optimizar un archivo de 15 KiB ahorrando 13 KiB o inyectar estilos críticos rompiendo el *Separation of Concerns* constituye sobreingeniería pura. La corrección semántica (atributos de imagen) es suficiente para garantizar el 100/100 real.

Siguiente paso o deuda: Iniciar la Fase 7 y diseñar el pipeline de publicación automatizada.

2026-04-23 — QA: Auditoría Manual de Accesibilidad (Lighthouse)

Contexto: Google PageSpeed Insights (Lighthouse) reporta 10 comprobaciones de accesibilidad que no pueden ser verificadas automáticamente (como el orden lógico de tabulación, trampas de foco y visibilidad de elementos fuera de pantalla). Era necesario certificar el cumplimiento de estos puntos para asegurar un proyecto verdaderamente inclusivo.

Hecho: - Se ejecutó una prueba exhaustiva de navegación exclusivamente por teclado (Tabulación). - Se verificaron los estados de foco, el flujo visual-vs-DOM y el comportamiento del menú fuera de pantalla. - Se revalidó la implementación de *Landmarks* semánticos y etiquetas `aria-label` .

Detalle técnico: Se confirmó que el núcleo estático no contiene "trampas de foco" (focus traps) y que los elementos interactivos personalizados (`<button id="menu-toggle">`) están contruidos sobre etiquetas nativas con atributos WAI-ARIA descriptivos, obviando la necesidad de inyectar `role="button"` artificialmente. Se comprobó que el anillo de foco (`outline`) nativo del navegador es claramente visible.

Motivo / criterio: La automatización tiene límites. Un "100/100" en herramientas automatizadas es una ilusión si un usuario con tecnologías de asistencia no puede navegar lógicamente por la página. La auditoría manual cierra la brecha entre la métrica técnica y la empatía con el usuario final.

Siguiente paso o deuda: Iniciar la Fase 7 y diseñar el pipeline de publicación automatizada.

2026-04-23 — Fix: Refinamiento de HSTS y justificación de deuda en Trusted Types

Contexto: Tras la migración de cabeceras de seguridad a Nginx, la auditoría reportó dos advertencias restantes: la ausencia de la directiva `preload` en el HSTS y la falta de `Trusted Types` en la CSP.

Hecho: - Se añadió la directiva `preload` a la cabecera `Strict-Transport-Security` en CloudPanel. - Se desestimó explícitamente la implementación de `require-trusted-types-for` en la CSP.

Detalle técnico: El uso de `preload` inscribe el dominio en las listas maestras de los navegadores para garantizar conexiones HTTPS desde la primera solicitud (mitigando el primer milisegundo de vulnerabilidad). Por otro lado, la directiva `Trusted Types` bloquea el uso de sumideros del DOM basados en cadenas de texto (como `innerHTML`); activar esta directiva fracturaría la operatividad de WordPress, sus plugins y el editor de bloques (Gutenberg), ya que su código base aún no es compatible de forma nativa con esta API estricta.

Motivo / criterio: Pragmatismo arquitectónico. La seguridad extrema no debe destruir la funcionalidad core del producto. Aceptar la advertencia de `Trusted Types` se clasifica como una *Deuda Técnica conocida y asumida* derivada del uso de un CMS maduro como WordPress. Con esta acción se cierra formalmente la subfase 5.5.

Siguiente paso o deuda: Iniciar el diseño del flujo de publicación automatizado en la Fase 7.

2026-04-23 — Feat: Hardening avanzado de cabeceras HTTP (CSP, HSTS, COOP)

Contexto: La auditoría de Google PageSpeed Insights señaló la ausencia de cabeceras de seguridad críticas (HSTS, COOP) y una implementación débil de la Política de Seguridad de Contenido (CSP) mediante etiqueta `<meta>`, considerándola no efectiva contra ataques XSS.

Hecho: - Se ha definido un bloque de cabeceras de seguridad para Nginx. - Se ha migrado la CSP de la etiqueta `<meta>` a una cabecera `Content-Security-Policy` HTTP. - Se han añadido las cabeceras `Strict-Transport-Security` (HSTS), `Cross-Origin-Opener-Policy` (COOP), `Cross-Origin-Embedder-Policy` (COEP), `Referrer-Policy` y `X-Content-Type-Options`. - Se ha documentado el proceso de inyección en el VHost de CloudPanel.

Detalle técnico: La implementación vía cabecera HTTP es el método de aplicación (enforcement) correcto. La CSP se ha ajustado con `style-src 'self' 'unsafe-inline'` como compromiso de compatibilidad con la barra de administración de WordPress. Se ha documentado la complejidad de `Trusted Types` como una mejora futura. Las cabeceras se inyectan en el bloque `server` del VHost de Nginx.

Motivo / criterio: Elevar la postura de seguridad del Boilerplate al máximo nivel posible, mitigando vectores de ataque como XSS, Clickjacking, MIME-sniffing y ataques de canal lateral (Spectre), siguiendo las mejores prácticas de la industria recomendadas por Google.

Siguiente paso o deuda: Aplicar las cabeceras en el VHost de producción, eliminar la etiqueta `<meta>` de los archivos HTML y re-auditar en PageSpeed Insights para validar la corrección.

2026-04-23 — Fix: Inyección de meta descripción dinámica en WordPress (SEO)

Contexto: Se detectó que las páginas dinámicas generadas por WordPress (incluyendo la Tienda de WooCommerce) carecían de la etiqueta `<meta name="description">` en el `<head>`, lo que penaliza la auditoría SEO y afecta a

la presentación en los motores de búsqueda. Las páginas del núcleo estático sí la tenían implementada manualmente.

Hecho: - Se creó la función `merci_inyectar_metadatos_seo` en `src/wp-theme/merci-theme/functions.php`. - Se ancló la función al hook `wp_head`.

Detalle técnico: WordPress no genera descripciones meta de forma nativa. La función implementada evalúa el contexto (`is_shop()` , `is_category()` , `is_singular()`) para extraer dinámicamente extractos de artículos o textos por defecto. Incluye una validación (`class_exists`) para apagarse automáticamente si en un futuro se instala un plugin de SEO especializado (como Yoast), evitando etiquetas duplicadas.

Motivo / criterio: Mantener la máxima puntuación (100/100) en SEO técnico sin obligar a la instalación inmediata de plugins pesados de terceros. Esto respeta la filosofía de "0 dependencias externas" y el principio de austeridad tecnológica del Boilerplate.

Siguiente paso o deuda: Verificar la aparición de la etiqueta en el código fuente de la tienda dinámica y dar por cerrada definitivamente la Fase 6 de despliegue.

2026-04-23 — Validación: Core Web Vitals en rutas dinámicas (WooCommerce)

Contexto: Tras resolver los conflictos con el proxy Varnish y desactivar el modo mantenimiento intrusivo, era imperativo auditar el rendimiento real de la tienda en producción (`/blog/tienda`) mediante Google PageSpeed Insights para confirmar la viabilidad de la arquitectura.

Hecho: - Se analizaron los reportes de PageSpeed para las vistas móvil y de escritorio de la ruta dinámica de WooCommerce. - Se validó la retención de las métricas de excelencia logradas previamente en el entorno estático puro.

Detalle técnico: Alcanzar la perfección en Core Web Vitals (LCP, INP, CLS) dentro de un ecosistema WooCommerce es atípico. Esto certifica que el "escudo de rendimiento" codificado en `functions.php` (desencolado del script `wc-cart-fragments` , bloqueo de `global-styles` y uso estricto del atributo `defer` en

JS) funciona a la perfección. El proxy inverso de Nginx/CloudPanel despacha el HTML dinámico con una eficiencia comparable a un archivo plano.

Motivo / criterio: Validación empírica del esfuerzo arquitectónico. La separación de responsabilidades y el enfoque "Shift-Left" en rendimiento demuestran que es posible utilizar un CMS pesado para gestión de datos sin sacrificar en absoluto la velocidad de carga ni la experiencia de usuario (UX).

Siguiente paso o deuda: Dar por clausurada la Fase 6 de despliegue y auditoría, y comenzar la Fase 7 (Automatización y Clasificación).

2026-04-23 — Fix: Desactivación del modo "Coming Soon" de WooCommerce

Contexto: Tras restaurar con éxito la carga de estilos, la web no mostraba el diseño del Child Theme, sino un mensaje genérico ("Tenemos grandes proyectos por anunciar..."). Se diagnosticó que se trataba de la pantalla de mantenimiento nativa de WooCommerce.

Hecho: - Se accedió al panel de administración de WordPress en producción. - Se desactivó el modo "Próximamente" (Coming Soon) en los ajustes de visibilidad de WooCommerce, cambiándolo a "Público" (Live).

Detalle técnico: Las versiones modernas de WooCommerce (≥ 9.0) activan por defecto una opción de visibilidad en la base de datos (`woocommerce_coming_soon`) tras su instalación. Este modo inyecta una plantilla predeterminada que secuestra el enrutamiento (`template_include`), ignorando por completo los archivos `index.php` o `woocommerce.php` de nuestro *Child Theme*. Los estilos sí cargaban correctamente porque WordPress sigue ejecutando el archivo `functions.php` en segundo plano.

Motivo / criterio: Separación Código/Estado. Al igual que las páginas o taxonomías, el estado de los plugins reside en la base de datos y no viaja a través de Git. Conocer y documentar los comportamientos intrusivos de herramientas de terceros evita depurar código estructural que es válido pero está siendo ignorado por la configuración temporal del CMS.

Siguiente paso o deuda: Recargar el frontend para validar que ahora sí se ejecuta la estructura HTML5 y BEM dinámica del Child Theme.

2026-04-23 — Fix: Resolución de inyección de puerto 8080 por Varnish

Contexto: Tras el intento de usar URLs relativas al protocolo (`//`), la web seguía cargando sin estilos ("parecía otra página"). El diagnóstico revela que Varnish en CloudPanel no solo ofusca el protocolo, sino que inyecta su puerto interno (`8080`) en la variable `$_SERVER['HTTP_HOST']` . Esto generaba URLs inválidas como `//mercedev.es:8080/css/main.css` , las cuales eran bloqueadas por los navegadores (especialmente Firefox).

Hecho: - Se refactorizó `$domain_root` en `src/wp-theme/merci-theme/functions.php` . - Se eliminó la dependencia absoluta de `$_SERVER['HTTP_HOST']` . - Se implementó la función nativa `home_url()` de WordPress, recortando el sufijo `/blog` mediante la expresión regular `preg_replace` .

Detalle técnico: La función `home_url()` lee la ruta base configurada directamente en la base de datos (`https://mercedev.es/blog`), la cual ya es completamente segura y agnóstica a los puertos internos del proxy inverso. Al aplicar `preg_replace('#/blog/?$#', '', home_url())` , extraemos dinámicamente la raíz absoluta real (`https://mercedev.es` o `http://localhost`), garantizando que los estáticos se encolen correctamente independientemente de la topología del servidor.

Motivo / criterio: Resiliencia arquitectónica extrema. Leer variables de servidor brutas (`$_SERVER`) detrás de un proxy de alto rendimiento (Nginx + Varnish) es un antipatrón propenso a fallos. Confiar en la abstracción nativa del framework (WP) que ya está sanitizada por la configuración es la solución definitiva (Single Source of Truth).

Siguiente paso o deuda: Validar en producción la carga exitosa de estilos tanto en Chrome como en Firefox y avanzar con la auditoría de rendimiento.

2026-04-23 — Fix: Resolución de Mixed Content detrás de proxy Varnish

Contexto: Tras purgar la caché de Varnish, la página web dejó de cargar los estilos (desconfiguración de diseño) tanto en PC como en móviles. Al estar detrás de un proxy inverso (Varnish/CloudPanel en el puerto 8080), la función `is_ssl()` de WordPress devolvía `false`. Esto provocaba que la web forzara la URL del CSS mediante `http://`, siendo bloqueada por los navegadores por políticas de *Mixed Content* en una web segura (HTTPS).

Hecho: - Se refactorizó la variable `$domain_root` en `src/wp-theme/merci-theme/functions.php`. - Se sustituyó el condicional `is_ssl()` por una URL relativa al protocolo (`//`). - Se unificaron y corrigieron las entradas malformadas previas en la bitácora que interrumpían el flujo de `merci-commit.py`.

Detalle técnico: Una URL que empieza por `//` instruye al navegador a utilizar el mismo protocolo que la página actual. Esto sorteaba la "ceguera" de PHP frente al estado SSL cuando la terminación TLS se realiza en capas superiores (Nginx). Además, se corrigieron las etiquetas en el registro histórico para asegurar que las expresiones regulares (Regex) de `merci-commit.py` encuentren exactamente los delimitadores de inicio (ej. `**Contexto:**` en lugar de `**Contexto (Desafío):**`).

Motivo / criterio: Resiliencia arquitectónica en DevSecOps. Delegar la resolución del protocolo al cliente (navegador) es más seguro y eficiente que intentar adivinar la topología del servidor desde el backend. Mantener la estructura estricta en el Markdown es vital para la automatización atómica.

Siguiente paso o deuda: Validar la restitución del diseño en producción y verificar que el orquestador de commits procesa correctamente la bitácora saneada.

2026-04-21 — Fix: Cache Busting global y purga de Varnish en producción

Contexto: Al visitar la tienda en dispositivos móviles y tras desplegar la nueva plantilla, se visualizaba una estructura rota. La agresiva caché de los navegadores y la retención del proxy apuntaban a versiones obsoletas del documento HTML y del archivo `main.css`.

Hecho: - Se inyectó el parámetro `?v=2` en las etiquetas `<link>` de `public/index.html` y `public/biblioteca/index.html` . - Se actualizó el parámetro de versión de `'1.0.0'` a `'1.0.1'` en la función `wp_enqueue_style` dentro de `src/wp-theme/merci-theme/functions.php` . - Se purgó la caché del servidor (Clear Cache / Purge All) directamente desde la interfaz de CloudPanel.

Detalle técnico: CloudPanel enruta el tráfico PHP a través del puerto 8080 hacia Varnish. Alterar plantillas PHP no invalida automáticamente este snapshot. Además, los dispositivos carecen de *hard refresh*, por lo que alterar la cadena de consulta (query string) de la URL del recurso estático obliga al navegador a descargar las nuevas reglas compiladas.

Motivo / criterio: Control de Caché en arquitecturas de alto rendimiento. Cualquier pase a producción que modifique el diseño visual debe incrementar versiones en los enlaces de carga y purgar la capa de Varnish para garantizar la paridad visual.

Siguiente paso o deuda: Validar la visualización final tras la purga de caché.

2026-04-21 — Chore: Resolución de linter de acrónimos para AJAX

Contexto (Desafío): Al ejecutar el orquestador `merci-total` , el auditor (`merci-audit.py`) reportó una advertencia (WARN) por el acrónimo "AJAX" sin expandir en el nuevo documento `biblioteca/auditoria-rendimiento.md` .

Hecho (Maniobra): - Se expandió el acrónimo AJAX (Asynchronous JavaScript and XML - JavaScript Asíncrono y XML) en el archivo correspondiente.

Detalle técnico: Para cumplir con el estándar de `0 errores`, `0 advertencias` impuesto por el pipeline de integración local, se aplicó la convención de expansión de acrónimos a la documentación técnica recién creada.

Motivo / criterio (Aprendizaje): Disciplina documental y cero fricción técnica. Ningún aviso del linter debe ignorarse si se busca la excelencia técnica absoluta. Expandir los acrónimos facilita la comprensión del documento a cualquier nivel, respetando la filosofía pedagógica del proyecto.

Siguiente paso o deuda: Ejecutar `merci-total` por última vez para confirmar la ausencia total de advertencias y realizar el commit final de despliegue.

2026-04-21 — Docs: Elaboración del reporte de Core Web Vitals (100/100)

Contexto (Desafío): Tras completar el despliegue en producción de la arquitectura híbrida (Núcleo estático + WordPress + WooCommerce), se realizaron las auditorías en Google PageSpeed Insights obteniendo puntuación perfecta (100/100) en todos los pilares. Era necesario documentar este hito traduciendo las métricas a un activo de conocimiento.

Hecho (Maniobra): - Se creó el documento didáctico `biblioteca/auditoria-rendimiento.md`. - Se explicaron los 4 pilares auditados: Rendimiento (LCP/INP/CLS), Accesibilidad (WAI-ARIA/Contraste), Mejores Prácticas (CSP/WebP/HTTPS) y SEO (JSON-LD/Canónicas). - Se marcaron como completados los hitos de la Fase 6.4 en `README.md`.

Detalle técnico: El informe vincula empíricamente las decisiones arquitectónicas "Shift-Left" (Vanilla JS, SASS sin frameworks, desencolado de scripts en WooCommerce) con el resultado positivo en herramientas de auditoría externa. Sirve como validación definitiva del *Aislamiento Dinámico*.

Motivo / criterio (Aprendizaje): Gestión del Conocimiento. Los números perfectos no tienen valor a largo plazo si el equipo no comprende por qué se obtuvieron. Documentar el éxito cierra el ciclo DevSecOps y asienta las bases pedagógicas del proyecto (Regla de la Biblioteca).

Siguiente paso o deuda: Iniciar la Fase 6.3 (Verificación SEO Final) y preparar la transición hacia la Fase 7 (Automatización y Clasificación).

2026-04-21 — Feat: Orquestador maestro de pipeline (merci-total)

Contexto (Desafío): Ejecutar individualmente los scripts de optimización, compilación y auditoría antes de cada pase a producción generaba fricción operativa y riesgo de omisión de pasos críticos.

Hecho (Maniobra): - Se creó el script `scripts/merci/merci-total.py` para orquestar la ejecución secuencial de todas las herramientas. - Se inyectó el alias `merci-total` en el entorno local.

Detalle técnico: El script define un pipeline lógico: `merci-optimizer.py` (Assets) -> `merci-styles.py` (CSS) -> `merci-sitemap.py` (SEO - Search Engine Optimization) -> `merci-audit.py` (SAST - Static Application Security Testing) -> `merci-linkcheck.py` (DAST - Dynamic Application Security Testing). Implementa un patrón "Fail-Fast", deteniendo la ejecución si algún subproceso falla. Excluye explícitamente procesos interactivos (`merci-commit.py`) o demonios (`merci-watcher.py`).

Motivo / criterio (Aprendizaje): CI/CD (Continuous Integration / Continuous Deployment - Integración Continua / Despliegue Continuo) Local. Consolidar la cadena de suministro en un único comando garantiza que el código siempre se optimice y audite antes de integrarse, coronando la arquitectura de automatización del proyecto.

Siguiente paso o deuda: Validar la orquestación total y ejecutar el commit final.

2026-04-21 — Chore: Adición de alias faltantes y resolución de linter de acrónimos

Contexto (Desafío): Durante la preparación para el despliegue final, se constató que comandos como `merci-linkcheck` o `merci-sitemap` no tenían alias configurados en zsh, y el auditor de Markdown reportó el acrónimo "CPU" sin expandir en el análisis de Copilot.

Hecho (Maniobra): - Se inyectaron los alias faltantes (`merci-linkcheck` , `merci-sitemap`) en el archivo `~/.zshrc` . - Se expandió el acrónimo CPU (Central Processing Unit - Unidad Central de Procesamiento) en `docs/Analisi-exhaustivo-antes-de-produccion-copilot-github.md` .

Detalle técnico: Mantener los alias actualizados para todas las herramientas del ecosistema en el perfil de la terminal (zsh) elimina la fricción de tener que recordar las extensiones `.py` o las rutas absolutas, homogeneizando el flujo DevSecOps.

Motivo / criterio (Aprendizaje): Higiene de terminal y estricto cumplimiento de convenciones. Responder inmediatamente a los avisos no bloqueantes del auditor (WARN) previene la acumulación de deuda técnica documental, asegurando un pase a producción impecable sin advertencias.

Siguiente paso o deuda: Ejecutar el último commit atómico y proceder con el test real final en producción.

2026-04-21 — Fix: Prevención de Fatal Error por ausencia de dependencias (WooCommerce)

Contexto (Desafío): Al cargar la tienda en el entorno local (donde el plugin de WooCommerce no está instalado), la plantilla no renderizaba el catálogo y devolvía la vista genérica de artículo, además de presentar riesgo de colapso si se forzaba su ejecución.

Hecho (Maniobra): - Se envolvió la llamada principal en `src/wp-theme/mercitheme/woocommerce.php` con un escudo de seguridad (`if ( function_exists( 'woocommerce_content' ) )`).

Detalle técnico: La asimetría de entornos (Dev-Prod Parity) implica que no siempre existirán las mismas dependencias de base de datos o plugins. Sin WooCommerce, WordPress ignora `woocommerce.php` por defecto. Si forzara su carga, invocar `woocommerce_content()` provocaría un *Fatal Error* de PHP. El escudo condicional permite fallar con elegancia (Fail Gracefully).

Motivo / criterio (Aprendizaje): Resiliencia del código. El código fuente nunca debe asumir ciegamente que un plugin de terceros estará siempre activo. Proteger las llamadas externas garantiza que el núcleo del tema sobreviva a desactivaciones accidentales en producción o a entornos de desarrollo locales austeros.

Siguiente paso o deuda: Finalizar el ciclo de despliegue a producción, donde el plugin sí reside, y ejecutar la auditoría de Core Web Vitals en PageSpeed.

2026-04-21 — Fix: Inyección de Favicon dinámico y restauración de symlink local

Contexto (Desafío): El `favicon.ico` no se mostraba en las páginas de WordPress (`/blog`), y los cambios en los archivos `.php` locales no tenían efecto en el navegador, evidenciando una desconexión del entorno de desarrollo. No se han realizado modificaciones sobre el logotipo.

Hecho (Maniobra): - Se inyectó explícitamente la etiqueta `<link rel="icon" href="/favicon.ico?v=3" type="image/x-icon">` en el `<head>` de `src/wp-theme/merci-theme/index.php`. - Se eliminó la copia huérfana en `/var/www/wordpress/wp-content/themes/merci-theme` y se restauró el enlace simbólico local (`ln -s`) apuntando al repositorio.

Detalle técnico: WordPress no emite un favicon por defecto a menos que se configure en su base de datos. Al inyectarlo directamente en el `index.php` del Child Theme, se garantiza que el CMS utilice el mismo archivo físico de la raíz estática. La restauración del symlink soluciona el "falso negativo" del entorno local causado por purgas anteriores.

Motivo / criterio (Aprendizaje): Control estricto de la UI en entornos híbridos y paridad Dev-Prod. Confiar en que el CMS herede comportamientos visuales por defecto suele fallar. Además, el entorno de desarrollo local debe mantener exactamente la misma arquitectura de enlaces simbólicos que producción.

Siguiente paso o deuda: Comitear los cambios, desplegar a producción y ejecutar la auditoría de rendimiento final.

2026-04-21 — Aprovisionamiento manual de dependencias del CMS (WooCommerce)

Contexto (Desafío): Tras el despliegue del código y la configuración del entorno de producción, surgió la duda sobre el estado operativo de la "Tienda" y la presencia del motor de WooCommerce en el servidor.

Hecho (Maniobra): - Se constató que el plugin de WooCommerce no viaja a través del control de versiones (Git). - Se instruyó la instalación y activación manual del plugin desde el panel de administración de WordPress en producción, omitiendo el asistente de configuración.

Detalle técnico: En una arquitectura de aislamiento, el repositorio Git gobierna el código propietario y la configuración del proxy. Las carpetas de dependencias de terceros (`wp-content/plugins/`) quedan excluidas explícitamente. Las reglas de optimización inyectadas en `functions.php` permanecen latentes hasta que el plugin es activado.

Motivo / criterio (Aprendizaje): Inmutabilidad selectiva. Permitir que los plugins se gestionen visualmente en producción mientras el tema se gestiona estrictamente por código garantiza la operabilidad sin romper el escudo de rendimiento.

Siguiente paso o deuda: Auditar la ruta de la tienda (`/blog/tienda`) en PageSpeed Insights.

2026-04-21 — Fix: Resolución de error NXDOMAIN en emisión de certificado SSL

Contexto (Desafío): Al intentar emitir el certificado Let's Encrypt desde CloudPanel, el sistema devolvió un error de validación DNS (`NXDOMAIN`) para el subdominio `www.mercedev.es` .

Hecho (Maniobra): - Se eliminó el subdominio `www.mercedev.es` de la lista de dominios solicitados (SANs) en la interfaz de CloudPanel. - Se emitió el certificado SSL/TLS exclusivamente para el dominio raíz (apex domain): `mercedev.es` .

Detalle técnico: Let's Encrypt exige que todos los nombres de dominio de la solicitud resuelvan hacia la IP del servidor. Al carecer la Zona DNS de un registro 'A' o 'CNAME' explícito para el `www` , el desafío HTTP-01 fracasa.

Motivo / criterio (Aprendizaje): Austeridad técnica y URLs canónicas. El prefijo `www` es un artefacto de la web clásica. Renunciar a él reduce la complejidad de la Zona DNS y se alinea con la filosofía minimalista.

Siguiente paso o deuda: Comprobar la emisión exitosa del certificado para el dominio raíz y ejecutar la auditoría de rendimiento.

2026-04-21 — Fix: Emisión de Certificado SSL nativo en CloudPanel

Contexto (Desafío): El dominio en producción mostraba la advertencia de "Sitio no seguro" (HTTP). Se planteó la duda de si utilizar la herramienta tradicional `certbot` por terminal para instalar el certificado Let's Encrypt.

Hecho (Maniobra): - Se descartó el uso manual de `certbot` vía CLI (Command Line Interface - Interfaz de Línea de Comandos). - Se emitió el certificado SSL/

TLS (Secure Sockets Layer / Transport Layer Security) directamente desde la pestaña nativa de CloudPanel (Actions > New Let's Encrypt Certificate).

Detalle técnico: CloudPanel gestiona sus propios bloques `server` en Nginx mediante plantillas. Emitir el certificado desde su GUI asegura que las directivas `listen 443 ssl` y las rutas a las llaves criptográficas se inyecten limpiamente sin sobrescribir nuestro enrutamiento híbrido personalizado (Fase 4 del Playbook).

Motivo / criterio (Aprendizaje): Respeto por la abstracción del IaaS (Infrastructure as a Service). Mezclar herramientas de bajo nivel de sistema operativo con paneles de gestión genera conflictos de configuración (Configuration Drift). La integración nativa garantiza además la renovación automática del certificado sin necesidad de configurar *cronjobs* manuales.

Siguiente paso o deuda: Comprobar que la web carga bajo el protocolo HTTPS y ejecutar, ahora sí, la auditoría final de rendimiento.

2026-04-21 — Fix: Sincronización de estado (Páginas y Taxonomías) en producción

Contexto (Desafío): Tras el despliegue exitoso a producción, se detectó que el *hero* de la página "Tienda" no se renderizaba en el entorno público, a pesar de funcionar correctamente en local.

Hecho (Maniobra): - Se diagnosticó una asimetría de estado en la base de datos: la condición lógica `is_page('tienda')` fallaba silenciosamente porque la página física aún no existía en el WordPress de producción. - Se instruyó la creación manual de las páginas base (Tienda) y categorías taxonómicas (Art de Coté) en el panel de administración de producción.

Detalle técnico: El control de versiones (Git) transporta código inmutable y lógica condicional, pero no el estado de la base de datos. Las funciones de enrutamiento interno de WordPress (`is_page()` , `is_category()`) requieren que las entidades existan físicamente en las tablas `wp_posts` y `wp_terms` del entorno actual para que las sentencias `if` se resuelvan como verdaderas.

Motivo / criterio (Aprendizaje): Paridad Dev-Prod (Código vs. Datos). En despliegues de arquitecturas CMS, inyectar la plantilla (Child Theme) es solo la primera mitad de la integración. Siempre se requiere un proceso de aprovisionamiento de datos (Data Seeding) en producción para recrear las anclas de contenido sobre las que pivota el diseño condicional.

Siguiente paso o deuda: Validar la aparición del componente *hero* tras crear la página y proceder con la auditoría final de PageSpeed.

2026-04-21 — Fix: Inyección de Favicon dinámico y restauración de symlink local

Contexto (Desafío): El `favicon.ico` no se mostraba en las páginas de WordPress (`/blog`), y los cambios en los archivos `.php` locales no tenían efecto en el navegador, evidenciando una desconexión del entorno de desarrollo.

Hecho (Maniobra): - Se inyectó explícitamente la etiqueta `<link rel="icon" href="/favicon.ico?v=3" type="image/x-icon">` en el `<head>` de `src/wp-theme/merci-theme/index.php`. - Se eliminó la copia huérfana en `/var/www/wordpress/wp-content/themes/merci-theme` y se restauró el enlace simbólico local (`ln -s`) apuntando al repositorio.

Detalle técnico: WordPress no emite un favicon por defecto a menos que se configure en su base de datos. Al inyectarlo directamente en el `index.php` del Child Theme, se garantiza que el CMS (Content Management System - Sistema de Gestión de Contenidos) utilice el mismo archivo físico de la raíz estática. La restauración del symlink soluciona el "falso negativo" del entorno local causado por purgas anteriores.

Motivo / criterio (Aprendizaje): Control estricto de la UI en entornos híbridos y paridad Dev-Prod. Confiar en que el CMS herede comportamientos visuales por defecto suele fallar. Además, el entorno de desarrollo local debe mantener exactamente la misma arquitectura de enlaces simbólicos que producción para asegurar que el código que se edita en el IDE es el que el servidor local ejecuta.

Siguiente paso o deuda: Comitear los cambios, desplegar a producción (push/pull) y ejecutar la auditoría de rendimiento final (PageSpeed).

2026-04-21 — Fix: Caché y MIME Type del Favicon

Contexto (Desafío): A pesar de haber estandarizado el formato a `.ico` y corregido las rutas, los navegadores se negaban a renderizar el nuevo favicon. Se diagnosticó un problema combinado de tipo MIME incorrecto y caché agresiva del navegador.

Hecho (Maniobra): - Se corrigió el atributo `type` de `image/ico` a `image/x-icon` (el estándar oficial). - Se añadió la cadena de consulta `?v=2` (Cache Buster) a las referencias de `favicon.ico` en `public/index.html` y `public/biblioteca/index.html`.

Detalle técnico: Los navegadores web aplican la caché más agresiva posible a los archivos `favicon.ico`. Añadir un parámetro de versión (`?v=2`) en la URL obliga al navegador a considerar la petición como un recurso nuevo, ignorando la caché local. Además, `image/x-icon` es el tipo MIME universalmente reconocido para este formato.

Motivo / criterio (Aprendizaje): Control de Caché en Assets. Siempre que se sustituya un archivo estático crítico sin cambiar su nombre, se debe forzar la invalidación de la caché local del usuario (Cache Busting) para asegurar que los cambios visuales se propaguen inmediatamente a producción.

Siguiente paso o deuda: Desplegar el parche, validar la aparición del icono y ejecutar la auditoría de rendimiento.

2026-04-21 — Fix: Estandarización definitiva del Favicon a formato .ico

Contexto (Desafío): Se había introducido manualmente el archivo físico `favicon.ico` en el servidor y actualizado la portada (`index.html`), pero sin registrar la maniobra en el repositorio. Esto generó desincronización con las rutas previas y confusión en el diagnóstico del error 404.

Hecho (Maniobra): - Se oficializa el uso de `favicon.ico` como formato estándar para el icono del sitio. - Se actualizó la referencia en `public/biblioteca/index.html` para que coincida con la portada (`href="/favicon.ico"`).

Detalle técnico: El formato `.ico` es el estándar histórico y es solicitado automáticamente por los navegadores en la raíz del dominio. Utilizar este formato físicamente en la raíz pública evita peticiones redundantes, errores 404 de rastreadores y la necesidad de mantener múltiples formatos base.

Motivo / criterio (Aprendizaje): Trazabilidad de activos (Assets). Cualquier cambio manual en los archivos estáticos o en el servidor debe ser registrado en el control de versiones. Asentar el `.ico` como estándar simplifica la arquitectura y se alinea con la web clásica.

Siguiente paso o deuda: Desplegar el HTML sincronizado y proceder a la auditoría de PageSpeed Insights.

2026-04-21 — Validación: Compilación SASS exitosa tras refactorización

Contexto (Desafío): Tras una serie de intentos, los estilos de padding del componente `.section` seguían sin aplicarse, indicando un problema profundo en la cadena de compilación de SASS.

Hecho (Maniobra): - Se confirmó que la causa raíz era una combinación de una regla `.section` duplicada y conflictiva en `_home.scss` y la omisión de la importación de la carpeta `components` en el `main.scss`. - Se eliminó la regla duplicada de `_home.scss`, se creó el componente atómico `_section.scss` y se aseguró que la cadena de importación (`@use` / `@forward`) estuviera completa. - Se recompiló el CSS con éxito, aplicando correctamente los márgenes en el navegador.

Detalle técnico: La arquitectura SASS 7-1 depende de una cadena de importación sin ambigüedades. Un componente (`_section.scss`) debe ser reexportado por su índice local (`components/_index.scss`), y ese índice debe ser importado por el punto de entrada principal (`main.scss`).

Motivo / criterio (Aprendizaje): La depuración de SASS requiere seguir la cadena de compilación desde el componente hasta el `main.scss`. Un estilo ausente en el CSS de salida casi siempre se debe a un `@forward` u `@use` omitido. La atomización de componentes previene estos conflictos.

Siguiente paso o deuda: Con la integridad visual restaurada, proceder inmediatamente con la auditoría de Core Web Vitals (Fase 6.2) en el entorno de producción.

2026-04-21 — Fix: Conexión de índice de componentes en SASS 7-1

Contexto (Desafío): Tras la refactorización del componente `.section` a su propio archivo, los estilos de padding seguían sin aplicarse en el navegador. Un análisis del `main.css` compilado reveló la ausencia total de la regla.

Hecho (Maniobra): - Se eliminó la regla `.section` duplicada que persistía en `_home.scss`. - Se verificó y aseguró que el archivo `main.scss` (punto de entrada) incluyera la directiva `@use 'components';` para importar el índice de la carpeta de componentes.

Detalle técnico: La arquitectura SASS 7-1 es explícita. Si el archivo `main.scss` no importa el índice de un directorio (`components/_index.scss`), todos los componentes declarados en ese índice (`@forward 'section'`) son ignorados por el compilador.

Motivo / criterio (Aprendizaje): Depuración de la cadena de compilación. Cuando un estilo no se aplica, el primer paso es verificar el CSS de salida. Si la regla no está presente, el fallo reside en la cadena de importación (`@use / @forward`) del preprocesador, no en el HTML o en la especificidad.

Siguiente paso o deuda: Validar la correcta visualización de los márgenes en todas las páginas y proceder con la auditoría de rendimiento.

2026-04-21 — Fix: Resolución de omisiones en índices de SASS 7-1

Contexto (Desafío): Pese a refactorizar las clases en el HTML y el SASS, los estilos (como el padding de `.section` o el grid de `.home-card`) no se aplicaban en el navegador. Se diagnosticó que el archivo `src/scss/components/_index.scss` no estaba reexportando (`@forward`) los módulos recientes.

Hecho (Maniobra): - Se actualizó el archivo `_index.scss` para incluir las directivas `@forward` de los componentes faltantes (`card` , `home` y `section`).

Detalle técnico: En la arquitectura SASS 7-1, el archivo principal (`main.scss`) solo lee los índices de cada subdirectorio. Si un archivo parcial (ej. `_section.scss`) no está declarado explícitamente en su índice local, el compilador lo ignora silenciosamente y sus reglas CSS no se inyectan en el binario final.

Motivo / criterio (Aprendizaje): Trazabilidad del compilador. Al crear un nuevo archivo `.scss` (especialmente tras aislar componentes BEM), el primer paso innegociable debe ser registrarlo en su índice correspondiente. Esto previene "fugas de estilos" o falsos positivos durante el desarrollo.

Siguiente paso o deuda: Recompilar el CSS maestro, validar los márgenes en el navegador y proceder con la auditoría de los Core Web Vitals en producción.

2026-04-21 — Fix: Desacoplamiento de padding y atomización de `.section`

Contexto (Desafío): Al aplicar la etiqueta semántica `<section>` con la clase heredada `.main--padded` , los márgenes no se renderizaban en el navegador. Se diagnosticó que la clase SASS estaba fuertemente acoplada a su etiqueta original y no funcionaba como componente transversal.

Hecho (Maniobra): - Se estableció definitivamente la clase atómica `.section` en la etiqueta `<section>` de `src/wp-theme/merci-theme/index.php` . - Se trasladó la responsabilidad del espaciado (`padding`) directamente a la clase `.section` en la arquitectura SASS, purgando el modificador obsoleto `.main--padded` .

Detalle técnico: Desacoplar las clases CSS de las etiquetas HTML específicas permite que el diseño sobreviva a las refactorizaciones semánticas (cambio de `div`s a `sections`). Ahora `.section` actúa como un Layout universal.

Motivo / criterio (Aprendizaje): Especificidad y modularidad SASS. Los modificadores BEM atados a contextos específicos rompen la reusabilidad. Al centralizar el padding en `.section` , se cumple el principio DRY (Don't Repeat

Yourself) y se garantiza coherencia absoluta en todas las vistas, sean servidas por Nginx o por el motor de PHP.

Siguiente paso o deuda: Validar los márgenes tras recompilar el SASS y proceder a la auditoría de los Core Web Vitals en producción.

2026-04-21 — Fix: Restauración de modificador de padding en sección dinámica

Contexto (Desafío): Al sustituir la clase `.main--padded` por `.section` en `index.php` para unificar estilos, se perdió el espaciado (padding) interno. En la arquitectura SASS actual, el padding de las vistas de contenido está explícitamente vinculado al modificador `.main--padded` y no a la clase estructural `.section`.

Hecho (Maniobra): - Se restauró la clase `.main--padded` en la etiqueta `<section>` del archivo `src/wp-theme/merci-theme/index.php`.

Detalle técnico: Se mantiene la mejora semántica de usar `<section>` (HTML5) introducida anteriormente, pero se le devuelve la clase CSS que controla físicamente los márgenes (`4rem 2rem`) en el diseño base, asegurando que se visualice correctamente en `localhost`.

Motivo / criterio (Aprendizaje): Conocimiento del estado del SASS. Reemplazar clases asumiendo comportamientos genéricos (como que `.section` tiene padding universal) sin verificar las reglas compiladas genera regresiones visuales. El modificador `.main--padded` debe mantenerse hasta que se decida refactorizar el SASS globalmente.

Siguiente paso o deuda: Validar la vista en local, comitear y auditar los Core Web Vitals en producción.

2026-04-21 — Atomización de estilos en secciones dinámicas

Contexto (Desafío): Los textos de la capa dinámica (WordPress) aparecían pegados al borde izquierdo sin margen. Esto se debía a que las plantillas usaban la clase modificadora antigua `.main--padded` en lugar de heredar los estilos atómicos estructurales de la portada.

Hecho (Maniobra): - Se reemplazó la clase `.main--padded` por la clase atómica `.section` en `src/wp-theme/merci-theme/index.php`. - (Nota: Esta misma convención atómica debe replicarse en las vistas estáticas como la Biblioteca).

Detalle técnico: Al igual que se hizo con `.hero`, el uso de `.section` centraliza el padding responsivo y la alineación. Cualquier ajuste en SASS sobre el componente `_section.scss` se propagará automáticamente al contenido dinámico.

Motivo / criterio (Aprendizaje): Principio DRY (Don't Repeat Yourself). La atomización evita incoherencias visuales (como saltos de márgenes entre páginas) y elimina la necesidad de mantener modificadores CSS redundantes para el mismo propósito estructural.

Siguiente paso o deuda: Replicar esta clase `.section` en las páginas estáticas que lo requieran y validar los Core Web Vitals en producción.

2026-04-21 — Refactorización semántica en plantillas dinámicas (HTML5)

Contexto (Desafío): Se detectó una inconsistencia semántica entre la portada estática y las vistas dinámicas de WordPress. Mientras la portada utiliza etiquetas `<section>` para agrupar bloques temáticos de contenido, el archivo `index.php` del CMS envolvía los listados de artículos en un `<div>` genérico (`<div class="main--padded">`).

Hecho (Maniobra): - Se reemplazó el contenedor `<div>` por una etiqueta `<section>` en `src/wp-theme/merci-theme/index.php`.

Detalle técnico: Las etiquetas `<section>` introducen un nuevo nodo en el "outline" (esquema) del documento HTML5, lo cual es interpretado correctamente por tecnologías de asistencia y crawlers (SEO) para identificar bloques de contenido autónomos (como el loop de posts o productos).

Motivo / criterio (Aprendizaje): Coherencia arquitectónica y accesibilidad estricta. Un `<div>` carece de valor semántico. Envolver el contenido dinámico dentro de un `<section>` respeta la política de semántica HTML5 del proyecto y asegura

que la calidad técnica no se degrade al transicionar del núcleo estático al dinámico.

Siguiente paso o deuda: Desplegar la corrección estructural y continuar con la medición del rendimiento en producción.

2026-04-21 — Fix: Resolución de enrutamiento de assets en producción

Contexto (Desafío): Tras el despliegue, los assets (como el logotipo) devolvían un error 404. La causa era que el `Document Root` de Nginx apuntaba a `/public`, pero la carpeta `/assets` residía fuera de ella, haciéndola inaccesible para el servidor web.

Hecho (Maniobra): - Se ha creado un tercer enlace simbólico para proyectar la carpeta `/assets` dentro de `/public`. - Se ha actualizado el `deployment-playbook.md` para incluir este nuevo paso.

Detalle técnico: El comando `ln -s /home/mercedev-php/htdocs/mercedev.es/assets /home/mercedev-php/htdocs/mercedev.es/public/assets` resuelve el problema de rutas sin necesidad de reestructurar el repositorio ni de añadir directivas `alias` complejas en la configuración de Nginx de CloudPanel.

Motivo / criterio (Aprendizaje): Consistencia arquitectónica. El uso de enlaces simbólicos es la estrategia unificada de este proyecto para conectar componentes desacoplados. Cualquier recurso que deba ser servido por la web debe residir (o aparentar residir) bajo el `Document Root`.

Siguiente paso o deuda: Validar la correcta visualización del logotipo en la portada y en el blog, y proceder con la auditoría de rendimiento de la Fase 6.2.

2026-04-21 — Docs: Actualización del Deployment Playbook para CloudPanel

Contexto (Desafío): El manual de despliegue (`docs/deployment-playbook.md`) poseía instrucciones genéricas de enrutamiento y carecía del paso del puente del

Child Theme. Era vital alinear el "Runbook" con la ejecución real realizada en el servidor de producción.

Hecho (Maniobra): - Se precisaron las rutas absolutas (`mercedev-php` , `mercedev.es`) en la Fase 3 y se incluyó el comando del segundo enlace simbólico para el Child Theme. - Se refactorizó la Fase 4 para reflejar el proceso nativo de CloudPanel: modificación del *Document Root* vía UI, edición específica del VHost en el bloque del puerto 8080 y la activación de Enlaces Permanentes.

Detalle técnico: Detallar que el enrutamiento de Nginx en CloudPanel se inyecta en el bloque `server` que escucha en el puerto `8080` (procesamiento PHP/Varnish) previene romper la configuración de los servidores públicos de los puertos 80 y 443.

Motivo / criterio (Aprendizaje): Reproducibilidad. Un playbook debe ser un guión ejecutable sin ambigüedades. Incorporar el aprovisionamiento post-instalación (Enlaces permanentes) en el manual asegura que la base de datos y Nginx queden sincronizados en futuros despliegues o reconstrucciones de la infraestructura.

Siguiente paso o deuda: Iniciar la Fase 6.2 (Auditoría de rendimiento y accesibilidad) con herramientas externas para validar los Web Vitals.

2026-04-21 — Docs: Refactorización de documento de integración para CloudPanel

Contexto (Desafío): El documento `docs/integracion-wordpress.md` reflejaba la configuración del entorno local (LEMP nativo en `/var/www/`). Tras el despliegue en producción, existía una deuda documental ("Drift" o deriva de configuración) respecto a la arquitectura real en CloudPanel.

Hecho (Maniobra): - Se actualizaron las rutas absolutas a `/home/mercedev-php/htdocs/` . - Se incluyó el segundo enlace simbólico destinado al *Child Theme*. - Se reemplazó el Virtual Host completo por la metodología de CloudPanel (modificación de `Document Root` vía UI e inyección de reglas `location` en el bloque 8080).

Detalle técnico: Adaptar la documentación a las variables `{{root}}` de CloudPanel es vital para que las reglas inyectadas en el VHost no entren en conflicto con el IaaS (Infrastructure as a Service - Infraestructura como Servicio).

Motivo / criterio (Aprendizaje): Single Source of Truth (Única Fuente de Verdad). La documentación arquitectónica no puede ser un artefacto teórico fosilizado. Si la infraestructura en producción se adapta a un panel de control, los documentos del repositorio deben actualizarse para que cualquier réplica futura sea exacta.

Siguiente paso o deuda: Iniciar la Fase 6.2 (Auditoría de rendimiento y accesibilidad) con herramientas externas para validar los Web Vitals del entorno real.

2026-04-21 — Docs: Actualización de la arquitectura de integración de WordPress

Contexto (Desafío): El documento `docs/integracion-wordpress.md` contenía el plan teórico de despliegue. Tras la implementación exitosa en producción (CloudPanel), era imperativo actualizar la documentación para que reflejara la arquitectura real y los comandos ejecutados.

Hecho (Maniobra): - Se ha reescrito por completo el documento `docs/integracion-wordpress.md`. - La nueva versión detalla el proceso específico para un entorno gestionado con CloudPanel.

Detalle técnico: El documento ahora incluye la arquitectura de "carpetas hermanas", la creación de los dos enlaces simbólicos (para `/blog` y para el `merci-theme`), y la configuración VHost adaptada al motor de plantillas de CloudPanel (modificación del Document Root vía UI y del enrutador PHP en el bloque del puerto 8080).

Motivo / criterio (Aprendizaje): La documentación debe ser un reflejo fiel de la infraestructura en producción, no un artefacto teórico. Este documento actualizado sirve ahora como un "Runbook" fiable para futuras reinstalaciones o para la depuración de la arquitectura híbrida.

Siguiente paso o deuda: Iniciar la Fase 6.2 (Auditoría de rendimiento y accesibilidad) para medir los Core Web Vitals en el entorno de producción real.

2026-04-21 — Fase 4.2: Enlace simbólico del Child Theme en producción

Contexto (Desafío): Tras inicializar la base de datos de producción, el "Merci Theme" no aparecía en el panel de WordPress porque el código reside en el repositorio Git inmutable (`mercedev.es/src/...`) y el CMS está enjaulado en un directorio hermano (`wordpress/`).

Hecho (Maniobra): - Se trazó un enlace simbólico físico (`ln -s`) desde el código del tema en el repositorio hacia el directorio `wp-content/themes/merci-theme` de la instalación asilada de WordPress. - Se verificó y activó el tema en el panel de administración en producción.

Detalle técnico: Este puente lógico bidireccional garantiza que cualquier actualización de diseño (CSS/PHP) que ingrese vía `git pull` se refleje inmediatamente en el CMS sin necesidad de mover o copiar archivos manualmente.

Motivo / criterio (Aprendizaje): Aislamiento con automatización cero-fricción. El motor PHP de WordPress y los plugins de terceros viven fuera del control de versiones, pero nuestra capa visual a medida (Child Theme) permanece estrictamente gobernada por Git, respetando la filosofía "Single Source of Truth".

Siguiente paso o deuda: Resolver la deuda técnica visual (rutas de assets y menú rotos en el frontend dinámico) derivada de la diferencia de la URI base entre la raíz estática y la subruta `/blog` .

2026-04-21 — Aprovisionamiento de base de datos y separación Código/Estado

Contexto (Desafío): Tras configurar el enrutamiento Nginx, se requería inicializar el CMS en producción. Se constató la necesidad de clarificar por qué es obligatorio repetir la configuración web (creación de admin, etc.) que ya se hizo en local. Asimismo, se observó que el Child Theme "Merci" no estaba disponible para activación en el panel de WordPress.

Hecho (Maniobra): - Se completó la instalación web (aprovisionamiento) alimentando la nueva base de datos `mercedev_wp_prod` . - Se sincronizó el

enrutamiento configurando los Enlaces Permanentes a "Nombre de la entrada". - Se documentó la lección arquitectónica sobre la asimetría de Git: transporta código inmutable, no estado.

Detalle técnico: Un CMS desplegado en una nueva infraestructura nace en blanco. La configuración de Permalinks (`/%postname%/`) es crítica para que el proxy inverso de Nginx (`/blog/index.php?$args`) interprete correctamente la URI dinámica. La ausencia del Child Theme se debe a que este reside en el repositorio inmutable (`src/wp-theme/merci-theme`) y requiere ser enlazado explícitamente a la instalación asilada del CMS.

Motivo / criterio (Aprendizaje): Principio de Separación de Responsabilidades. La base de datos nunca se sube mediante control de versiones para evitar colisiones de URLs (`localhost` vs producción), credenciales débiles y fugas de seguridad (Shift-Left). Mantener ambas piezas separadas obliga a un aprovisionamiento seguro desde cero.

Siguiente paso o deuda: Trazar el enlace simbólico del Child Theme desde el repositorio Git hacia el directorio `wp-content/themes/` del WordPress aislado y activarlo.

2026-04-20 — Fix: Adaptación de enrutamiento Nginx a plantillas de CloudPanel

Contexto (Desafío): Al configurar el enrutamiento Nginx (VHost) para separar la capa estática de la dinámica, se detectó que CloudPanel utiliza un motor de plantillas (variable `{{root}}`). Reemplazar estas variables manualmente por rutas absolutas en el editor de texto amenazaba con romper la integración del panel.

Hecho (Maniobra): - Se actualizó el *Document Root* desde la interfaz visual de CloudPanel (pestaña Settings) añadiendo `/public` al final, lo que propagó el cambio de forma segura a todas las variables `{{root}}` . - En la configuración VHost (pestaña VHost), dentro del bloque `server` del puerto 8080 (procesamiento interno de PHP), se eliminó la regla global `try_files` y se aislaron los tráficos usando dos bloques `location` dedicados (`/` y `/blog`).

Detalle técnico: El bloque estático `location /` devuelve un error 404 de coste cero si el archivo no existe, protegiendo la raíz de ejecución de scripts no autorizados. El bloque dinámico `location /blog` atrapa el tráfico hacia el CMS aislado pasándolo por `/blog/index.php?$args`.

Motivo / criterio (Aprendizaje): Respetar la capa de abstracción del proveedor (IaaS/Panel). Forzar modificaciones estáticas sobre un entorno gobernado por plantillas dinámicas genera deuda técnica y fragilidad ante actualizaciones del sistema. Separar el ajuste del "Document Root" (vía UI) del "Enrutador PHP" (vía VHost) es la práctica DevOps correcta.

Siguiente paso o deuda: Validar en el navegador la carga de la página estática y la aparición de la instalación de WordPress en la ruta dinámica.

2026-04-20 — Fase 3 de Despliegue: Aislamiento y Hardening de WordPress en Producción

Contexto: Era imperativo desplegar el CMS en producción sin vulnerar la integridad del núcleo estático recién clonado.

Hecho: - Se creó la base de datos `mercedev_wp_prod` aislada en CloudPanel. - Se descargó y extrajo la última versión de WordPress en un directorio hermano (`~/htdocs/wordpress`). - Se blindó el `wp-config.php` inyectando Salts criptográficos oficiales y aplicando permisos de solo lectura para el dueño (`chmod 600`). - Se estableció el puente lógico creando un enlace simbólico desde `~/htdocs/mercedev.es/public/blog` hacia el directorio aislado de WordPress.

Detalle técnico: La configuración manual del `wp-config.php` y la restricción estricta de permisos de sistema operativo (CHOWN/CHMOD) evitan depender del instalador web de WordPress, bloqueando cualquier posible vector de ataque o ejecución no autorizada durante el provisionamiento (Shift-Left Security).

Motivo / criterio: Aislar los riesgos del entorno dinámico. Si WordPress sufre una vulnerabilidad de escalada a través de un plugin en el futuro, el atacante se encontrará encapsulado en un directorio externo sin permisos para modificar el código fuente inmutable (HTML/CSS/JS) de la landing principal (`mercedev.es/public`).

Siguiente paso o deuda: Configurar el VHost (Virtual Host) de Nginx en CloudPanel para orquestar el enrutamiento híbrido.

2026-04-17 — Verificación de artefactos estáticos y refactorización de portada

Contexto: Era necesario cumplir con el hito del Roadmap "Verificar artefactos finales del núcleo estático antes del deploy". Durante la revisión, se identificó que la lista HTML de la sección de características rompía el diseño Boxed Layout.

Hecho: - Se transformó la lista de características (`<ul>`) en una cuadrícula de tarjetas (`.home-grid` > `.home-card`) en `public/index.html` . - Se marcó el hito de verificación de artefactos como completado en `README.md` .

Detalle técnico: Al homogeneizar la estructura de la portada utilizando los componentes BEM preexistentes, se erradican los problemas de desbordamiento por el `padding` nativo de las listas HTML y se consolida un diseño de "Landing Page" robusto.

Motivo / criterio: Rigor técnico y UI coherente. Antes de subir el código al servidor remoto (CloudPanel), el núcleo estático debe estar visual y semánticamente impecable, garantizando que el diseño validado en local sea exactamente el que se despliega.

Siguiente paso o deuda: Continuar con la configuración de la base de datos e instalación de WordPress en el entorno de producción.

2026-04-17 — Fix: Regeneración de Deploy Key SSH para el usuario correcto

Contexto (Desafío): Al ejecutar el `git clone` en el servidor de producción, GitHub devolvió un error `Permission denied (publickey)` , bloqueando la descarga del repositorio.

Hecho (Maniobra): - Se generó un nuevo par de claves SSH (`ssh-keygen -t ed25519`) bajo el usuario `mercedev-php` . - Se sustituyó la Deploy Key obsoleta en los ajustes del repositorio de GitHub por la nueva clave pública.

Detalle técnico: Las claves SSH están vinculadas estrictamente al directorio `$HOME/.ssh/` del usuario que las ejecuta. La clave generada inicialmente pertenecía al usuario incorrecto (`mercedev`), por lo que el proceso de Git bajo `mercedev-php` carecía de credenciales válidas para la autenticación criptográfica contra GitHub.

Motivo / criterio (Aprendizaje): Autenticación estricta de Linux. En arquitecturas IaaS y paneles como CloudPanel, la identidad del proceso (quién ejecuta el comando) define qué anillo de claves se utiliza. Emparejar correctamente el usuario del sistema de archivos web con su propia clave SSH es vital para un despliegue CI/CD sin fricciones.

Siguiente paso o deuda: Confirmar la clonación exitosa del código e iniciar la Fase 3 (Aislamiento de WordPress en CloudPanel).

2026-04-17 — Refinamiento final de UI: Boxed Layout y alineación

Contexto: Era necesario pulir los detalles visuales finales antes de dar por cerrado el diseño: alinear el menú y el logotipo por su base inferior, separar los títulos del header y limitar el ancho de la web para evitar que el contenido desbordara los márgenes del menú en pantallas ultrapanorámicas.

Hecho: - Se ajustó `.header` con `align-items: flex-end` en `_header.scss`.
- Se aplicó `max-width: 1200px` y `margin: 0 auto` directamente a la etiqueta `body` en `_reset.scss`. - Se aumentó el valor de la variable `$spacing-xl` a `6rem` en `_variables.scss` para dar más respiro vertical a las cabeceras.

Detalle técnico: Limitar el ancho máximo en el `body` (Boxed Layout) es la estrategia más limpia para sincronizar los ejes verticales de `.header`, `.main` y `.footer` sin necesidad de envolver todo en contenedores `.container` adicionales, reduciendo drásticamente el peso del DOM.

Motivo / criterio: Equilibrio visual. Alinear los elementos por su línea base (baseline/flex-end) y dar aire a las secciones respirables mejora la legibilidad y la jerarquía de la página. Se confirma formalmente que el uso de Vanilla JS para el menú móvil respeta íntegramente la norma de "Cero dependencias externas" de la arquitectura.

Siguiente paso o deuda: Validar los ajustes estéticos y ejecutar el despliegue del Boilerplate a producción.

2026-04-17 — Erradicación total de estilos en línea (Inline CSS)

Contexto: Se detectaron estilos en línea residuales (`style="padding: 4rem 2rem;"` y `style="margin-bottom: 3rem;"`) en las vistas estáticas (Biblioteca, Contacto) y en la plantilla dinámica de WordPress, lo cual vulneraba la metodología BEM y la filosofía atómica del proyecto.

Hecho: - Se crearon los modificadores BEM `.main--padded` y `.home-grid--spaced` en `src/scss/pages/_home.scss` . - Se eliminaron todos los atributos `style` residuales de `public/biblioteca/index.html` , `public/contacto/index.html` y `src/wp-theme/merci-theme/index.php` .

Detalle técnico: Se estandarizó la aplicación del espaciado interno asignando la clase `.main--padded` al contenedor principal `<main>` para garantizar consistencia estructural entre las vistas estáticas servidas por Nginx y las vistas dinámicas servidas por WordPress. Las variables globales `$spacing-xl` y `$spacing-lg` asumen el control de la separación.

Motivo / criterio: Arquitectura limpia y escalabilidad. La purga de atributos `style` asegura que cualquier modificación futura en los márgenes de la interfaz se resuelva editando un único archivo SASS, respetando la filosofía "Single Source of Truth" (Única Fuente de Verdad).

Siguiente paso o deuda: Compilar, verificar visualmente el diseño y ejecutar el paso final de la Fase 6 (Despliegue en CloudPanel).

2026-04-17 — Refactorización atómica: Eliminación de estilos en línea (Inline CSS)

Contexto: Las cabeceras de presentación en las páginas estáticas y dinámicas (Biblioteca, Contacto, Tienda, Blog) usaban el atributo `style="color: #ea580c;"` inyectado directamente en el HTML, violando la separación de responsabilidades y la metodología BEM.

Hecho: - Se creó el modificador BEM `.home-card__title--highlight` en `src/scss/pages/_home.scss` vinculado a `$color-primary`. - Se eliminaron todos los atributos `style` en línea de `index.html` (Biblioteca, Contacto) y de `index.php` (Child Theme).

Detalle técnico: Al añadir un modificador BEM, se delega el control absoluto de la interfaz a la capa SASS. Cualquier cambio futuro en `$color-primary` (ubicado en `_variables.scss`) se propagará ahora correctamente sin necesidad de buscar código HTML "hardcoded" a lo largo de los archivos.

Motivo / criterio: Arquitectura limpia (Clean Code). Los estilos en línea son un antipatrón perjudicial para el mantenimiento a escala. La filosofía atómica y BEM exige que las variaciones visuales se gestionen estrictamente mediante modificadores CSS en el sistema de diseño central.

Siguiente paso o deuda: Compilar los estilos y proceder con la Fase 6.

2026-04-17 — Actualización de paleta de colores (Naranja oscuro)

Contexto: Se decidió reemplazar el color de acento primario (azul) por un naranja oscuro en todo el Boilerplate para ajustarse mejor a la identidad visual deseada.

Hecho: - Se modificó la variable `$color-primary` a `#ea580c` en `_variables.scss`. - Se reemplazaron los valores hexadecimales hardcoded azules por la variable `$color-primary` en los componentes `_card.scss` y `_home.scss`. - Se actualizaron los estilos en línea de los títulos en `public/biblioteca/index.html`, `public/contacto/index.html` y `src/wp-theme/merci-theme/index.php`.

Detalle técnico: Al utilizar la variable `$color-primary`, el compilador SASS propaga automáticamente el nuevo color naranja a todos los enlaces (`a:hover`), bordes de tarjetas y efectos visuales, asegurando cohesión en el diseño. Se refactorizaron estilos fijos para mejorar la escalabilidad del Boilerplate.

Motivo / criterio: Consistencia y escalabilidad UI. Mantener colores "quemados" (hardcoded) en HTML o en módulos SASS específicos dificulta el mantenimiento. Centralizar el color de acento en una variable global respeta la arquitectura SASS 7-1.

Siguiente paso o deuda: Compilar los estilos con `merci-watch` , verificar la consistencia visual en el navegador y continuar con el despliegue en producción.

2026-04-17 — Eliminación automática de contenido por defecto de WP (laC)

Contexto: Las instalaciones limpias de WordPress inyectan contenido de relleno ("¡Hola, mundo!" y "Página de ejemplo") en la base de datos, lo cual restaba profesionalidad a la presentación visual del Boilerplate recién desplegado.

Hecho: - Se amplió la función `merci_boilerplate_auto_setup` en `functions.php` .

Detalle técnico: Se utilizaron las funciones `get_post()` y `wp_delete_post(id, true)` para buscar los IDs 1 y 2. Si sus *slugs* coinciden con los predeterminados (en español o inglés), se fuerza su borrado permanente (bypass de la papelera) directamente desde el código.

Motivo / criterio: Infraestructura como Código (laC - Infrastructure as Code). Un Boilerplate verdaderamente automatizado debe autolimpiarse tras su despliegue inicial. Obligar al desarrollador a acceder al CMS para borrar contenido basura manualmente rompe la filosofía de automatización y 0 fricción.

Siguiente paso o deuda: Comprobar la desaparición del artículo en el frontend local, realizar el commit final e iniciar la Fase 6.

2026-04-17 — Unificación tipográfica y maquetación de vistas dinámicas

Contexto: Existía una ligera discrepancia visual entre las páginas estáticas (Biblioteca, Contacto) y las páginas dinámicas de WordPress (Art de Coté, Tienda) en cuanto a márgenes, fondos de tarjeta y coloración de enlaces.

Hecho: - Se actualizó `<main>` en `index.php` para igualar el padding de las vistas estáticas (`4rem 2rem`). - Se alinearon las propiedades de `_card.scss` para ser idénticas a `_home.scss` (fondo transparente, padding ampliado, hover azul). - Se forzó el color oscuro (`$color-text-base`) en encabezados globales y enlaces de menú (`.nav__link`).

Detalle técnico: Se utilizó `color: inherit` dentro de las etiquetas de encabezado (`h1-h6`) en `_typography.scss` para asegurar que los enlaces dinámicos de título generados por WordPress (`<a href="...">`) sobrescriban el azul por defecto y adopten el negro base.

Motivo / criterio: Coherencia de Interfaz (UI). Un Boilerplate profesional no debe presentar saltos de diseño entre sus distintas vistas. Homogeneizar contenedores y tipografía garantiza una experiencia de usuario (UX) fluida, independientemente de si la ruta es resuelta por Nginx directo o por el motor de PHP.

Siguiente paso o deuda: Confirmar la estética general en el navegador, ejecutar el commit y proceder con el despliegue al entorno de producción (Fase 6).

2026-04-17 — Fix: Variables obsoletas en CSS Reset

Contexto (Desafío): Al compilar el SASS tras la migración al Light Mode, el compilador devolvía un error de variables no definidas en `_reset.scss` , deteniendo la ejecución de `merci-watcher.py` .

Hecho (Maniobra): - Se actualizaron las variables en `src/scss/base/_reset.scss` a `$color-bg-base` y `$color-text-base` .

Detalle técnico: Se sustituyeron las antiguas variables del modo oscuro que habían quedado huérfanas tras la refactorización de `_variables.scss` en la sesión anterior.

Motivo / criterio (Aprendizaje): En refactorizaciones globales de sistemas de diseño (Design Systems), es común que algún archivo base mantenga dependencias obsoletas. El compilador SASS actúa de forma estricta, protegiendo la integridad del CSS final e impidiendo que llegue código roto a producción.

Siguiente paso o deuda: Verificar que el compilador finalice con éxito y volver al enfoque de despliegue en producción (Fase 6).

2026-04-17 — Unificación de UI a "Light Mode" (Modo Claro)

Contexto: La paleta de colores oscura limitaba la versatilidad de la plantilla. Se requería unificar la estética de las 5 páginas principales bajo un esquema "Light Mode" limpio y profesional.

Hecho: - Se refactorizaron las variables en `_variables.scss` renombrando referencias de Dark a Base (`$color-bg-base: #ffffff`). - Se eliminaron los colores quemados (hardcoded) en componentes como `_card.scss` , `_hero.scss` y `_home.scss` sustituyéndolos por variables dinámicas. - Se ajustó la estructura flex en `_header.scss` para alinear el logotipo a la izquierda y el menú a la derecha.

Detalle técnico: El uso de módulos `@use '../abstracts' as *` permitió inyectar el nuevo esquema a lo largo de toda la arquitectura SASS 7-1. Los bordes divisorios se mantuvieron utilizando funciones de canal alfa (`rgba`) sobre el nuevo texto oscuro, asegurando contraste accesible.

Motivo / criterio: Escalabilidad de diseño. Un Boilerplate debe proveer un lienzo neutral y altamente legible por defecto. Las variables semánticas (`-base` en lugar de `-dark/-light`) permiten que futuros usuarios de la plantilla cambien todo el aspecto de la web modificando solo dos líneas de código SASS.

Siguiente paso o deuda: Validar la nueva interfaz en el navegador y ejecutar el commit.

2026-04-17 — Configuración de alias de terminal para Merci Watcher

Contexto: Para mantener la agilidad del flujo de trabajo local y seguir la convención del resto de herramientas del sistema Merci, se requería un comando rápido para invocar el vigilante de SASS.

Hecho: - Se añadió el alias `merci-watch` a la configuración de la terminal (`~/.zshrc`).

Detalle técnico: El alias ejecuta `python3 $MERCY_ROOT/scripts/merci/merci-watcher.py` , aprovechando la variable de entorno global del proyecto definida en configuraciones anteriores para que funcione desde cualquier directorio.

Motivo / criterio: Consistencia operativa y reducción de fricción (DX). Abstraer la ruta del script en un comando corto fomenta el uso constante del compilador en tiempo real durante las sesiones de diseño visual.

Siguiente paso o deuda: Validar el alias en la terminal, realizar el commit atómico y proceder con el ajuste de variables (Light Mode) en SASS.

2026-04-17 — Restauración del vigilante SASS (merci-watcher.py)

Contexto: Al igual que ocurrió con el compilador, el script `merci-watcher.py` no sobrevivió a la limpieza y eliminación de la rama de diseño, perdiéndose la automatización de la compilación en tiempo real.

Hecho: - Se ha restaurado el script `scripts/merci/merci-watcher.py`.

Detalle técnico: El script se ha recreado con su lógica original utilizando `path.stat().st_mtime` para monitorizar la carpeta `src/scss/` e invocar a `merci-styles.py` mediante `subprocess.run()`.

Motivo / criterio: Resiliencia de la infraestructura local. Recuperar las herramientas de DX (Developer Experience - Experiencia del Desarrollador) es imperativo para mantener la agilidad del Boilerplate. Si una herramienta se pierde en el control de versiones por falta de trackeo, la documentación debe permitir su reconstrucción inmediata.

Siguiente paso o deuda: Reanudar la refactorización de variables a modo claro (Light Mode) en el SASS.

2026-04-17 — Fix: Resolución de advertencias de deprecación en Dart Sass

Contexto (Desafío): Al compilar los estilos SASS, el compilador emitía advertencias (Deprecation Warnings) indicando que las funciones globales de color (`scale-color`) serán eliminadas en Dart Sass 3.0.0.

Hecho (Maniobra): - Se migró el uso de `scale-color` al módulo moderno `color.scale`. - Se añadió la importación `@use 'sass:color';` en los archivos `_typography.scss`, `_footer.scss` y `_hero.scss`.

Detalle técnico: Dart Sass está abandonando las funciones globales en favor de un sistema de módulos integrados (built-in modules). El uso de `color.scale()` previene que el compilador rompa la compilación en futuras actualizaciones del binario standalone de SASS.

Motivo / criterio (Aprendizaje): Mantenibilidad a largo plazo. Un Boilerplate no debe generar advertencias (warnings) de compilación "out of the box". Atender las deprecaciones a tiempo es una práctica fundamental de higiene técnica.

Siguiente paso o deuda: Migrar el esquema de colores a variables agnósticas (Light Mode) en los archivos `abstracts` y eliminar colores quemados (hardcoded).

2026-04-17 — Corrección de usuario y ruta web en CloudPanel

Contexto (Desafío): Al intentar acceder al directorio del sitio vía SSH para clonar el repositorio, la navegación fallaba debido a que la ruta teórica no coincidía con la generada por el panel de control.

Hecho (Maniobra): - Se verificó la ruta absoluta real desde la interfaz web de CloudPanel, resultando ser `/home/mercedev-php/htdocs/mercedev.es`. - Se actualizaron las referencias en `docs/deployment-playbook.md` para utilizar rutas absolutas explícitas.

Detalle técnico: CloudPanel genera automáticamente usuarios de sistema anexando sufijos (como `-php`) dependiendo del tipo de aplicación seleccionada (PHP Site) para evitar colisiones de nombres. La asunción de que el usuario del sitio era exactamente el ingresado en el formulario causó el error de navegación.

Motivo / criterio (Aprendizaje): Verificación empírica. La interfaz de gestión (GUI) del panel expone la configuración final del servidor (Document Root absoluto). Es prioritario confiar en los datos de la plataforma IaaS o Panel de Control por encima de las asunciones teóricas al interactuar con el CLI.

Siguiente paso o deuda: Iniciar sesión como `mercedev-php` y ejecutar `git clone` en la carpeta web correcta (Fase 2).

2026-04-17 — Corrección de rutas absolutas a relativas (Home) en manual de despliegue

Contexto (Desafío): Al intentar navegar y listar archivos (`ls`) en el servidor de producción bajo el usuario del sitio de CloudPanel, el sistema devolvía "Permission denied" debido a una confusión en las rutas documentadas en el manual.

Hecho (Maniobra): - Se actualizaron las rutas en `docs/deployment-playbook.md` cambiando `/htdocs/...` por `~/htdocs/...`.

Detalle técnico: CloudPanel aísla (chroot/jail) a los usuarios de los sitios. Intentar acceder a `/htdocs` desde la raíz absoluta del servidor de Ubuntu interfiere con los permisos de `root`. La ruta correcta del directorio web reside dentro del `$HOME` del usuario (`~` que se traduce en `/home/usuario/htdocs/ dominio.com`).

Motivo / criterio (Aprendizaje): Seguridad de sistema operativo (Linux). Los aislamientos en jaulas evitan que un sitio web comprometido acceda a los archivos de otro sitio en el mismo servidor. Respetar el uso del directorio `$HOME` (`~`) es vital en arquitecturas multi-tenant o paneles de control.

Siguiente paso o deuda: Completar la clonación del repositorio en la carpeta del sitio.

2026-04-17 — Fix: Preservación de transparencia (Canal Alpha) en Merci Optimizer

Contexto (Desafío): Al procesar imágenes originales con fondos transparentes (ej. logos en formato PNG), la salida WebP resultante inyectaba un fondo opaco, rompiendo el diseño de la UI en el Frontend.

Hecho (Maniobra): - Se añadió una validación del espacio de color (`img.mode`) en `merci-optimizer.py` antes del proceso de guardado y redimensionado. - Se actualizó el archivo de pruebas `test_optimizer.py` para mockear el objeto resultante de la conversión.

Detalle técnico: Las imágenes guardadas en paleta indexada (Modo `P`) o con alpha explícito (`RGBA` , `LA`) pierden sus propiedades de transparencia al ser procesadas directamente a WebP por Pillow si no se convierten antes a un modo compatible. El bloque `img = img.convert('RGBA')` soluciona esto en memoria, preservando el canal de opacidad para el binario final.

Motivo / criterio (Aprendizaje): Fiabilidad de la herramienta local. Una herramienta de optimización multimedia no puede degradar el aspecto visual (UX/

UI) a expensas del tamaño. Gestionar los modos de color garantiza que las imágenes transparentes se empaqueten correctamente en WebP.

Siguiente paso o deuda: Re-ejecutar el optimizador para recuperar el logotipo sin fondo y continuar el despliegue con CloudPanel.

2026-04-17 — Inclusión de la Fase 0 (DNS e Infraestructura) en manual de despliegue

Contexto: El manual de despliegue (`deployment-playbook.md`) asumía infraestructura preexistente. Al tratarse de un "Boilerplate", se requería explicar el proceso conceptual desde la compra del dominio para guiar a usuarios desde cero.

Hecho: - Se refactorizó `docs/deployment-playbook.md` incluyendo la nueva "Fase 0: Fundamentos y Preparación de Infraestructura". - Se reescribió el documento completo utilizando voz impersonal y verbos en infinitivo.

Detalle técnico: Se incluyeron las instrucciones explícitas para separar el Registro del Dominio del proveedor IaaS (Infrastructure as a Service), junto con la directriz de modificar el registro DNS tipo 'A'. Se expandieron acrónimos clave (VPS, SSL, SSH) en su primera aparición.

Motivo / criterio: Completitud pedagógica, alineada a las `instrucciones.md` . Un Boilerplate no solo provee código, sino conocimiento operativo. Guiar sobre los DNS (Domain Name System) desmitifica el proceso de paso a producción y previene confusiones habituales de enrutamiento temprano.

Siguiente paso o deuda: Ejecutar los pasos documentados del manual sobre el entorno de producción.

2026-04-17 — Adopción de CloudPanel para la administración de producción

Contexto: Se requiere simplificar la administración a largo plazo del servidor de producción (certificados SSL, bases de datos, versiones de PHP) sin sacrificar la arquitectura LEMP de alto rendimiento diseñada en local.

Hecho: - Se actualizó el `docs/deployment-playbook.md` para reemplazar el aprovisionamiento manual por la instalación de CloudPanel.

Detalle técnico: CloudPanel es un panel de control server-level optimizado para Nginx, PHP-FPM y MariaDB. Dado que instala su propia pila hiper-optimizada, requiere un sistema operativo Ubuntu completamente limpio. La configuración de enrutamiento inverso (WordPress aislado) se aplicará a través de la interfaz VHost nativa del panel.

Motivo / criterio: Eficiencia operativa (DevOps). Automatizar la gestión del servidor reduce la fricción de mantenimiento. CloudPanel se alinea perfectamente con la arquitectura del Boilerplate al utilizar Nginx de forma nativa, permitiendo inyectar reglas de proxy inverso y enlaces simbólicos sin bloqueos.

Siguiente paso o deuda: Destruir y recrear el Droplet (para garantizar un sistema 100% limpio) e iniciar la instalación del panel.

2026-04-17 — Diagnóstico de enrutamiento DNS y evaluación de proveedores IaaS

Contexto (Desafío): Pérdida de conectividad con el dominio `mercedev.es` tras el reaprovisionamiento del servidor, sumado al deseo de explorar alternativas a DigitalOcean para el alojamiento del entorno de producción.

Hecho (Maniobra): - Se diagnosticó una desincronización en la Zona DNS: el Registro 'A' del dominio apuntaba a la IP del Droplet destruido (Singapur) en lugar del nuevo nodo europeo. - Se propusieron proveedores IaaS (Infrastructure as a Service) alternativos (Hetzner, Linode, Vultr) compatibles con el `deployment-playbook.md`.

Detalle técnico: Al destruir y recrear máquinas virtuales, la dirección IPv4 pública cambia. Es imperativo actualizar el registro 'A' (y 'AAAA' si se usa IPv6) en el registrador del dominio y esperar el tiempo de propagación (TTL). La arquitectura basada en Ubuntu + LEMP nativo garantiza cero *vendor lock-in*.

Motivo / criterio (Aprendizaje): Separación entre Dominio (Registrador) e Infraestructura (Hosting). La resolución DNS es independiente del estado del

servidor. Elegir un proveedor IaaS "Bare Metal" o VPS puro (como Hetzner) permite aplicar la Fase 6.1 de despliegue de forma estandarizada y universal.

Siguiente paso o deuda: Actualizar la IP en los registros DNS, elegir el proveedor VPS definitivo y ejecutar el aprovisionamiento LEMP de la Fase 6.

2026-04-17 — Diagnóstico de latencia y reaprovisionamiento de infraestructura

Contexto (Desafío): Al iniciar la conexión al servidor de producción (Droplet), se detectó una latencia inaceptable y constante de ~290 ms mediante un test de `ping`, lo que imposibilitaba un trabajo fluido por SSH y amenazaba el rendimiento final del sitio.

Hecho (Maniobra): - Se diagnosticó un error en la elección geográfica del Datacenter durante la creación del Droplet (posiblemente ubicado en Asia/Oceanía). - Se decidió destruir la máquina virtual actual y reaprovisionar una nueva en una región europea cercana (Frankfurt/Ámsterdam).

Detalle técnico: Latencias sostenidas cercanas a los 300ms sin pérdida de paquetes (packet loss) son un síntoma inequívoco de distancia transcontinental debido a las limitaciones físicas de la fibra óptica, no de saturación de red local.

Motivo / criterio (Aprendizaje): Física de redes y Core Web Vitals. Por mucho que se optimice el código (Shift-Left) y el tamaño de los assets (WebP), la ubicación física del servidor dicta el TTFB (Time to First Byte) base. Seleccionar la región Edge adecuada es el primer paso innegociable de un despliegue.

Siguiente paso o deuda: Recrear el Droplet, obtener la nueva IP, validar la latencia y proceder con la Fase 1 del Deployment Playbook.

2026-04-17 — Inicio de Fase 6 y creación del Deployment Playbook

Contexto: Con la auditoría local en verde y el Boilerplate consolidado, es momento de transicionar el proyecto desde el entorno de desarrollo (localhost) hacia la infraestructura de producción (DigitalOcean Droplet).

Hecho: - Se ha redactado el manual de operaciones `docs/deployment-playbook.md` . - Se ha marcado el primer hito de la Fase 6.1 en el `README.md` .

Detalle técnico: El Playbook divide el despliegue en 5 fases operativas: Aprovisionamiento LEMP, Clonación vía Git, Aislamiento WP (Symlink), Enrutamiento Nginx+SSL y Verificación final.

Motivo / criterio: Reducción de riesgo y estrés operativo. Documentar el paso a paso ("Runbook" o "Playbook") antes de tocar el servidor de producción previene errores por omisión, asegura que se replican las políticas de seguridad estrictas (Shift-Left) y convierte el despliegue en una tarea rutinaria y auditable.

Siguiente paso o deuda: Conectar vía SSH al servidor de producción y ejecutar la Fase 1 del Playbook (Aprovisionamiento LEMP).

2026-04-17 — Auditoría arquitectónica externa y fijación de dependencias

Contexto: Se sometió el repositorio a un análisis externo automatizado (GitHub Copilot) para evaluar su madurez (readiness) antes del paso a producción (Fase 6).

Hecho: - Se revisó el documento `docs/Analisi-exhaustivo-antes-de-produccion-copilot-github.md` . - Se modificó `requirements.txt` cambiando `Pillow>=10.0.0` por el anclaje estricto `Pillow==10.2.1` .

Detalle técnico: El análisis validó la arquitectura híbrida, la seguridad (CSP) y el aislamiento DevSecOps otorgándole la máxima calificación. Identificó correctamente la carencia de políticas de Backup/Rollback (esperadas en la inminente Fase 6) y alertó sobre el riesgo de mutación de dependencias no ancladas en Python.

Motivo / criterio: Reproducibilidad absoluta. En DevOps, usar operadores `>=` en gestores de paquetes expone el despliegue a rupturas (breaking changes) si se publica una actualización mayor de la librería. Fijar versiones con `==` garantiza que el entorno de producción instalará exactamente los mismos binarios que se auditaron en local. Se descartaron recomendaciones de sobreingeniería (Redis, AWS) por violar la premisa de austeridad del proyecto.

Siguiente paso o deuda: Diseñar el "Deployment Playbook" (Backups, Rollback, Deploy) como primer hito de la Fase 6.1.

2026-04-17 — Restauración del compilador SASS (merci-styles.py)

Contexto: Se detectó la ausencia del script compilador `merci-styles.py` tras las maniobras de limpieza y fusión de ramas de diseño, amenazando la mantenibilidad de la arquitectura CSS de la plantilla.

Hecho: - Se ha restaurado y refactorizado el script `scripts/merci/merci-styles.py`.

Detalle técnico: El script recupera su lógica autónoma: descarga automáticamente el binario standalone de Dart Sass en `scripts/merci/bin/` (ignorando Node/NPM host) y compila `src/scss/main.scss` a `public/css/main.css`.

Motivo / criterio: Resiliencia. Un boilerplate debe contener todas las herramientas necesarias para su propia construcción de forma intrínseca. Si una pieza de infraestructura se pierde (debido a exclusiones o fallos en el trackeo de Git), se debe restituir inmediatamente antes de avanzar a producción.

Siguiente paso o deuda: Validar la compilación con `merci-watcher.py` e iniciar la Fase 6 de despliegue con garantías.

2026-04-17 — Corrección de URLs canónicas en vistas estáticas

Contexto: La auditoría integral previa al despliegue (`merci-audit.py`) detectó dos advertencias no bloqueantes (`WARN SEO_CANONICAL`) por la falta de la etiqueta canónica en las nuevas páginas de la plantilla.

Hecho: - Añadida etiqueta `<link rel="canonical">` a `public/biblioteca/index.html` y `public/contacto/index.html`.

Detalle técnico: Se implementaron explícitamente las rutas absolutas (`https://mercedev.es/biblioteca` y `https://mercedev.es/contacto`) utilizando la etiqueta `<link rel="canonical">`, que actúa como la declaración oficial de la "fuente de la verdad" para cada documento.

Motivo / criterio: Rigor técnico y SEO "Shift-Left". Los motores de búsqueda (como Google) penalizan el contenido duplicado, algo que ocurre accidentalmente si un usuario accede a la web con `www` , sin `www` , o mediante enlaces con parámetros de rastreo (ej. `?utm_source=twitter`). La etiqueta canónica consolida toda la autoridad SEO de esas variantes en una única URL oficial. Solventar esta advertencia garantiza el estándar de calidad (100/100) del Boilerplate.

Siguiente paso o deuda: Confirmar auditoría a 0 advertencias e iniciar definitivamente la Fase 6 (Preparación de Release).

2026-04-16 — Auditoría integral pre-despliegue (Sanity Check)

Contexto: Antes de iniciar oficialmente la Fase 6 (Preparación de release), se requiere una validación cruzada de todos los sistemas locales para certificar la estabilidad de la plantilla "Merci Boilerplate".

Hecho: - Se ejecutó la batería de pruebas unitarias (`unittest`). - Se ejecutó la auditoría estática estricta (`merci-audit.py --strict-json-ld`). - Se ejecutó el rastreador dinámico HTTP (`merci-linkcheck.py`).

Detalle técnico: La validación abarca lógica algorítmica (tests), análisis estático de código/SEO/seguridad y verificación dinámica de enrutamiento a través del proxy Nginx.

Motivo / criterio: Rigor DevSecOps (Pre-flight check). Un pase a producción debe estar precedido por la confirmación empírica (sin errores ni advertencias) de todas las herramientas de aseguramiento de calidad (QA) implementadas en las fases anteriores.

Siguiente paso o deuda: Iniciar la Fase 6 (Despliegue y Auditoría Final) tras confirmar el éxito (código de salida 0) de todos los scripts.

2026-04-16 — Fix: Enlace de Tienda en Child Theme

Contexto: El rastreador `merci-linkcheck.py` detectó un único enlace roto restante (`/tienda`) originado desde las páginas servidas por WordPress (`/blog`).

Hecho: - Se actualizó el `href` en `src/wp-theme/merci-theme/index.php` de `/tienda` a `/blog/tienda`.

Detalle técnico: Las páginas estáticas se actualizaron previamente, pero la plantilla dinámica conservaba la ruta obsoleta. La corrección alinea el 100% de los menús de navegación con la ruta real bajo el proxy inverso de Nginx.

Motivo / criterio: Coherencia absoluta en la navegación. La experiencia de usuario debe ser transparente sin importar si el visitante se encuentra en la capa estática o dinámica.

Siguiente paso o deuda: Validar el script de enlaces a 0 errores e iniciar la Fase 6 de Preparación de Release.

2026-04-16 — Reestructuración de enrutamiento Nginx y resolución de API REST WP

Contexto (Desafío): Persistían errores 404 en rutas dinámicas y la API de WordPress (`wp-json`), impidiendo guardar páginas en el editor de bloques ("La respuesta no es una respuesta JSON válida"). El origen era un conflicto al combinar la directiva `alias` con el motor PHP en Nginx.

Hecho (Maniobra): - Se sustituyó la directiva `alias` por un enlace simbólico físico (`ln -s /var/www/wordpress/public/blog`). - Se simplificó drásticamente el bloque `location` en el Virtual Host de Nginx (`mercedev-local`). - Se forzó el reseteo de los Enlaces Permanentes en WP.

Detalle técnico: El bloque `location /blog` pasó de usar `alias` a confiar en la resolución natural del `root` a través del symlink en `public/blog`. Esto repara variables globales vitales para el enrutamiento interno de WP (como `$_SERVER['REQUEST_URI']`). Tras recargar Nginx (`sudo systemctl reload nginx`) y guardar permalinks, la API REST volvió a operar con normalidad.

Motivo / criterio (Aprendizaje): Robustez de infraestructura. Los alias en Nginx con PHP generan "bugs" históricos de enrutamiento. Un enlace simbólico es una solución nativa del sistema operativo, completamente transparente para el servidor web, resolviendo la raíz arquitectónica del problema en lugar de aplicar parches en el código.

Siguiente paso o deuda: Corregir el último enlace roto (`/tienda`) en el Child Theme detectado por el rastreador local.

2026-04-16 — Creación de herramienta de rastreo dinámico (Merci LinkCheck)

Contexto: La auditoría estática (`merci-audit.py`) no puede validar el enrutamiento real generado por Nginx y WordPress. Se requería una herramienta para asegurar la ausencia de enlaces rotos (404) a nivel de infraestructura HTTP antes del despliegue.

Hecho: - Se implementó `scripts/merci/merci-linkcheck.py` .

Detalle técnico: El script es un *crawler* construido con la librería estándar (`urllib` y `html.parser`). Recorre el dominio local iterativamente resolviendo anclas (`<a>`), hojas de estilo (`<link>`) e imágenes (`<img>`), verificando que devuelvan códigos HTTP válidos (200 OK). Mantiene un registro de rutas procesadas y la fuente del enlace roto para facilitar la depuración.

Motivo / criterio: Robustez de la arquitectura híbrida. Comprobar dinámicamente el proyecto es la única forma empírica de certificar que el CMS y el núcleo estático están comunicándose y resolviendo las URLs correctamente (Shift-Right testing ejecutado en Shift-Left).

Siguiente paso o deuda: Ejecutar el rastreador localmente (`python3 scripts/merci/merci-linkcheck.py`) para certificar que el Boilerplate no tiene enlaces rotos antes de iniciar el despliegue de la Fase 6.

2026-04-16 — Purga manual y definitiva del bucle de enlaces (Symlink Loop)

Contexto (Desafío): Al utilizar `git restore` para recuperar la carpeta `merci-theme` , el bucle infinito reapareció, revelando que el enlace simbólico erróneo había quedado registrado en un commit anterior en el historial de Git.

Hecho (Maniobra): - Se extrajeron temporalmente los archivos críticos (`index.php` , `functions.php` , `style.css`). - Se eliminó y recreó manualmente

el directorio `src/wp-theme/merci-theme/` . - Se devolvieron los archivos a la carpeta limpia para forzar la actualización del índice.

Detalle técnico: La secuencia de comandos `mv` , `rm -rf` y `mkdir` permitió destruir físicamente el enlace recursivo a nivel de sistema operativo. Al realizar el commit posterior, se sobrescribe el estado del árbol en Git, purgando permanentemente la referencia al enlace simbólico fantasma.

Motivo / criterio (Aprendizaje): `git restore` recupera fielmente el historial, incluyendo los errores. La cirugía manual de directorios es la intervención más segura y pragmática para romper dependencias circulares (filesystem loops) antes de conciliar el estado limpio con el control de versiones.

Siguiente paso o deuda: Finalizar el commit atómico y arrancar con la Fase 6 (Despliegue y Auditoría Final).

2026-04-16 — Resolución de bucle infinito (Symlink Loop) en Child Theme

Contexto (Desafío): El directorio `src/wp-theme/merci-theme/` mostraba una recursividad de subcarpetas aparentemente infinitas, provocando confusión y amenazando con bloquear el escaneo del editor de código o de Git.

Hecho (Maniobra): - Se ha identificado la presencia de un bucle de enlaces simbólicos (symlink loop). - Se han eliminado las subcarpetas/enlaces erróneos dentro del directorio del tema mediante los comandos `rm -rf src/wp-theme/merci-theme/*/` y `find -type l -delete` .

Detalle técnico: Este fenómeno óptico del sistema de archivos ocurre cuando un enlace simbólico se crea accidentalmente dentro de la misma ruta a la que apunta (o a su padre), creando una referencia circular. El tamaño real en disco es cero, pero los indexadores (como VS Code o Git) pueden colgarse intentando seguir el "pasillo infinito".

Motivo / criterio (Aprendizaje): Mantener el aislamiento absoluto de los componentes. El directorio `merci-theme` solo debe albergar la tríada de archivos planos (`index.php` , `functions.php` , `style.css`). Cualquier directorio

anidado ahí dentro es, por definición de esta arquitectura, un residuo que debe ser purgado.

Siguiente paso o deuda: Comprobar la estabilidad del árbol de directorios y avanzar hacia la Fase 6 de despliegue.

2026-04-16 — Eliminación de archivo fantasma en el Child Theme

Contexto: Un archivo `index.html` residual (con el contenido temporal de la página de Contacto) persistía dentro del directorio del tema de WordPress (`src/wp-theme/merci-theme/`), ensuciando la arquitectura del CMS.

Hecho: - Eliminado `src/wp-theme/merci-theme/index.html` mediante `git rm` .

Detalle técnico: La existencia de archivos `.html` estáticos dentro de un tema de WordPress no afecta al motor de renderizado PHP por defecto, pero vulnera los principios de limpieza estructural (Clean Code) y causa confusión.

Motivo / criterio: Higiene del código y rigor. La plantilla de WordPress solo debe contener los archivos estrictamente necesarios para su funcionamiento e integración dinámica (`index.php` , `style.css` , `functions.php`).

Siguiente paso o deuda: Confirmar la limpieza del repositorio e iniciar por fin la Fase 6 de Preparación de Release.

2026-04-16 — Corrección de fronteras Nginx y reubicación de página Contacto

Contexto: Al validar la navegación híbrida, los enlaces hacia Tienda y Contacto devolvían error. Se constató un fallo en la generación de archivos y una violación de las fronteras de enrutamiento definidas para el CMS.

Hecho: - Se reubicó el archivo `index.html` de Contacto a su ruta estática correcta (`public/contacto/index.html`). - Se corrigieron los enlaces de navegación de la Tienda de `/tienda` a `/blog/tienda` en todas las cabeceras.

Detalle técnico: El archivo de contacto se había generado erróneamente en el Child Theme. Respecto a la Tienda (WooCommerce), al estar WordPress

encapsulado bajo Nginx en la ruta `/blog` , cualquier página dinámica que genere (incluyendo el catálogo) hereda el prefijo de esa ruta base.

Motivo / criterio: Arquitectura de aislamiento. Nginx actúa como muro: lo estático vive en la raíz (`/`) y lo dinámico en `/blog` . Intentar acceder a `/tienda` provoca que Nginx busque un archivo estático inexistente, reforzando la necesidad de que los enlaces respeten las fronteras de infraestructura.

Siguiente paso o deuda: Validar la navegación estática y dinámica de todo el menú principal.

2026-04-16 — Adecuación de la vista pública (Demo Boilerplate)

Contexto: Tras el pivote estratégico para convertir el proyecto en "Merci Boilerplate", el archivo estático `index.html` aún contenía textos (copy) específicos de una web personal.

Hecho: - Se refactorizaron los textos del `index.html` para transformarlo en una página de presentación técnica del Boilerplate. - Se mantuvo la marca de autora (`mercedev.es`) incrustada por diseño en el footer, header y metadatos.

Detalle técnico: Se reemplazaron las tarjetas de "Art de Coté" y "Merci" por explicaciones de la "Capa Dinámica" y el "Núcleo Estático". Se actualizó la etiqueta `<title>` para reflejar el nombre de la plantilla.

Motivo / criterio: Coherencia de producto. Alguien que clone este repositorio debe encontrar una "Landing Page" que le explique qué acaba de instalar y cómo está estructurado, sirviendo a su vez como demostración visual de los componentes SASS (`.home-grid` , `.home-card`).

Siguiente paso o deuda: Iniciar formalmente la Fase 6 (Preparación de Release y Auditoría de Rendimiento).

2026-04-16 — Integración y limpieza de rama de diseño

Contexto: La rama `feat/fase-3-diseno` cumplió su objetivo de aislar el desarrollo del sistema SASS (Grid/Cards) y el optimizador de imágenes.

Hecho: - Se ha fusionado (merge) la rama `feat/fase-3-diseno` hacia `main` . - Se ha eliminado la rama de desarrollo.

Detalle técnico: Se utilizaron los comandos `git checkout main` , `git merge feat/fase-3-diseno` y `git branch -d feat/fase-3-diseno` .

Motivo / criterio: Higiene de control de versiones. Las ramas de funcionalidad deben tener ciclos de vida cortos y eliminarse inmediatamente tras su integración para prevenir repositorios inflados con ramas "zombis" y mantener el árbol de Git limpio y legible.

Siguiente paso o deuda: Limpiar los textos e imágenes del `index.html` para reflejar la vista demo del nuevo "Merci Boilerplate".

2026-04-16 — Pivote Estratégico: Transición a "Merci Boilerplate"

Contexto: Se identificó que el valor real de la arquitectura desarrollada no reside en una página web personal específica, sino en la infraestructura híbrida, de seguridad y automatización subyacente.

Hecho: - Pivote del proyecto de web personal (`mercedev.es`) a plantilla de desarrollo (`Merci Boilerplate`). - Actualización de `README.md` e `instrucciones.md` para reflejar la nueva misión del repositorio.

Detalle técnico: Se preserva toda la integración dinámica (WordPress aislado, Nginx proxy) y la automatización DevSecOps (`merci-audit.py` , `merci-optimizer.py`). El objetivo del código ahora es servir como base "clonable" para futuros proyectos web.

Motivo / criterio: Separación de responsabilidades a nivel macro (Arquitectura vs. Producto final). Construir un boilerplate permite abstraer y reutilizar las estrictas medidas de seguridad (Shift-Left) y rendimiento en múltiples webs futuras, maximizando el retorno del tiempo de ingeniería invertido.

Siguiente paso o deuda: Limpiar el HTML del núcleo estático (`index.html`) para adaptarlo a un formato de plantilla genérica de demostración.

2026-04-16 — Integración de componentes SASS (Grid/Cards) en plantillas dinámicas

Contexto: Era necesario aplicar el nuevo diseño visual a la capa de WordPress para que los listados de la Biblioteca y el Blog utilizaran la cuadrícula y las tarjetas BEM recién creadas.

Hecho: - Modificado `index.php` en `merci-theme` introduciendo una bifurcación de renderizado mediante `is_singular()`. - Inyectadas las clases `.grid` y `.card` para los listados (archivos). - Implementada lógica condicional en PHP para alternar entre `.card--book` y `.card--booklet`.

Detalle técnico: Se usa `has_category('fichas')` para determinar el contexto temático e inyectar el modificador BEM correspondiente en la etiqueta `<article>`. En la vista de lista se llama a `the_excerpt()` para mejorar la maquetación, reservando `the_content()` solo para lecturas individuales.

Motivo / criterio: Rendimiento y mantenimiento. Concentrar el enrutamiento visual en un único archivo `index.php` inteligente evita la proliferación de plantillas (template hierarchy clutter). Reutilizar el CSS del núcleo estático avala la arquitectura de UI unificada.

Siguiente paso o deuda: Validar la visualización creando entradas de prueba en las diferentes categorías de WordPress.

2026-04-16 — Unificación de conceptos y navegación principal

Contexto: La nomenclatura utilizada en la navegación ("Fichas Técnicas", "Catálogo") resultaba ambigua y se requería establecer los términos definitivos que representarán la arquitectura de la información de cara al usuario.

Hecho: - Se ha unificado la taxonomía principal: Biblioteca, Blog, Art de Coté, Tienda y Contacto. - Se han actualizado los enlaces de navegación en el núcleo estático (`index.html`) y en la capa dinámica (`index.php` del Child Theme).

Detalle técnico: Se reemplazaron las anclas en los elementos `<nav>`. Las rutas proyectadas son `/biblioteca`, `/blog`, `/blog/category/art-de-cote`, `/`

tienda y /contacto . El término "Catálogo" se reserva exclusivamente como definición técnica del funcionamiento interno de WooCommerce.

Motivo / criterio: Claridad cognitiva (UX). Estandarizar los nombres de las secciones principales empleando terminología web universal evita fricción cognitiva en los visitantes y asienta la convención de negocio.

Siguiente paso o deuda: Inyectar las clases SASS (`.grid` , `.card`) diseñadas en los archivos PHP de WordPress para renderizar el contenido dinámico con el nuevo diseño unificado.

2026-04-16 — Desacoplamiento visual de la Portada y redefinición de navegación

Contexto: Se detectó que usar los componentes genéricos (`.grid` , `.card`) en la página de inicio limitaba la capacidad de tener un diseño de "Landing Page" diferenciado de las vistas de lectura (Blog/Biblioteca). Además, la nomenclatura de navegación ("Blog", "Tienda") resultaba ambigua para el propósito del proyecto.

Hecho: - Renombrados los enlaces de navegación a "Fichas Técnicas", "Art de Coté" y "Catálogo" en `index.html` e `index.php` . - Refactorizado `index.html` para usar clases BEM exclusivas (`.home-grid` , `.home-card`). - Creado el archivo SASS `src/scss/pages/_home.scss` para aislar los estilos de la portada.

Detalle técnico: Las rutas de navegación ahora apuntan directamente a las taxonomías de WordPress (`/blog/category/fichas` y `/blog/category/art-de-cote`), estableciendo una arquitectura de la información clara. La portada ahora consume estilos independientes, permitiendo que `_card.scss` evolucione específicamente para el contenido dinámico.

Motivo / criterio: Separación de responsabilidades a nivel de Interfaz de Usuario (UI). Una Landing Page tiene objetivos de marketing y presentación distintos a los de un archivo documental. Desacoplar sus clases CSS previene regresiones visuales (efectos cascada no deseados) al escalar el diseño del CMS.

Siguiente paso o deuda: Crear las categorías correspondientes en el panel de administración de WordPress y diseñar el interior de los artículos (`single.php`).

2026-04-16 — Fix: Restauración de colores del Header y composición de portada

Contexto: La portada (`index.html`) sufrió una alteración visual no deseada tras compilar los nuevos componentes SASS. El header tomó un color claro rompiendo el modo oscuro, y las tarjetas perdieron su cuadrícula original.

Hecho: - Corregido `background-color` a oscuro (`rgba(15, 23, 42, 0.95)`) en `_header.scss` . - Sustituida clase heredada `grid-cols-1-2` por el nuevo componente `.grid` en `index.html` .

Detalle técnico: En SASS, la reescritura de un componente base como `.card` afecta a todo el DOM (Document Object Model - Modelo de Objetos del Documento) que lo invoque. Al crear el componente `_grid.scss` , era imperativo actualizar el HTML estático para que las tarjetas de la portada heredasen el nuevo layout responsivo (CSS Grid) unificado.

Motivo / criterio: Mantenimiento de la cohesión del diseño (UI). El núcleo estático debe consumir los mismos componentes (Grid, Cards) que la capa dinámica para justificar la arquitectura de estilos SASS unificada.

Siguiente paso o deuda: Aplicar las nuevas clases BEM a las plantillas dinámicas de WordPress.

2026-04-16 — Arquitectura de Información: Separación visual de Blog y Biblioteca

Contexto: Necesidad de alinear el diseño visual (SASS) con la estructura conceptual del proyecto, diferenciando la "Biblioteca" (libros técnicos, atemporales, temáticos) del "Blog / Art de Coté" (cuadernillos divulgativos, cronológicos).

Hecho: - Creación de los componentes `_grid.scss` y `_card.scss` en la arquitectura SASS. - Implementación de modificadores BEM `.card--book` y `.card--booklet` .

Detalle técnico: Se ha evitado crear componentes HTML separados, optando por el estándar BEM. `.card--booklet` utiliza acentos azules para el contenido fluido, mientras que `.card--book` utiliza acentos verdes para denotar

documentación técnica consolidada. El `_grid.scss` proporciona una cuadrícula responsiva genérica.

Motivo / criterio: Separar el diseño visual permite que WordPress (cuyo comportamiento por defecto es cronológico) pueda renderizar distintos tipos de contenido usando la misma estructura HTML base, modificando únicamente la clase CSS según la categoría o el tipo de post.

Siguiente paso o deuda: Aplicar estas clases HTML en las plantillas PHP (`index.php` o `archive.php`) del Child Theme para que WordPress escupa el contenido con este nuevo diseño.

2026-04-16 — Ajuste de estilos estructurales del header (BEM)

Contexto: Tras la inclusión del logotipo y el menú de navegación, el componente `.header` presentaba desalineación visual y dependía de estilos en línea temporales.

Hecho: - Se limpiaron los estilos en línea del `<nav>` en `public/index.html` . - Se actualizaron las reglas SASS para `.header` , usando Flexbox para alinear `.header__brand` y `.header__nav` .

Detalle técnico: Se aplicó `display: flex; justify-content: space-between; align-items: center;` al contenedor principal `.header` . Se definió un `gap: 1.5rem` explícito en `.header__nav` de acuerdo con la metodología BEM.

Motivo / criterio: Separación estricta de responsabilidades (Separation of Concerns). Los estilos en línea son un antipatrón en una arquitectura escalable. Toda la lógica visual debe residir en los archivos SASS correspondientes.

Siguiente paso o deuda: Diseñar las tarjetas dinámicas (`_card.scss`) y la cuadrícula (`_grid.scss`) para la Biblioteca y el Catálogo.

2026-04-16 — Ajuste en auditoría para excepción de Favicon

Contexto (Desafío): Se detectó que la nueva regla de auditoría `IMG_FORMAT` bloquearía incorrectamente el commit del archivo `public/favicon.png` , que debe permanecer en formato no optimizado por razones de compatibilidad.

Hecho (Maniobra): - Se ha modificado la función `audit_image_path` en `merci-audit.py` .

Detalle técnico: Se ha añadido una condición de salida temprana (`return`) que ignora la validación si el archivo se llama `favicon.png` y reside directamente en la carpeta `public/` .

Motivo / criterio: El favicon es un archivo de sistema con requisitos de compatibilidad que priman sobre la optimización general de assets de contenido. El sistema de auditoría debe ser lo suficientemente inteligente para gestionar estas excepciones arquitectónicas.

Siguiente paso o deuda: Realizar el commit de todos los cambios acumulados en la rama `feat/fase-3-diseno` .

2026-04-16 — Validación final del optimizador y flujo de assets

Contexto: Tras los parches y la creación de tests, se procedió a la prueba de fuego del flujo de optimización con los assets reales del proyecto (`logo.png` y `favicon.png`).

Hecho: - Se ha colocado `favicon.png` en `public/` . - Se ha colocado `logo.png` en `.assets-raw/` . - Se ha ejecutado `merci-optimizer.py` con éxito, generando `assets/logo.webp` y las variantes responsivas correspondientes.

Detalle técnico: El script ha validado su lógica de no-escalado, omitiendo la generación de imágenes más grandes que el original. Se ha confirmado que el `favicon.png` se sirve correctamente desde la raíz y el `logo.webp` desde `/assets` .

Motivo / criterio: El flujo de gestión de assets está completo y validado. La infraestructura de optimización está lista para soportar el futuro contenido visual del blog y el catálogo.

Siguiente paso o deuda: Ajustar el CSS del componente `.header` para alinear y estilizar correctamente el nuevo logotipo.

2026-04-16 — Integración de identidad visual (Favicon y Logo) y fix en optimizador

Contexto: Al proceder a integrar los primeros assets visuales (logo y favicon), se detectó que `merci-optimizer.py` omitiría imágenes con dimensiones inferiores al target mínimo (400px), dejando fuera los logotipos estándar.

Hecho: - Se ha parcheado `merci-optimizer.py` para generar siempre una versión `.webp` base del tamaño original, además de las versiones escaladas. - Se actualizó `test_optimizer.py` para cubrir el nuevo comportamiento. - Se implementaron etiquetas `<img>` para el logo y `<link rel="icon">` para el favicon en `index.html` e `index.php`.

Detalle técnico: Se diferenció la arquitectura de assets: el `favicon.png` reside sin procesar en `public/` por compatibilidad nativa de navegadores antiguos y crawlers, mientras que el logo viaja por el pipeline de optimización (`.assets-raw/` a `assets/logo.webp`). Se añadieron los atributos `width` y `height` en el HTML para mitigar el Cumulative Layout Shift (CLS).

Motivo / criterio: Las automatizaciones no deben convertirse en bloqueadores del diseño. Generar siempre la copia base asegura compatibilidad con cualquier asset visual independientemente de su tamaño. Las medidas en el tag `img` son obligatorias para mantener el Core Web Vitals en verde.

Siguiente paso o deuda: Proveer físicamente las imágenes, compilar y revisar en el navegador.

2026-04-16 — Lección de TDD: Corrección de `AttributeError` en `unittest.mock`

Contexto (Desafío): Al ejecutar el test para `merci-optimizer.py`, se produjo un `AttributeError: 'PosixPath' object attribute 'glob' is read-only`, bloqueando la validación.

Hecho (Maniobra): - Se ha refactorizado `scripts/merci/tests/test_optimizer.py` para corregir el objetivo de los decoradores `@patch`.

Detalle técnico: El error se debía a que se intentaba parchear un método (`.glob` , `.mkdir`) en una *instancia* de un objeto `Path` (`SOURCE_DIR`), lo cual no está permitido. La solución correcta es parchear el método en la *clase* `Path` dentro del espacio de nombres del módulo que se está probando. Los decoradores se cambiaron a `@patch("merci_optimizer.Path.glob")` y `@patch("merci_optimizer.Path.mkdir")` .

Motivo / criterio (Aprendizaje): Lección fundamental de `unittest.mock` : se debe parchear el objeto "donde se busca" (`where it's looked up`), no "donde se define". Al parchear la clase, cualquier instancia creada dentro del test usará la versión simulada del método, respetando la inmutabilidad de los objetos `pathlib` .

Siguiente paso o deuda: Re-ejecutar el test para confirmar el éxito y proceder con la optimización de assets.

2026-04-16 — Pruebas unitarias de optimizador y auditoría de extensiones

Contexto: Antes de utilizar operativamente `merci-optimizer.py` , era imperativo aplicar la regla de TDD (crear su test) y asegurar que el hook de pre-commit bloqueara la adición accidental de formatos no optimizados.

Hecho: - Añadida función `audit_image_path` en `merci-audit.py` para bloquear archivos `.png` , `.jpg` , `.jpeg` . - Creado el test unitario `scripts/merci/tests/test_optimizer.py` .

Detalle técnico: El test utiliza `unittest.mock` para interceptar llamadas a `Pillow` y el sistema de archivos (`Path.glob` , `Image.open`), validando que la lógica de iteración sobre `TARGET_WIDTHS` se cumple sin grabar archivos reales. El auditor ahora filtra extensiones de imagen sin leerlas como texto UTF-8.

Motivo / criterio: Rigor DevSecOps. Se previene proactivamente la degradación del rendimiento por despistes humanos (subir un `.png` directo a producción) y se garantiza que la herramienta de optimización está cubierta por test antes de integrarla en el flujo.

Siguiente paso o deuda: Ejecutar los tests, confirmar su éxito y pasar a la inclusión del logotipo y favicon en formato optimizado.

2026-04-16 — Fase 3.4: Implementación del optimizador de imágenes

Contexto: Dentro de la rama `feat/fase-3-diseno`, se aborda el hito 3.4 para automatizar la creación de imágenes responsivas y optimizadas para la web.

Hecho: - Se ha creado el archivo `requirements.txt` para gestionar las dependencias de Python, añadiendo `Pillow`. - Se ha implementado el script `scripts/merci/merci-optimizer.py`. - Se ha marcado el hito 3.4 como completado en el `README.md`.

Detalle técnico: El script escanea `.assets-raw/` en busca de imágenes, y para cada una, genera múltiples versiones `.webp` en la carpeta `assets/` con diferentes anchos (1920, 1280, 800, 400px), manteniendo la relación de aspecto.

Motivo / criterio: Rendimiento (Core Web Vitals). Servir imágenes en formato WebP y con el tamaño adecuado para cada dispositivo (responsive) reduce drásticamente el peso de la página y acelera los tiempos de carga, lo cual es un pilar de la filosofía del proyecto.

Siguiente paso o deuda: Instalar las dependencias (`pip install -r requirements.txt`), probar el script con una imagen de ejemplo y proceder con el diseño SASS de las plantillas dinámicas.

2026-04-16 — Creación de rama de desarrollo para diseño y optimización

Contexto: Iniciar el desarrollo visual (SASS/BEM) y la optimización de multimedia aislando el trabajo para proteger la estabilidad del núcleo ya validado en la rama `main`.

Hecho: - Se aprueba la creación de la rama `feat/fase-3-diseno`. - Se define el *sprint* de tareas: favicon, logotipo, script `merci-optimizer.py` (Fase 3.4) y plantillas dinámicas (`single.php`).

Detalle técnico: El trabajo se desarrollará fuera de `main` usando `git checkout -b feat/fase-3-diseno`. Una vez auditado y finalizado, se integrará (merge) de vuelta.

Motivo / criterio: Práctica estándar de Git y DevSecOps. Proteger la rama principal garantiza que siempre exista una versión estable y desplegable del proyecto si el trabajo de diseño experimental sufre regresiones.

Siguiente paso o deuda: Crear la rama, implementar `merci-optimizer.py` y añadir los assets estáticos base.

2026-04-16 — Definición de tipología de contenidos (Biblioteca y Art de Coté)

Contexto: Antes de aplicar diseño visual (Fase 3) o desplegar (Fase 6), es necesario definir cómo se estructurarán los contenidos para que el diseño responda a necesidades reales del producto.

Hecho: - Se ha conceptualizado el formato "Libro/Ficha Técnica" para proyectos mayores (ej. este mismo repositorio). - Se ha conceptualizado el formato "Cuadernillo" para Art de Coté, basado en la estructura de 3 átomos (Desafío, Maniobra, Aprendizaje).

Detalle técnico: Esta arquitectura de información requerirá el uso de categorías en WordPress y la creación de la plantilla `single.php` en el Child Theme. Dicha plantilla debe usar clases BEM específicas (`.booklet__challenge`, `.booklet__maneuver`) para soportar el diseño en SASS.

Motivo / criterio: El diseño (CSS) sigue a la función (Semántica). No se puede diseñar la interfaz de un proyecto sin saber qué datos contiene. Esta definición adelanta requisitos de la Fase 7 integrándolos coherentemente en la fase actual de diseño.

Siguiente paso o deuda: Crear las plantillas HTML/PHP base para estos tipos de contenido y comenzar su diseño SASS.

2026-04-16 — Pivote estratégico: Diseño visual de rutas dinámicas (Catálogo y Blog)

Contexto: Se constató que, aunque la infraestructura del catálogo (WooCommerce) y el blog está integrada y asegurada, visualmente carecen de diseño ("no hay web en condiciones"). Esto se debe a la eliminación deliberada de los estilos por defecto para proteger el rendimiento.

Hecho: - Pausa de la entrada a la Fase 6 (Despliegue). - Retorno al espacio de Ingeniería de Estilos (Fase 3) aplicado a la capa dinámica.

Detalle técnico: WooCommerce y WordPress renderizan marcado HTML crudo al haber desencolado `global-styles` y los estilos por defecto. Es necesario construir los componentes SASS (`_card.scss` , `_grid.scss`) y adaptar las plantillas de PHP a la metodología BEM del núcleo estático.

Motivo / criterio: Una arquitectura perfecta no cumple su propósito si la interfaz de usuario (UX/UI) parece rota o inacabada. Hay que vestir el chasis dinámico con el sistema de diseño propio antes de presentar el proyecto públicamente como un producto maduro.

Siguiente paso o deuda: Diseñar e implementar los componentes SASS para las tarjetas de productos y estructurar la vista del catálogo.

2026-04-16 — Conexión del núcleo estático con rutas dinámicas

Contexto: La página de inicio (`public/index.html`) carecía de enlaces hacia los sistemas dinámicos recién integrados (`/blog` y `/tienda`), manteniendo un `TOD0` pendiente de la Fase 2.

Hecho: - Se ha reemplazado el comentario `TOD0` en `public/index.html` por enlaces funcionales. - Se ha alineado la estructura del `<header>` estático con la del *Child Theme* de WordPress para mantener coherencia semántica.

Detalle técnico: Se han añadido etiquetas `<a>` con las clases BEM `header__brand` y `nav__link` apuntando a las rutas que gestiona Nginx como proxy inverso (`/blog` y `/tienda`).

Motivo / criterio: Una vez que las rutas dinámicas están aseguradas, aisladas y operativas a nivel de servidor (Fases 4 y 5), es seguro exponerlas en el frontend público para permitir la navegación del usuario final.

Siguiente paso o deuda: Iniciar la Fase 6 (Despliegue y Auditoría Final).

2026-04-16 — Apertura del repositorio: Licencia y reenfoque arquitectónico

Contexto: Preparativos finales para hacer público el repositorio en GitHub. Se requería una licencia formal y ajustar el *copy* de la página de inicio para reflejar la verdadera naturaleza técnica del proyecto.

Hecho: - Añadido archivo `LICENSE` (MIT). - Actualizado apartado de Licencia en `README.md`. - Refactorizado texto de `public/index.html` para enfocarlo en Arquitectura de Software y DevSecOps.

Detalle técnico: Se implementó la Licencia MIT por ser el estándar para compartir herramientas de código abierto (como el ecosistema de scripts Merci). El HTML se adaptó para destacar conceptos como "Shift-Left", "Aislamiento de sistemas" y "Trazabilidad".

Motivo / criterio: Un repositorio público es la carta de presentación técnica. El proyecto no es una web estándar, sino una infraestructura automatizada; el lenguaje empleado debe transmitir esa madurez ingenieril a cualquier visitante o reclutador técnico.

Siguiente paso o deuda: Iniciar la Fase 6 (Despliegue y Auditoría Final).

2026-04-16 — Cierre de Fase 5: Consolidación del Documento de Hardening

Contexto: Finalizar la Fase 5 (Quality Assurance y Hardening) dejando un registro auditable de todas las medidas de seguridad implementadas en las diferentes capas del proyecto.

Hecho: - Se ha creado el documento `docs/checklist-hardening.md`. - Se ha marcado el último hito de la Fase 5.4 como completado en el `README.md`.

Detalle técnico: El documento recopila las directivas CSP, los hooks de bloqueo en WordPress (XML-RPC, generadores), la política estricta de permisos de servidor (`chmod 600` para `wp-config.php`) y las reglas bloqueantes del auditor DevSecOps.

Motivo / criterio: La seguridad no es un estado, es un proceso. Documentar estas medidas en forma de *checklist* garantiza que no se pierda conocimiento arquitectónico y proporciona una herramienta de validación vital para futuros despliegues a producción (Fase 6).

Siguiente paso o deuda: Iniciar la Fase 3 (Ingeniería de Estilos) para aplicar SASS y BEM al diseño visual.

2026-04-16 — Fase 5.4: Auditoría integral exitosa sin hallazgos

Contexto: Tras lanzar la ejecución en todo el repositorio de `merci-audit.py --strict-json-ld` , era necesario confirmar el estado del código base.

Hecho: - Se superó la auditoría estricta sin `ERROR` ni `WARN` . - Se actualizaron los hitos de la Fase 5.4 en el `README.md` (pasada integral y verificación de ausencia de secretos).

Detalle técnico: El script verificó sintaxis, secretos, funciones peligrosas de PHP y SEO técnico en HTML, devolviendo un código de salida `0` .

Motivo / criterio: Una validación en verde a este nivel de exigencia confirma que las prácticas de seguridad y calidad (Shift-Left) se han mantenido desde la Fase 1.

Siguiente paso o deuda: Consolidar el checklist de hardening para dar por cerrada definitivamente la Fase 5.

2026-04-16 — Fase 5.4: Verificación integral de seguridad y consistencia

Contexto: Iniciar la última fase de aseguramiento de la calidad antes del despliegue, ejecutando una auditoría completa sobre todo el repositorio para detectar inconsistencias o errores residuales.

Hecho: - Se ha ejecutado el comando de auditoría estandarizado sobre todo el proyecto. - Se ha actualizado el `README.md` para reflejar el avance.

Detalle técnico: Se utilizó el comando `python3 scripts/merci/merci-audit.py --strict-json-ld` para forzar la revisión de todos los archivos con el máximo nivel de exigencia, incluyendo la validación estricta de JSON-LD.

Motivo / criterio: Garantizar que no quedan cabos sueltos. Una pasada final sobre el estado completo del repositorio es crucial para validar que las integraciones parciales no han introducido regresiones o vulnerabilidades en otras áreas del proyecto.

Siguiente paso o deuda: Corregir los hallazgos críticos que reporte el auditor, si los hubiera.

2026-04-16 — Fase 5.3: Documentación de criterios de fallo del auditor

Contexto: Abordar el último hito de la Fase 5.3, que consiste en documentar explícitamente la diferencia entre los hallazgos bloqueantes y no bloqueantes del sistema de auditoría.

Hecho: - Se ha añadido un párrafo en la sección "Flujo de Contribución y Validación" del `README.md`. - Se ha clarificado que los `ERROR` bloquean los commits, mientras que las `WARN` solo informan. - Se ha marcado la Fase 5.3 como completada en el Roadmap.

Detalle técnico: La distinción se basa en el código de salida de `merci-audit.py`. Un `ERROR` provoca un código de salida `1`, que es interpretado por el hook de `pre-commit` de Git como un fallo que debe detener la operación.

Motivo / criterio: Claridad y predictibilidad para el desarrollador. Es fundamental que el equipo sepa qué tipo de hallazgos detendrán su trabajo y cuáles son meras sugerencias, optimizando así la experiencia de desarrollo (DX).

Siguiente paso o deuda: Iniciar la Fase 5.4 (Verificación de seguridad y consistencia) o retomar la Fase 3 (Ingeniería de Estilos).

2026-04-16 — Fase 5.3: Estandarización del flujo de auditoría local

Contexto: Se clarificó que la Fase 5 no estaba completa. El siguiente paso pendiente era estandarizar la ejecución de auditorías para garantizar la consistencia en el control de calidad antes de cualquier integración de código.

Hecho: - Se ha añadido una sección "Flujo de Contribución y Validación" en el `README.md`. - Se ha definido el comando `python3 scripts/merci/merci-audit.py --strict-json-ld` como la auditoría completa oficial.

Detalle técnico: La estandarización se logra mediante documentación. Al fijar un comando único y oficial, se elimina la ambigüedad y se asegura que todos los desarrolladores validen el código con el mismo nivel de rigurosidad (incluyendo la validación estricta de JSON-LD).

Motivo / criterio: Reproducibilidad y fiabilidad. Un flujo de validación estandarizado es fundamental en DevSecOps para que la calidad no dependa de la memoria o disciplina individual, sino del proceso documentado.

Siguiente paso o deuda: Abordar el último punto de la Fase 5.3: "Documentar criterios de fallo/bloqueo".

2026-04-16 — Fase 5.3: Ampliación de auditoría de seguridad para PHP

Contexto: Con la introducción de WordPress, es necesario que el auditor `merci-audit.py` pueda detectar patrones de código PHP peligrosos que son vectores comunes para vulnerabilidades de Ejecución Remota de Código (RCE).

Hecho: - Se ha implementado la función `audit_php_smells` en `merci-audit.py`. - Se ha actualizado el Roadmap para reflejar el avance en la Fase 5.3.

Detalle técnico: La nueva función utiliza una expresión regular para buscar en archivos `.php` el uso de funciones de alto riesgo como `eval()`, `exec()`, `shell_exec()`, `system()`, etc. Emite una advertencia (`WARN`) para que el desarrollador revise el contexto manualmente.

Motivo / criterio: Seguridad "Shift-Left". Al detectar el uso de estas funciones antes de que el código llegue al repositorio, se reduce drásticamente la probabilidad de introducir una puerta trasera accidentalmente, especialmente a través de código de terceros (plugins o temas).

Siguiente paso o deuda: Probar el auditor contra el `functions.php` y decidir la siguiente regla de QA a implementar.

2026-04-16 — Lección de Flujo: Reparación de historial Git y parcheo manual

Contexto (Desafío): Tras un commit exitoso, se intentó corregir una advertencia del linter (`WARN MD_ACRONYM`) con un commit manual. El comando `git add` falló por un error de ruta relativa y un posterior `merci-commit` generó un commit duplicado con un mensaje incorrecto.

Hecho (Maniobra): - Se ha reparado el historial de Git fusionando los dos últimos commits con `git rebase -i HEAD~2` . - Se ha definido el flujo correcto para parches menores: navegar a la raíz del proyecto y usar `git add <archivo>` y `git commit -m "prefijo: mensaje"` manualmente.

Detalle técnico: El error de `git add` se debió a ejecutarlo desde una subcarpeta. El commit duplicado ocurrió porque `merci-commit` re-leyó la última entrada de la bitácora. La solución `fixup` en el rebase interactivo fusiona los cambios y descarta el mensaje del commit secundario.

Motivo / criterio (Aprendizaje): Las herramientas de automatización como `merci-commit` son para hitos principales justificados por la bitácora. Los parches de documentación o correcciones menores deben gestionarse con comandos manuales de Git desde la raíz del proyecto para mantener un historial limpio y semántico.

Siguiente paso o deuda: Retomar la elección de la siguiente fase del roadmap (Fase 3 o 5.3).

2026-04-16 — Fase 4.4: Erradicación de CSS en línea y carga diferida (Defer)

Contexto: El análisis del código fuente reveló que WordPress 6.x seguía inyectando bloques `<style>` en línea (como `global-styles` y `classic-theme-styles`), saltándose el `wp_dequeue_style` estándar. Además, faltaba garantizar que futuros scripts no bloquearan el renderizado.

Hecho: - Se añadieron reglas `remove_action` para `wp_enqueue_global_styles`. - Se desencholó `classic-theme-styles`. - Se implementó un filtro global (`merci_defer_js_frontend`) para inyectar `defer` en etiquetas `<script>`.

Detalle técnico: La función `wp_enqueue_global_styles` se vincula a los hooks `wp_enqueue_scripts` y `wp_body_open`. Eliminar la acción ataja la raíz del problema. El filtro `script_loader_tag` busca `src` y lo reemplaza por `defer src` condicionado por `!is_admin()`.

Motivo / criterio: Rendimiento puro (Core Web Vitals). El CSS en línea masivo rompe la limpieza del DOM (Document Object Model - Modelo de Objetos del Documento) y retrasa el TTFB (Time to First Byte - Tiempo hasta el Primer Byte). El uso de `defer` asegura que el parseo HTML nunca sea interrumpido por JS, garantizando un LCP (Largest Contentful Paint - Despliegue del Contenido Más Extenso) inmediato.

Siguiente paso o deuda: Dar por finalizada la configuración dinámica y decidir el siguiente paso entre diseño frontend (Fase 3 / 4.5) o QA y Seguridad (Fase 5.3).

2026-04-16 — Parche: Forzar URL absoluta para CSS estático

Contexto: El CSS unificado devolvía 404. WordPress interceptaba el prefijo `/css/main.css` y lo reescribía automáticamente a `http://localhost/blog/css/main.css` en la función `wp_enqueue_style`.

Hecho: - Restaurada la construcción de `$domain_root` dinámico en `functions.php`. - Forzado el parámetro de URL a una ruta absoluta como `http://[host]/css/main.css`.

Detalle técnico: Se implementó `$domain_root = (is_ssl() ? 'https://' : 'http://') . $_SERVER['HTTP_HOST'];` concatenado explícitamente con `/css/main.css`.

Motivo / criterio: Aislar el CMS exige forzar la ruta mediante HTTP absoluto para que Nginx la despache directamente desde `public/css/main.css` sin que el motor interno de WordPress manipule el segmento de red.

Siguiente paso o deuda: Validar la carga de estilos e iniciar la Fase 4.4.

2026-04-16 — Fase 4.2: Corrección de enrutamiento de assets estáticos en WordPress

Contexto: El "escudo de rendimiento" limpiaba correctamente el HTML, pero la hoja de estilos devolvía un error 404. WordPress prefijaba la ruta del CSS con `/blog/`, rompiendo el proxy de Nginx que sirve los assets desde la raíz estática.

Hecho: - Se refactorizó la llamada `wp_enqueue_style` en `functions.php`. - Se implementó la construcción dinámica de la URL absoluta usando `$_SERVER['HTTP_HOST']`.

Detalle técnico: WordPress interpreta las rutas como `/assets/main.css` como relativas a su `siteurl`. Se cambió a `$domain_root = (is_ssl() ? 'https://' : 'http://') . $_SERVER['HTTP_HOST'];` para forzar la petición a `http://localhost/assets/main.css` (directo al bloque Nginx).

Motivo / criterio: Aislar el CMS (Content Management System) significa que este no debe gobernar cómo se sirven los estáticos. Al forzar la petición a la raíz del dominio, Nginx intercepta la llamada y la sirve con máxima velocidad (caché), protegiendo las métricas de rendimiento.

Siguiente paso o deuda: Recargar el frontend para validar la carga del CSS sin errores 404 y verificar la estructura generada por el `index.php` del Child Theme.

2026-04-16 — Fase 4.2: Resolución de permisos para enlaces simbólicos (Child Theme)

Contexto: WordPress no detectaba el "Merci Theme" enlazado simbólicamente porque el usuario del servidor web (`www-data`) no tenía permisos para atravesar el directorio personal del usuario local.

Hecho: - Se otorgaron permisos de ejecución/paso a la ruta del repositorio anfitrión. - Se validó la aparición y activación del tema en el panel de administración de WordPress.

Detalle técnico: Se aplicó `chmod +x` a las carpetas `/home/hildegahr/` , `Escritorio/` y `PROYECTO_mercedev.es/` . Esto resuelve el "Permiso denegado" permitiendo a `www-data` resolver el enlace simbólico hacia `style.css` e `index.php` .

Motivo / criterio: En entornos LEMP locales, es un desafío común la colisión de permisos entre el usuario de escritorio y el demonio web. Dar permiso de ejecución (`+x`) a los directorios anfitriones permite la lectura a través del symlink sin comprometer la política estricta de permisos de los archivos finales.

Siguiente paso o deuda: Validar en el frontend (`http://localhost/blog`) que el "escudo de rendimiento" limpia el código fuente inyectado por defecto.

2026-04-16 — Fase 4.0: Configuración de wp-config.php y despliegue final

Contexto: Conectar la instancia aislada de WordPress con su base de datos dedicada local y asegurar sus permisos de servidor post-instalación.

Hecho: - Se ha creado y configurado `wp-config.php` con credenciales de base de datos (`wp_mercedev_local`) y claves de seguridad generadas. - Se ha ejecutado el instalador de WordPress a través del proxy inverso de Nginx (`http://localhost/blog`). - Se ha aplicado el *hardening* de permisos (`chown` y `chmod`) al directorio `/var/www/wordpress/` . - Se da por finalizada la Fase 4.0 del Roadmap.

Detalle técnico: Se aplicó el principio de mínimo privilegio tras la instalación: directorios a `755` , archivos a `644` y un estricto `600` para `wp-config.php` , asignando la propiedad completa a `www-data:www-data` .

Motivo / criterio: La instalación local no exime de aplicar prácticas de seguridad de producción. Blindar `wp-config.php` y los permisos del CMS desde el minuto uno garantiza que la arquitectura probada localmente es segura para su posterior migración al servidor de producción.

Siguiente paso o deuda: Validar la visualización del Child Theme (Fase 4.2) ahora que existe un WordPress real donde activarlo.

2026-04-16 — Fase 4.0: Configuración de Nginx para entorno local

Contexto: Configurar el servidor web Nginx en el entorno de desarrollo local para replicar la arquitectura de enrutamiento inverso (reverse proxy) definida en `docs/integracion-wordpress.md` .

Hecho: - Se ha creado un nuevo archivo de configuración de sitio en `/etc/nginx/sites-available/mercedev-local` . - Se ha adaptado la configuración para el entorno local, apuntando la raíz estática a la carpeta del proyecto y manteniendo el alias para WordPress. - Se ha añadido un bloque `location /assets` con una directiva `alias` para servir correctamente los recursos compartidos (CSS). - Se ha activado el nuevo sitio y desactivado el sitio por defecto de Nginx.

Detalle técnico: Se creó el archivo `/etc/nginx/sites-available/mercedev-local` y se enlazó simbólicamente a `/etc/nginx/sites-enabled/` . Se verificó la sintaxis con `sudo nginx -t` y se recargó el servicio con `sudo systemctl reload nginx` . Se instruyó sobre cómo verificar la versión del socket de PHP-FPM en `/run/php/` .

Motivo / criterio: Es imprescindible que el entorno de desarrollo local simule fielmente la configuración de producción. La configuración de Nginx es el componente clave que une el núcleo estático y el CMS dinámico, permitiendo probar y validar la arquitectura de aislamiento antes del despliegue.

Siguiente paso o deuda: Configurar el archivo `wp-config.php` de WordPress y ejecutar el instalador web para finalizar la instalación.

2026-04-16 — Fase 4.0: Creación de base de datos y usuario para WordPress local

Contexto: Crear el esquema de base de datos y el usuario dedicado para la instancia local de WordPress, aislando sus datos del resto del sistema.

Hecho: - Se ha accedido a MariaDB con `sudo mysql` . - Se ha creado la base de datos `wp_mercedev_local` y el usuario `wp_user_local` .

Detalle técnico: Se ejecutaron las siguientes sentencias SQL:

```
CREATE DATABASE wp_mercedev_local;
CREATE USER 'wp_user_local'@'localhost' IDENTIFIED BY
'tu_contraseña_elegida';
GRANT ALL PRIVILEGES ON wp_mercedev_local.* TO
'wp_user_local'@'localhost';
FLUSH PRIVILEGES;
```

Motivo / criterio: El uso de una base de datos y un usuario específicos para cada aplicación es una práctica de seguridad fundamental (principio de mínimo privilegio), incluso en un entorno de desarrollo local.

Siguiente paso o deuda: Configurar el bloque de servidor de Nginx para el enrutamiento del núcleo estático y el proxy inverso hacia WordPress.

2026-04-16 — Fase 4.0: Instalación de pila LEMP y configuración base de datos local

Contexto: Preparación del entorno de desarrollo local anfitrión con Nginx, MariaDB y PHP para albergar la instancia aislada de WordPress, replicando la arquitectura de producción de forma nativa.

Hecho: - Se han instalado los paquetes de la pila LEMP (`nginx` , `mariadb-server` , `php-fpm` , `php-mysql`). - Se ha asegurado la instalación local de MariaDB estableciendo contraseña root y eliminando usuarios anónimos.

Detalle técnico: Se utilizó `sudo apt install` para la provisión de dependencias y `sudo mysql_secure_installation` con autenticación `unix_socket` activada para endurecer el motor de base de datos local.

Motivo / criterio: La dependencia de herramientas preempaquetadas (como LocalWP) ofusca la configuración del servidor web, impidiendo auditar y replicar la estrategia de enrutamiento inverso (reverse proxy) de Nginx definida en la Fase 4.1.

Siguiente paso o deuda: Crear la base de datos específica para WordPress local, descargar el CMS y configurar el bloque de servidor en Nginx.

2026-04-16 — Reajuste de entorno: De servidor a PC local y actualización de directrices

Contexto: Confusión entre el entorno de producción (droplet de DigitalOcean) y el entorno de desarrollo (PC local con Ubuntu). Se intentaba configurar bases de datos para el despliegue final cuando el entorno local aún no disponía de la pila tecnológica necesaria para probar la arquitectura aislada.

Hecho: - Se ha añadido la regla 13 a `instrucciones.md` para forzar la verificación de dependencias de entorno antes de avanzar en la configuración. - Se ha introducido la subfase 4.0 en el `README.md` para formalizar la preparación del entorno local LEMP.

Detalle técnico: La configuración local requiere replicar el ecosistema de producción (Linux, Nginx, MariaDB, PHP-FPM) nativamente en el sistema operativo anfitrión (`~/Escritorio/`) para validar el enrutamiento inverso de Nginx sin depender de herramientas aisladas como LocalWP que ofuscan la configuración del servidor.

Motivo / criterio: DevSecOps y "Shift-Left" requieren que el entorno de desarrollo local sea una réplica fiel de la arquitectura de producción. No se puede auditar ni endurecer un CMS localmente sin las herramientas nativas.

Siguiente paso o deuda: Iniciar la Fase 4.0 instalando Nginx, MariaDB y PHP nativos en el Ubuntu local.

2026-04-16 — Fase 5.2: Instalación de la infraestructura de base de datos (MariaDB)

Contexto: Al intentar crear la base de datos para WordPress, se detectó que no había ningún servidor de bases de datos instalado en el droplet (error `mysql: orden no encontrada`).

Hecho: - Se ha instalado el servidor de bases de datos MariaDB, el sustituto directo y recomendado de MySQL en Ubuntu. - Se ha ejecutado el script `mysql_secure_installation` para aplicar un endurecimiento de seguridad inicial.

Detalle técnico: Se utilizaron los comandos `sudo apt update` , `sudo apt install mariadb-server` y `sudo mysql_secure_installation` . Se configuró la autenticación `unix_socket` para el usuario root y se eliminaron las configuraciones inseguras por defecto.

Motivo / criterio: WordPress requiere una base de datos para funcionar. MariaDB es el estándar de la industria para este stack tecnológico. Asegurar la instalación desde el inicio es un paso fundamental de la filosofía "Shift-Left Security".

Siguiente paso o deuda: Proceder con la creación de la base de datos y el usuario específicos para la instancia de WordPress.

2026-04-15 — Incorporación de regla de sincronización del Roadmap

Contexto: Evitar la desincronización entre el código implementado y el estado de las fases documentadas en el proyecto.

Hecho: - Añadir la regla 12 en `instrucciones.md` que obliga a actualizar el `README.md` inmediatamente tras finalizar una tarea.

Detalle técnico: Se formaliza la práctica de marcar con `- [x]` los hitos del `README.md` en la misma sesión de trabajo en la que se consigue el avance.

Motivo / criterio: Mantener una única fuente de verdad (Single Source of Truth) del estado del proyecto. Al estar documentada, el asistente de IA asimila la directriz de proponer la actualización automáticamente.

Siguiente paso o deuda: Finalizar sesión y retomar mañana con la Fase 5.2 (Permisos del servidor de WordPress).

2026-04-15 — Incorporación de Conventional Commits a las directrices

Contexto: Necesidad de estandarizar la nomenclatura de los mensajes de commit (especialmente en parches manuales) para mantener un historial de Git semántico y fácil de auditar.

Hecho: - Añadir la regla 11 sobre la convención de prefijos en `instrucciones.md`.

Detalle técnico: Se definen los prefijos estándar de la industria (`feat:` , `fix:` , `chore:` , `docs:` , `refactor:` , `perf:` , `test:` , `style:`) como parte inmutable de las directrices del repositorio.

Motivo / criterio: La claridad en el control de versiones permite comprender el propósito de cualquier cambio de un solo vistazo. Es un paso clave de madurez DevSecOps que facilitará escalar o retomar el código en el futuro sin fricción.

Siguiente paso o deuda: Iniciar la auditoría de permisos del servidor para WordPress (Fase 5.2).

2026-04-15 — Soporte para commits menores manuales en merci-commit

Contexto: Tareas menores de mantenimiento (como eliminación de duplicados) no ameritan entradas completas en la bitácora, pero la herramienta `merci-commit.py` bloqueaba la acción o duplicaba mensajes forzando una fricción innecesaria.

Hecho: - Añadir comprobación `check_repo_changes` para abortar tempranamente si no hay modificaciones reales en Git. - Permitir el ingreso de un mensaje manual por terminal si hay cambios de código pero la bitácora está intacta.

Detalle técnico: Se implementa `git status --porcelain` para comprobar el estado real de los archivos. Si existen cambios pero no en `bitacora-`

`mercedev.md` , se solicita confirmación para un parche menor y se captura el título vía `input()` de Python, saltándose la extracción de la bitácora.

Motivo / criterio: Equilibrio entre DevSecOps y usabilidad (DX). Ofrecer una válvula de escape estructurada para mantenimientos menores mantiene el historial limpio, no desincentiva el uso de la herramienta y agiliza al desarrollador.

Siguiente paso o deuda: Validar este nuevo flujo mixto y auditar los permisos del servidor de WordPress (Fase 5.2).

2026-04-15 — Endurecimiento (Hardening) de WordPress mediante Child Theme

Contexto: Reducir la superficie de ataque del CMS desactivando endpoints obsoletos y evitando fugas de información que faciliten intrusiones.

Hecho: - Añadir reglas de seguridad (Fase 5.2) en `src/wp-theme/merci-theme/functions.php` . - Actualizar checklist del `README.md` .

Detalle técnico: Se usa `remove_action` para eliminar el metadato generador de versión, `wlwmanifest` y `rsd_link` . Se desactiva completamente la API (Application Programming Interface - Interfaz de Programación de Aplicaciones) XML-RPC mediante el filtro `xmlrpc_enabled` para prevenir ataques de fuerza bruta. Se ofuscan los errores de autenticación con `login_errors` .

Motivo / criterio: Principio de mínima exposición. XML-RPC es un vector común para ataques DDoS (Distributed Denial of Service - Ataque Distribuido de Denegación de Servicio). Ocultar la versión exacta de WP dificulta el escaneo automatizado de vulnerabilidades conocidas.

Siguiente paso o deuda: Auditar la configuración de `wp-config.php` y los permisos del servidor para completar el hardening.

2026-04-15 — Resolución de 404 por favicon ausente (Higiene de logs)

Contexto: Durante la prueba del servidor local, el registro mostró un error 404 persistente al intentar cargar `favicon.ico` .

Hecho: - Añadir `<link rel="icon" href="data:,">` en el `<head>` de `public/index.html`.

Detalle técnico: Los navegadores solicitan automáticamente `/favicon.ico` a la raíz del servidor web. Al no existir el archivo, se genera una petición HTTP (Hypertext Transfer Protocol - Protocolo de Transferencia de Hipertexto) fallida. Se inyecta un URI (Uniform Resource Identifier - Identificador de Recursos Uniforme) de datos vacío para cancelar la petición de red en origen.

Motivo / criterio: Rendimiento e higiene del servidor. Un error 404 consume procesamiento innecesario. Un Data URI vacío silencia el comportamiento automático del navegador manteniendo la política de cero dependencias externas.

Siguiente paso o deuda: Diseñar el isotipo definitivo para el favicon en fases posteriores. Continuar con el Hardening de WordPress (Fase 5.2).

2026-04-15 — Validación local de Content Security Policy (CSP)

Contexto: Verificar empíricamente que la política de seguridad estricta no interfiere con la carga de los recursos legítimos del núcleo estático.

Hecho: - Desplegar servidor local de pruebas
(`python3 -m http.server 8000 -d public/`). - Validar ausencia de bloqueos en la consola de herramientas para desarrolladores del navegador.

Detalle técnico: Al no poseer dependencias de terceros (como tipografías externas o analíticas), la regla `default-src 'self'` permite cargar correctamente el documento HTML y su hoja de estilos unificada. No se registran errores de tipo "CSP violation".

Motivo / criterio: En DevSecOps (Development, Security, and Operations - Desarrollo, Seguridad y Operaciones), la imposición de una política de seguridad siempre debe ir acompañada de una validación funcional para evitar degradación del servicio o bloqueos de UX (User Experience - Experiencia de Usuario).

Siguiente paso o deuda: Comenzar el Hardening de WordPress (Fase 5.2).

2026-04-15 — Fase 5: Implementación de Content Security Policy (CSP)

Contexto: Iniciar la fase de Hardening del núcleo estático protegiéndolo contra ataques de inyección de código.

Hecho: - Añadir directiva CSP (Content Security Policy - Política de Seguridad de Contenidos) en el `<head>` de `public/index.html`.

Detalle técnico: Se establece una política estricta mediante etiqueta `<meta>`:
`default-src 'self'` restringe todos los recursos al dominio actual. Se bloquean plugins (`object-src 'none'`) y la inyección de bases (`base-uri 'self'`).

Motivo / criterio: Aplicación del principio de seguridad "Shift-Left". Una CSP estricta mitiga el riesgo de vulnerabilidades XSS (Cross-Site Scripting - Secuencias de Comandos en Sitios Cruzados) prohibiendo scripts externos o en línea no autorizados.

Siguiente paso o deuda: Validar la carga de la portada en el navegador local para confirmar que la política no bloquea assets legítimos y avanzar con el Hardening de WordPress.

2026-04-15 — Refinamiento de la política de acrónimos (Linter y directrices)

Contexto: La regla estricta de expandir siempre los acrónimos (Inglés - Español) resultaba tediosa para términos que ya estaban muy arraigados en el proyecto.

Hecho: - Actualizar `instrucciones.md` eximiendo de expansión a los acrónimos que aparezcan más de 3 veces. - Implementar una función de conteo global (`get_global_acronym_count`) en `merci-audit.py`.

Detalle técnico: El auditor ahora escanea todo el repositorio buscando archivos `.md`. Si localiza un acrónimo de la *watchlist* que no está expandido, verifica su conteo global. Si es mayor a 3, asume que es un término consolidado y omite la advertencia `WARN MD_ACRONYM`. Se emplea un caché (`GLOBAL_ACRONYM_COUNTS`) para evitar leer el disco repetidas veces.

Motivo / criterio: Reducir la fricción y el tedio en el flujo DevSecOps. Se equilibra la necesidad de claridad técnica inicial con la fluidez una vez que un concepto ya es de dominio público en el repositorio.

Siguiente paso o deuda: Comitear los cambios del linter y comenzar oficialmente la Fase 5: Quality Assurance y Hardening.

2026-04-15 — Validación exitosa del linter de acrónimos

Contexto: El nuevo linter de acrónimos detectó correctamente la falta de expansión de "CMS" durante la ejecución de un commit rutinario, validando su eficacia.

Hecho: - Expandir el acrónimo CMS (Content Management System - Sistema de Gestión de Contenidos) en el registro histórico. - Confirmar el funcionamiento de la regla `WARN` en `merci-audit.py`.

Detalle técnico: El auditor emitió la advertencia `WARN MD_ACRONYM` indicando la línea exacta sin bloquear la creación del commit atómico. Esto permitió mantener la fluidez del proceso informando simultáneamente sobre la deuda técnica de redacción.

Motivo / criterio: Dejar constancia de que el sistema de vigilancia pasiva (Watchlist) cumple su función como corrector de estilo automatizado (DevSecOps) sin añadir fricción paralizante.

Siguiente paso o deuda: Iniciar la Fase 5: Quality Assurance y Hardening.

2026-04-15 — Implementación de linter de acrónimos en Merci Audit

Contexto: Automatizar la verificación de la regla de estilo que exige expandir los acrónimos técnicos en la bitácora y la documentación (Inglés - Español).

Hecho: - Crear la función `audit_md_acronyms` en `scripts/merci/merci-audit.py`. - Definir una lista de vigilancia (*watchlist*) para los acrónimos más críticos.

Detalle técnico: La función utiliza expresiones regulares para detectar si un acrónimo de la lista está presente en archivos `.md`. Si lo encuentra, verifica que exista al menos una instancia con el patrón `ACRÓNIMO ( . . . )` en el documento. Se clasifica como `warn` para no bloquear commits por falsos positivos.

Motivo / criterio: Reducir la carga cognitiva de revisión manual. La automatización parcial mediante *watchlist* es más fiable que una expresión regular genérica para mayúsculas, la cual generaría excesivos falsos positivos.

Siguiente paso o deuda: Validar el comportamiento del auditor con un commit y avanzar a la Fase 5: Quality Assurance y Hardening.

2026-04-15 — Análisis de impacto de wc-cart-fragments (Deuda de conocimiento)

Contexto: Comprensión arquitectónica de los motivos por los que el script `wc-cart-fragments` de WooCommerce degrada el rendimiento web estándar.

Hecho: - Documentar el comportamiento del script AJAX (Asynchronous JavaScript and XML - JavaScript Asíncrono y XML) de fragmentos de carrito.

Detalle técnico: El script invoca una petición `POST` a `/?wc-ajax=get_refreshed_fragments` en cada carga de página. Al ser un `POST` que verifica sesiones y bases de datos mediante PHP (Hypertext Preprocessor - Procesador de Hipertexto), esquiva las capas de caché estáticas (Varnish, Redis, Nginx FastCGI) elevando drásticamente el consumo de CPU (Central Processing Unit - Unidad Central de Procesamiento) y el TTFB (Time to First Byte - Tiempo hasta el Primer Byte).

Motivo / criterio: Dejar constancia del motivo de su desencolado en la Fase 4.3. En arquitecturas en Modo Catálogo, este script aporta 0 funcionalidad a costa de sacrificar métricas críticas de Core Web Vitals como el INP (Interaction to Next Paint - Interacción hasta el Siguiente Pintado).

Siguiente paso o deuda: Consolidar el documento en Git e iniciar la Fase 5: Quality Assurance y Hardening.

2026-04-15 — Fase 4.3: Configuración de WooCommerce en modo catálogo

Contexto: Integrar WooCommerce para mostrar el merchandising de Merci sin el impacto de rendimiento que supone una tienda completa con pasarelas de pago y scripts de carrito AJAX (Asynchronous JavaScript and XML - JavaScript Asíncrono y XML).

Hecho: - Añadir soporte de WooCommerce al `functions.php` del Child Theme.
- Eliminar las acciones de añadir al carrito (`remove_action`). - Desencolar el script `wc-cart-fragments` .

Detalle técnico: Se usa `add_theme_support( 'woocommerce' )` para habilitar las plantillas base. Se bloquea la generación de botones de compra anulando `woocommerce_template_loop_add_to_cart` y `woocommerce_template_single_add_to_cart` . El script de fragmentos de carrito se desencola con prioridad 100.

Motivo / criterio: Rendimiento puro. WooCommerce inyecta JS (JavaScript) pesado por defecto para gestionar el carrito en tiempo real en todas las páginas. Al funcionar como mero catálogo, prescindimos de esta carga protegiendo el Web Vitals score.

Siguiente paso o deuda: Validar la visualización del catálogo e iniciar la fase de endurecimiento y QA (Fase 5).

2026-04-15 — Corrección de importación en pruebas (test_sitemap.py)

Contexto: El archivo de pruebas `test_sitemap.py` quedó roto tras estandarizar el nombre del script principal a `merci-sitemap.py` (con guion medio). Python no permite importar módulos con guiones usando la sintaxis estándar de `import` .

Hecho: - Refactorizar `scripts/merci/tests/test_sitemap.py` . - Implementar carga dinámica de módulos con `importlib.util` .

Detalle técnico: Se reemplazó el `sys.path.append` por `spec_from_file_location` y `module_from_spec` de `importlib.util` . Esto

permite cargar el archivo `merci-sitemap.py` asociándolo al namespace interno seguro `merci_sitemap` para el parcheo con `unittest.mock`.

Motivo / criterio: Mantener la convención de nombres de archivos con guiones en el sistema (ej. `merci-audit.py`, `merci-sitemap.py`) sin sacrificar la cobertura de las pruebas unitarias.

Siguiente paso o deuda: Ejecutar los tests para validar el fix y consolidar los cambios con `merci-commit`.

2026-04-15 — Creación de `index.php` del Child Theme con metodología BEM

Contexto: Proveer una plantilla base para que WordPress renderice contenido dinámico respetando el estándar HTML5 y las clases CSS del núcleo estático.

Hecho: - Crear `src/wp-theme/merci-theme/index.php`. - Implementar "The Loop" de WordPress en una estructura unificada.

Detalle técnico: Se prescinde de la fragmentación tradicional (`get_header()`, `get_footer()`) para concentrar el marcado en un solo archivo. Se incluyen `wp_head()` y `wp_footer()` para permitir la inyección de nuestros assets estáticos controlados. Se aplican clases BEM (`article`, `article__title`, `article__content`).

Motivo / criterio: Minimalismo extremo y reducción de carga de procesamiento I/O de PHP. Al escribir el HTML directamente, se evita que WordPress genere contenedores `<div>` basura o estructuras que rompan el diseño semántico del núcleo.

Siguiente paso o deuda: Validar la vista dinámica y proceder con la configuración de WooCommerce en modo catálogo (Fase 4.3).

2026-04-15 — Creación de `functions.php` como escudo de rendimiento

Contexto: Necesidad de bloquear la inyección de código basura por defecto de WordPress (scripts de emojis, estilos globales, CSS de Gutenberg) para proteger el rendimiento del frontend.

Hecho: - Crear `src/wp-theme/merci-theme/functions.php` . - Implementar reglas de limpieza y desencolado (`dequeue`).

Detalle técnico: Se emplea `remove_action` para detener los scripts de emojis y `wp_dequeue_style` enganchado a la acción `wp_enqueue_scripts` (con prioridad 100) para bloquear `wp-block-library` y `global-styles` . Finalmente, se encola `/assets/main.css` apuntando a la ruta absoluta expuesta por Nginx.

Motivo / criterio: Aislar la vista dinámica del CMS de sus dependencias heredadas pesadas. Si no se bloquea, WordPress inyecta múltiples llamadas de red y estilos en línea que degradarían la métrica de Core Web Vitals lograda en el núcleo estático. **Motivo / criterio:** Aislar la vista dinámica del CMS (Content Management System - Sistema de Gestión de Contenidos) de sus dependencias heredadas pesadas. Si no se bloquea, WordPress inyecta múltiples llamadas de red y estilos en línea que degradarían la métrica de Core Web Vitals lograda en el núcleo estático.

Siguiente paso o deuda: Desarrollar `index.php` del tema para renderizar el esqueleto HTML5 alineado con la metodología BEM del proyecto.

2026-04-15 — Añadir salvaguarda a `merci-commit.py` contra commits sin bitácora

Contexto: Evitar la creación de commits duplicados o la omisión de la actualización de la bitácora, que son riesgos inherentes a un flujo de trabajo automatizado.

Hecho: - Modificar `scripts/merci/merci-commit.py` para añadir una verificación previa.

Detalle técnico: - El script ahora ejecuta `git diff --quiet HEAD -- <ruta_bitacora>` antes de proceder. - Si el comando devuelve un código de salida 0 (sin cambios), se emite una alerta en la terminal y se solicita confirmación explícita del usuario para continuar.

Motivo / criterio: Reforzar la disciplina de "documentación primero" y prevenir el ruido en el historial de Git. La confirmación del usuario mantiene la flexibilidad para casos excepcionales sin sacrificar la seguridad del flujo por defecto.

Siguiente paso o deuda: Retomar el desarrollo del `functions.php` del Child Theme (Fase 4.2).

2026-04-15 — Configuración de alias de terminal (zsh) para el Sistema Merci

Contexto: Necesidad de optimizar la experiencia de desarrollo (DX) y reducir la fricción al invocar los scripts de automatización desde distintas ubicaciones del proyecto.

Hecho: - Recapitular y definir bloque de alias en `~/.zshrc` para las herramientas base: `merci-audit`, `merci-styles`, `merci-optimizer` y el nuevo `merci-commit`.

Detalle técnico: Se emplea la variable estática `MERCI_ROOT` apuntando a `/home/hildegahr/Escritorio/PROYECTO_mercedev.es` para garantizar la resolución de rutas absolutas al invocar Python, sin importar el directorio de trabajo actual (`pwd`).

Motivo / criterio: La carga cognitiva de recordar y tipear rutas relativas largas desincentiva el uso frecuente de herramientas críticas (como la auditoría o los commits atómicos). Abstraer esto en la terminal refuerza el flujo DevSecOps.

Siguiente paso o deuda: Validar la usabilidad del flujo con `merci-commit` y arrancar el código del `functions.php` del Child Theme (Fase 4.2).

2026-04-15 — Refactorización de `merci-commit.py` (Auto-Stage)

Contexto: El script de automatización de commits no incluía los archivos modificados del código, limitándose a comitear únicamente la bitácora.

Hecho: - Modificar `scripts/merci/merci-commit.py` para ejecutar `git add .` en la raíz del repositorio antes del commit.

Detalle técnico: - Se utiliza el argumento `cwd=REPO_ROOT` en `subprocess.run` para asegurar que el comando `git add .` abarque todo el proyecto, independientemente de desde dónde se invoque el script.

Motivo / criterio: Agilizar el flujo de trabajo. La seguridad y prevención de adición de código basura (secretos, archivos pesados) queda delegada a la red de seguridad del pre-commit (`merci-audit.py` y `.gitignore`), manteniendo la arquitectura "Shift-Left" intacta.

Siguiente paso o deuda: Validar la automatización y retomar el `functions.php` del Child Theme (Fase 4.2).

2026-04-15 — Pausa de Fase 4.2 para automatización de commits (I+D)

Contexto: Necesidad de vincular estrechamente la actualización de la bitácora con el historial de Git para evitar desincronización entre documentación y código.

Hecho: - Pausar temporalmente el desarrollo del `functions.php` del Child Theme. - Diseñar conceptualmente una herramienta de automatización para commits impulsados por la bitácora.

Detalle técnico: Se descarta el "auto-commit al guardar" (file watcher) por generar ruido (commit spam) y romper la atomicidad de Git. Se opta por crear un extractor que utilice la última entrada redactada como mensaje estructurado del commit.

Motivo / criterio: Mantener un historial de Git semántico, asegurando que el código modificado y su justificación (bitácora) viajen siempre juntos en un único commit atómico.

Siguiente paso o deuda: Desarrollar `scripts/merci/merci-commit.py` e integrarlo en el flujo de trabajo local.

2026-04-15 — Iniciar Fase 4.2 y creación base del Child Theme

Contexto: Iniciar el desarrollo del tema hijo ultraligero para WordPress (Fase 4.2), asegurando cero dependencias externas y preparando el enlace con el núcleo estático.

Hecho: - Crear directorio `src/wp-theme/merci-theme/` . - Crear archivo manifiesto `style.css` .

Detalle técnico: El archivo `style.css` contiene exclusivamente la cabecera de comentarios (`Theme Name` , `Version` , etc.) requerida por WP para reconocer el tema en el panel de administración. No incluye directivas de diseño.

Motivo / criterio: Evitar la duplicidad de renderizado y el código basura de los temas por defecto. El diseño real se delegará al `main.css` del núcleo estático para proteger la métrica de rendimiento (Core Web Vitals).

Siguiente paso o deuda: Crear el archivo `functions.php` como escudo para bloquear los scripts y estilos inyectados por defecto por WordPress.

2026-04-15 — Definir Arquitectura de Aislamiento de WordPress (Fase 4.1)

Contexto: Integrar WordPress para `/blog` y `/tienda` sin comprometer la seguridad, inmutabilidad y rendimiento puro originado en el núcleo estático de la carpeta `public/` .

Hecho: - Crear el documento técnico `docs/integracion-wordpress.md` . - Definir el enrutamiento proxy inverso mediante **Nginx**. - Configurar de forma teórica la preservación de canónicas (`siteurl` bloqueado a su subdirectorio) y `sitemap_index.xml` .

Detalle técnico: - Plantear una estructura de "Common root": `public/` alberga estáticos, mientras que el CMS reside en otra ruta del sistema anfitrión (ej. `/var/www/wordpress/`). Unir ambos mundos transparentemente usando la directiva `location ^~ /blog` . - Restringir estrictamente permisos: el proceso PHP de WordPress nunca podrá escribir en `public/` .

Motivo / criterio: Aislar vectores de ataque del CMS. Si el CMS es vulnerado (plugins desactualizados), el Frontend estático queda ileso. Además, se evita degradar el Web Vitals score de la portada sirviendo estáticos directamente con el web server.

Siguiente paso o deuda: Iniciar la Fase 4.2 que consiste en desarrollar el "Child Theme ultraligero" para el ecosistema de WordPress aislado.

2026-04-15 — Refactorización para resolver descoordinación de archivos

Contexto: Conflicto de convenciones de nombres y pérdida de coordinación de los scripts locales (`merci_sitemap.py` vs `merci-sitemap.py`) y pérdida de la compilación CSS (`main.scss`).

Hecho: - Restaurar explícitamente `@use 'index';` en `src/scss/main.scss` garantizando compilación exitosa a `public/css/main.css` . - Traspasar duplicidades experimentales (`merci_ingestor.py` , `merci_sitemap.py` , `pre-commit.sh`) a `laboratorio/scripts_temporales/` para mantener limpio el entorno y respetar la no eliminación de código. - Restaurar el script `scripts/merci/pre-commit` con la llamada correcta a `merci-sitemap.py` . - Actualizar el `README.md` para asentar todos los apuntes con las rutas veraces.

Detalle técnico: - Se confirma visualmente la reaparición de `main.css` . - Se limpia la carpeta `scripts/merci/` manteniéndola con `-` en lugar de `_` como convención primaria. - Movimiento realizado: `mv scripts/merci/merci_ingestor.py scripts/merci/merci_sitemap.py scripts/merci/pre-commit.sh laboratorio/scripts_temporales/`

Motivo / criterio: Consistencia y correspondencia con "lo que existe". Todo el proyecto ya está nuevamente compilando y acoplado.

Siguiente paso o deuda: Ninguno, el lío de archivos quedó resuelto.

2026-04-15 — Restauración integral de archivos y estabilización modular

Contexto: Pérdida de contenido en archivos tras renombrados y reorganización de carpetas.

Hecho: - Reconstruir `public/robots.txt` y `public/sitemap.xml` . - Restaurar `merci_ingestor.py` y el arnés de pruebas en `/tests` . - Preservar el experimento de grabación en `/laboratorio/art-de-cote` .

Detalle técnico: - Se asegura que los scripts utilicen nombres de archivo con guion bajo (`merci_sitemap.py`) para ser importables. - Los archivos pesados de vídeo permanecen excluidos en `.gitignore` .

Motivo / criterio: Garantizar la integridad del repositorio antes de avanzar a la Fase 3.

Siguiente paso o deuda: Iniciar el desarrollo de estilos SASS.

2026-04-15 — Reorganización modular de la carpeta Merci

Contexto: Evitar la dispersión de archivos en la carpeta de automatización separando los scripts operativos de las pruebas y los experimentos.

Hecho: - Creación de las subcarpetas `tests/` y `experimental/` en `scripts/merci/` . - Reubicación de `test_sitemap.py` y el aviso de deprecación de `merci-recorder.py` .

Detalle técnico: - Ajuste de `sys.path` en los tests para localizar módulos en el directorio padre (`parents[1]`).

Motivo / criterio: Modularidad y limpieza. Mantener la carpeta raíz de Merci enfocada únicamente en scripts productivos y validados.

Siguiente paso o deuda: Migrar futuros tests a la nueva carpeta y mover scripts en desarrollo a la zona experimental.

2026-04-15 — Preservación de Merci Recorder como pieza de Art de Coté

Contexto: Aplicación de la filosofía del proyecto para no descartar código experimental valioso tras el cambio de estrategia hacia el Ingestor.

Hecho: - Trasladar la lógica funcional de grabación a `laboratorio/art-de-cote/recorder_experiment.py` . - Mantener `scripts/merci/merci-recorder.py` como un stub de aviso (deprecación).

Detalle técnico: - La lógica preservada incluye la corrección del flag `-nostdin` y el uso de `x11grab` (X Window System - Sistema de Ventanas X). - Se categoriza como "Artefacto de Laboratorio" para consulta futura.

Motivo / criterio: El script falló para el flujo de producción diario pero es un activo de conocimiento sobre automatización multimedia con Python y FFmpeg.

Siguiente paso o deuda: Validar el funcionamiento del Ingestor en una sesión real.

2026-04-15 — Cambio de estrategia: Ingesta de evidencias en lugar de grabación directa

Contexto: El script `merci-recorder.py` no funcionaba correctamente y la necesidad de gestionar evidencias existentes (capturas de pantalla, vídeos) de forma más flexible.

Hecho: - Deprecación de `scripts/merci/merci-recorder.py` . - Creación de `scripts/merci/merci_ingestor.py` para escanear carpetas de usuario y mover archivos recientes a `.assets-raw/` . - Actualización de `README.md` e `instrucciones.md` para reflejar la nueva estrategia.

Detalle técnico: - `merci_ingestor.py` busca archivos modificados en los últimos 30 minutos en `~/Pictures` , `~/Videos` , `~/Desktop` (configurable). - Ofrece al usuario la opción de mover todos, algunos o ninguno de los archivos encontrados a `.assets-raw/` .

Motivo / criterio: Priorizar la funcionalidad de ingesta de evidencias existentes, que es más robusta y menos propensa a problemas de entorno que la grabación en tiempo real, y alinear con la gestión de `.assets-raw/` .

Siguiente paso o deuda: Probar `merci_ingestor.py` con archivos de prueba y documentar su uso en el `README.md` .

2026-04-15 — Resolución definitiva para visualización de vídeos de evidencias

Contexto: Fallo persistente en la instalación de extensiones de vídeo en VS Code, incluso usando el CLI y IDs de extensiones válidos.

Hecho: - Confirmar que la instalación de `b-ryan.vscode-video` vía CLI también falla. - Decidir utilizar reproductores externos (sistema o navegador web) para visualizar los archivos `.mp4` de `laboratorio/evidencias/` .

Detalle técnico: - El problema parece ser una limitación del entorno de VS Code o su acceso al Marketplace, no de la existencia de las extensiones. - La visualización externa es una solución robusta que no bloquea el flujo de trabajo.

Motivo / criterio: Priorizar el avance del proyecto y la generación de evidencias sobre la resolución de un problema de configuración del IDE que consume tiempo.

Siguiente paso o deuda: Iniciar la grabación de 30 minutos y proceder con la Fase 3 (Ingeniería de Estilos).

2026-04-15 — Incidencia persistente con el Marketplace de VS Code

Contexto: No es posible localizar extensiones de vídeo por ID en el Marketplace de la instancia local de VS Code.

Hecho: - Intentar instalación de `moshfeu.video-player` y `frenco.vs-code-media-preview` sin éxito. - Proponer instalación vía **CLI** (Command Line Interface - Interfaz de Línea de Comandos) de la extensión `b-ryan.vscodex-video`.

Detalle técnico: - Comando de rescate: `code --install-extension b-ryan.vscodex-video`. - Alternativa de visualización: uso del navegador host para validar evidencias MP4 si falla el IDE.

Motivo / criterio: Evitar la dispersión en problemas de configuración del entorno y priorizar el avance hacia la Fase 3 del Roadmap.

Siguiente paso o deuda: Validar visualización de la primera sesión de 30 min y proceder con SASS.

2026-04-15 — Clarificación sobre la extensión de visualización de video

Contexto: Dificultad para localizar la extensión "Video Player" (`moshfeu.video-player`) en el Marketplace de VS Code.

Hecho: - Reconfirmar la existencia y disponibilidad de la extensión. - Proporcionar instrucciones precisas para la búsqueda por ID (`moshfeu.video-player`).

Detalle técnico: - La búsqueda por ID es más robusta que por nombre, evitando ambigüedades o errores de tipografía.

Motivo / criterio: Asegurar que el desarrollador pueda instalar la herramienta necesaria para revisar las evidencias de video sin interrupciones.

Siguiente paso o deuda: Confirmar la instalación y reproducción de un video de prueba.

2026-04-15 — Corrección de herramienta: Extensión de visualización de video

Contexto: La extensión recomendada anteriormente (`frenco.vs-code-media-preview`) no se encuentra disponible en el Marketplace.

Hecho: Sustituir la recomendación por la extensión "Video Player" de moshfeu (`moshfeu.video-player`).

Detalle técnico: - La nueva extensión permite la previsualización de archivos `.mp4` y `.webm` directamente en el **IDE** (Integrated Development Environment - Entorno de Desarrollo Integrado).

Motivo / criterio: Garantizar que el flujo de revisión de evidencias en el laboratorio sea funcional con herramientas existentes y verificadas.

Siguiente paso o deuda: Validar la apertura de un vídeo de sesión de 30 minutos con esta nueva extensión.

2026-04-15 — Instalación de extensión para visualización de evidencias

Contexto: Necesidad de revisar los vídeos generados por `merci-recorder.py` sin romper el flujo de trabajo saliendo del editor.

Hecho: Seleccionar e instalar la extensión Media Preview (`frenco.vs-code-media-preview`).

Detalle técnico: - La extensión permite renderizar binarios de vídeo y audio en pestañas del **IDE** (Integrated Development Environment - Entorno de Desarrollo Integrado).

Motivo / criterio: Mantener la concentración en el entorno de desarrollo y facilitar la validación rápida de las capturas de pantalla antes de documentar en la bitácora.

Siguiente paso o deuda: Iniciar la grabación de 30 minutos y verificar la reproducción fluida dentro del editor.

2026-04-15 — Validación final y mejora de Merci Recorder

Contexto: Realizar prueba de humo del grabador y mejorar la flexibilidad para pruebas cortas.

Hecho: - Añadir soporte para argumentos de duración en `merci-recorder.py`. - Ejecutar prueba de 10 segundos exitosamente.

Detalle técnico: - Uso de `argparse` para parametrizar la duración. - Confirmación de que el flag `-nostdin` evita colisiones con la entrada de terminal. - Validación de `.gitignore`: los binarios generados no son trackeados por Git.

Motivo / criterio: Robustez y facilidad de prueba sin sacrificar la configuración por defecto de 30 min.

2026-04-15 — Corrección de error interactivo en Merci Recorder

Contexto: `ffmpeg` reportó un "Parse error" durante la grabación, causado por entrada inesperada del usuario en la terminal.

Hecho: - Identificar la causa del error como interacción accidental con el modo interactivo de `ffmpeg`. - Modificar `scripts/merci/merci-recorder.py` para añadir el flag `-nostdin`.

Detalle técnico: - El flag `-nostdin` evita que `ffmpeg` intente leer de la entrada estándar, previniendo errores de parseo por comandos no intencionados.

Motivo / criterio: Mejorar la robustez del script y la experiencia de usuario, evitando interrupciones por entradas accidentales.

Siguiente paso o deuda: Validar el comportamiento del script con el nuevo flag.

2026-04-15 — Prueba de humo y validación de Merci Recorder

Contexto: Verificar que el script de captura de pantalla funciona correctamente y que la exclusión en Git es efectiva.

Hecho: - Ejecución de prueba de `scripts/merci/merci-recorder.py` . - Verificación de salida en `laboratorio/evidencias/` .

Detalle técnico: - El script genera el contenedor `.mp4` usando el códec `libx264` . - `git status` confirma que los binarios de vídeo son ignorados por el sistema de control de versiones.

Motivo / criterio: Garantizar la trazabilidad visual de las sesiones de 30 min sin comprometer el peso del repositorio remoto.

2026-04-15 — Implementación de infraestructura de pruebas (QA)

Contexto: Ausencia de validación automatizada para los scripts de automatización de Merci.

Hecho: - Creación de `scripts/merci/test_sitemap.py` . - Definición de estrategia de pruebas unitarias usando la librería estándar de Python.

Detalle técnico: - Uso de `unittest.mock` para simular el sistema de archivos y evitar escrituras reales durante los tests. - Implementación de **TDD** (Test Driven Development - Desarrollo Dirigido por Pruebas) incipiente para los scripts de sistema.

Motivo / criterio: Garantizar la integridad de los metadatos de indexación y la estabilidad de las herramientas de automatización antes de avanzar a fases de diseño visual.

Siguiente paso o deuda: Ampliar la cobertura de pruebas a `merci-audit.py` .

2026-04-15 — Consolidación del flujo de grabación y protección de repositorio

Contexto: Asegurar que el nuevo sistema de grabación no impacte el tamaño del repositorio remoto.

Hecho: - Actualizar `.gitignore` para excluir binarios de vídeo en `laboratorio/evidencias/`. - Validar la integración de `merci-recorder.py` como herramienta de trazabilidad local.

Detalle técnico: - Adición de patrones `*.mp4` y `*.mov` específicos para la carpeta de evidencias.

Motivo / criterio: Autonomía en la captura de evidencias sin gestión manual de archivos externos, respetando la Regla 10 de austeridad en el repo remoto.

Siguiente paso o deuda: Iniciar la primera sesión de grabación de 30 minutos para validar el rendimiento del sistema.

2026-04-15 — Implementación de sistema de captura de vídeo (Merci Recorder)

Contexto: Necesidad de registrar sesiones de desarrollo de 30 minutos para trazabilidad del proceso en el Laboratorio.

Hecho: - Crear `scripts/merci/merci-recorder.py`. - Integrar lógica de captura automática de pantalla con FFmpeg.

Detalle técnico: - Uso de `x11grab` para la **GUI** (Graphical User Interface - Interfaz Gráfica de Usuario). - Configuración de duración fija a 1800 segundos (30 minutos). - Codificación en tiempo real optimizada para baja carga de **CPU** (Central Processing Unit - Unidad Central de Procesamiento).

Motivo / criterio: Facilitar la generación de evidencias sin interrumpir el flujo de trabajo manual, manteniendo la coherencia con la Regla 10 de gestión de archivos pesados.

Siguiente paso o deuda: Validar el peso de los archivos generados y ajustar el **CRF** (Constant Rate Factor - Factor de Tasa Constante) si superan los 50MB por sesión.

2026-04-15 — Política de gestión de evidencias pesadas en el Laboratorio

Contexto: Necesidad de evitar el crecimiento excesivo del repositorio Git por la inclusión de vídeos y capturas de pantalla de gran tamaño.

Hecho: - Definir regla de exclusión de binarios pesados en `laboratorio/evidencias/`. - Actualizar `instrucciones.md` con la norma de "Evidencias Pesadas".

Detalle técnico: - Se establece que `merci-optimizer.py` (o extensiones futuras) se encargará de reducir el material de pruebas antes de su clasificación. - Los archivos originales (brutos) se mantienen en la carpeta externa de capturas o en `.assets-raw/evidencias/` (fuera de Git).

Motivo / criterio: Mantener un repositorio ligero y profesional, evitando el bloqueo por cuotas de GitHub y asegurando clones rápidos.

Siguiente paso o deuda: Configurar `.gitignore` para excluir extensiones de vídeo (`.mp4` , `.mov`) dentro de la carpeta de evidencias.

2026-04-15 — Pruebas de visualización en navegador e hitos UX/UI (Fase 2)

Contexto: Validar el renderizado real del `index.html` tras la aplicación de la jerarquía semántica y la estructura BEM.

Hecho: - Generar informes PDF con capturas del sitio en navegador. - Crear carpeta `laboratorio/evidencias/` para organizar los artefactos de prueba.

Detalle técnico: (Aquí puedes anotar si detectaste algún error de alineación, fuentes o comportamiento responsivo en el PDF).

Motivo / criterio: Evitar la dispersión de archivos en la raíz del laboratorio y asegurar que las decisiones de diseño tienen un respaldo visual documentado.

Siguiente paso o deuda: (Anotar si hay que retocar algún margen o color tras ver el PDF).

2026-04-15 — Refactorización a Módulos SASS y Dart Sass Standalone (Fase 3)

Contexto: Se identificó que la librería Python `libsass` no soportaba las directivas modulares (`@use` , `@forward` , `_index.scss`) que permiten una arquitectura de estilos moderna y desacoplada.

Hecho: - Reconfiguración de `src/scss/` incluyendo archivos `_index.scss` que reexportan las partes. - `main.scss` simplificado a sólo incluir los índices de cada subcarpeta. - Eliminación de `libsass` de `requirements.txt` . - Modificación estructural de `scripts/merci/merci-styles.py` : ya no es un script de Python que importe librerías, sino un autómata que descarga la release oficial del binario *Dart Sass* para Linux, extrae el compilador localmente sin impactar el sistema operativo host, y procesa los estilos.

Detalle técnico: - Almacenaje de los binarios locales de SASS en `scripts/merci/bin/dart-sass/sass` . - Se llama al proceso aisladamente con `subprocess` de la librería estándar de Python.

Motivo / criterio: - Dar soporte al mejor estilo posible de escritura SASS modular pero evadir a toda costa la necesidad de forzar la instalación global de Node.js o NPM para usar un compilador web, protegiendo así el Paradigma base de "0 dependencias externas host".

Siguiente paso o deuda: Validar rendimiento continuo del compilador e iniciar implementación de hojas visuales para nuevos componentes.

2026-04-15 — Implementación de la Fase 3: SASS, BEM y Merci Optimizer

Contexto: Desplegar el sistema de estilos escalable (SASS) y preparar la automatización para multimedia.

Hecho: - Creación de la arquitectura 7-1 en `src/scss/` con punto de entrada único (`main.scss`). - Refactorización de `public/index.html` asimilando la

metodología BEM. - Creación de dos piezas fundamentales para Merci: `merci-styles.py` (compilador con libsass) y `merci-optimizer.py` (escalado WebP con Pillow). - `requirements.txt` ajustado para compilar localmente con Python.

Detalle técnico: - `merci-styles.py` invoca a libsass asilando su función y ahorrando uso manual de consola. - `.assets-raw/` será escrutado por Merci procesando imágenes WebP hacia `assets/` a medidas predeterminadas.

Motivo / criterio: Se eligió `libsass` de Python para unificar el DevSecOps de Merci sin depender de un entorno NodeJS global adicional en Ubuntu, en línea con la filosofía de austeridad tecnológica externa.

Siguiente paso o deuda: Validar la instalación con pip y hacer un chequeo de `index.html` estéticamente en navegador.

2026-04-14 — Validación de jerarquía de encabezados y landmarks (Fase 2.1)

Contexto: Asegurar la accesibilidad y la estructura semántica correcta en la página de inicio.

Hecho: - Añadir encabezado `<h2>` a la sección `#ecosistema` para evitar saltos de nivel. - Incorporar `aria-label` al elemento `<nav>`. - Actualizar hitos en `README.md`.

Detalle técnico: - Se garantiza que el árbol de encabezados sea secuencial: `h1 > h2 > h3`. - El uso de **Landmarks** (Puntos de referencia) facilita la navegación a usuarios con tecnologías de asistencia.

Motivo / criterio: Cumplir con los estándares de **WAI-ARIA** (Web Accessibility Initiative - Accessible Rich Internet Applications - Iniciativa de Accesibilidad Web - Aplicaciones de Internet Enriquecidas Accesibles) y SEO técnico.

Siguiente paso o deuda: Iniciar la Fase 3 (Ingeniería de Estilos).

2026-04-14 — Integración de merci-sitemap.py en el hook de pre-commit

Contexto: Automatizar la actualización de la fecha `<lastmod>` en `sitemap.xml` cada vez que se realicen cambios en la carpeta `public/`.

Hecho: Modificar `scripts/merci/pre-commit`.

Detalle técnico: - Se añadió lógica para detectar archivos staged en `public/`. - Si se detectan cambios, se ejecuta

`python3 scripts/merci/merci-sitemap.py` . - Se añade `public/sitemap.xml` al índice de Git (`git add public/sitemap.xml`) para incluir su modificación en el commit actual.

Motivo / criterio: Asegurar que `sitemap.xml` refleje siempre la fecha de la última modificación de contenido relevante, mejorando la precisión del SEO técnico.

Siguiente paso o deuda: Realizar un commit de prueba que incluya cambios en `public/` para validar el funcionamiento del hook.

2026-04-14 — Automatización de metadatos de indexación (Sitemap)

Contexto: Evitar la actualización manual de la fecha de última modificación en el `sitemap.xml` para mejorar el SEO técnico.

Hecho: Crear script `scripts/merci/merci-sitemap.py` para la gestión automática de fechas en archivos XML.

Detalle técnico: - Uso de la librería `datetime` para obtener la fecha del sistema. - Empleo de `re.sub` para manipular el contenido del XML sin necesidad de parsers pesados.

Motivo / criterio: Mantener la consistencia entre los cambios reales y lo que se informa a los motores de búsqueda de forma automatizada.

Siguiente paso o deuda: Integrar la ejecución de este script en el flujo de publicación o en un hook de post-commit.

2026-04-14 — Cierre de Fase 1 y creación de activos de indexación (Fase 2.3)

Contexto: Finalización formal de la infraestructura base y configuración de la visibilidad para buscadores del núcleo estático.

Hecho: - Actualizar `README.md` para reflejar la Fase 1 como completada. - Crear `public/robots.txt` y `public/sitemap.xml`.

Detalle técnico: - `robots.txt` : Configurado para permitir el rastreo total y apuntar al mapa del sitio. - `sitemap.xml` : Generado con la URL canónica raíz y prioridad máxima.

Motivo / criterio: Cumplir con los estándares de **SEO** (Search Engine Optimization - Optimización para Motores de Búsqueda) técnico definidos en el roadmap.

Siguiente paso o deuda: Validar la jerarquía de encabezados (Fase 2.1) para asegurar accesibilidad.

2026-04-14 — Validación de Fase 2 (HTML y SEO Técnico) con Merci Audit

Contexto: Verificación del primer documento semántico del núcleo estático frente a las reglas de auditoría.

Hecho: Ejecutar `merci-audit.py --strict-json-ld` sobre `public/index.html`.

Detalle técnico: - El archivo cumple con los requisitos de metadatos, charset y lenguaje. - Se valida el bloque JSON-LD (JavaScript Object Notation for Linked Data - Notación de Objetos JavaScript para Datos Enlazados) usando el esquema de `schema.org`.

Motivo / criterio: Garantizar que el sitio es indexable y cumple con los estándares de rendimiento y SEO (Search Engine Optimization - Optimización para Motores de Búsqueda) desde la primera línea de código.

Siguiente paso o deuda: Implementar navegación (Fase 2.1) y generar `robots.txt` / `sitemap.xml` (Fase 2.3).

2026-04-14 — Creación de proyecto y obtención de API Key vía AI Studio

Contexto: El error 404 inicial no era solo de configuración de software, sino de falta de infraestructura (proyecto) en el lado de Google.

Hecho: Generar una API Key a través de Google AI Studio vinculada a un proyecto nuevo creado automáticamente por la plataforma.

Detalle técnico: - Acceso a `aistudio.google.com` . - Uso de la opción "Create API key in new project" para evitar la configuración manual en GCP (Google Cloud Platform - Plataforma en la Nube de Google) Console.

Motivo / criterio: Vía más rápida para habilitar `gemini-1.5-pro` sin gestionar capas de facturación o cuotas complejas de Google Cloud de entrada.

Siguiente paso o deuda: Probar la conexión en Continue una vez la API Key esté activa y propagada.

2026-04-14 — Corrección de error 404 en Continue (Gemini 1.5 Pro)

Contexto: Fallo en la conexión con la API de Google al usar `gemini-1.5-pro` en Continue, con un error 404.

Hecho: Identificar que el `provider` en el archivo `/home/hildegahr/.continue/config.yaml` estaba configurado incorrectamente como `gemini` .

Detalle técnico: Modificar el `provider` de `gemini` a `google-generative-ai` para el modelo `gemini-1.5-pro` en la configuración de Continue.

Motivo / criterio: El `provider` `google-generative-ai` es el nombre correcto para interactuar con la API de Google Gemini a través de Continue.

Siguiente paso o deuda: Crear el proyecto en Google Cloud / AI Studio.

2026-04-12 — Fase 1: infraestructura, Merci Audit y primer commit

Contexto: Arranque del repositorio bajo las directrices de `instrucciones.md` (rendimiento, seguridad shift-left, pedagogía). Objetivo de la Fase 1: estructura de carpetas, script de auditoría local y base Git.

Hecho:

- Estructura aprobada en la raíz: `docs/` , `biblioteca/` , `laboratorio/` , `scripts/merci/` , `assets/` , `.assets-raw/` (las carpetas vacías se versionan con `.gitkeep` para que un `git clone` conserve el esqueleto).
- `scripts/merci/merci-audit.py` : auditoría con biblioteca estándar de Python (sin dependencias pip obligatorias en esta fase). Comprueba entre otras cosas patrones de secretos, sintaxis de `.py` , JSON, avisos en JS (`eval` / `new Function`) y reglas SEO mínimas en `.html` / `.htm` .
- `scripts/merci/pre-commit` : shell que ejecuta `merci-audit.py --git-staged` (solo lo que va al commit).
- Enlace local de Git: `.git/hooks/pre-commit` → `../../scripts/merci/pre-commit` (los hooks no viajan con el clone; hay que recrear el enlace en cada máquina o documentar un bootstrap).
- `.gitignore` para `.venv/` , cachés y artefactos de build; `requirements.txt` reservado para fases posteriores (p. ej. Pillow en optimizador).
- Commit inicial en rama `main` con mensaje tipo *chore: commit inicial — Fase 1 (estructura, Merci Audit, directrices)*.

Detalle técnico:

- Auditoría sobre todo el árbol: `python3 scripts/merci/merci-audit.py`
- Solo índice (staged), pensado para hook: `python3 scripts/merci/merci-audit.py --git-staged`
- Exigir JSON-LD en HTML cuando toque endurecer CI: flag `--strict-json-ld`
- Instalar hook (desde la raíz del repo): `chmod +x scripts/merci/pre-commit scripts/merci/merci-audit.py` y `ln -sf ../../scripts/merci/pre-commit .git/hooks/pre-commit`

- Saltar el hook solo si es deliberado: `git commit --no-verify`

Motivo / criterio: Automatizar comprobaciones antes de integrar cambios encaja con “seguridad shift-left” y con el papel de `merci-audit.py` descrito en instrucciones. Staged-only evita auditar el mundo en cada commit y acelera el flujo.

Siguiente paso o deuda: Fase 2 — HTML semántico, JSON-LD e indexación; primer documento público o plantilla que pase el audit sin `--no-verify`.

2026-04-12 — Registro cronológico acumulativo (no sustituir historial)

Contexto: Asegurar que la bitácora no pierda contexto al añadir sesiones nuevas.

Hecho: En `instrucciones.md` (regla 6) y en «Cómo mantenerlo» de este archivo quedó explícito: nuevas entradas **solo al final** del registro; no reemplazar ni borrar bloques ya escritos salvo corrección puntual o retirada de datos sensibles, con motivo claro.

Detalle técnico: N/A.

Motivo / criterio: El historial del laboratorio es activo de trazabilidad; sobrescribirlo rompería la línea temporal para el «yo futuro» y para el traslado a `biblioteca/`.

Siguiente paso o deuda: Seguir añadiendo entradas bajo «Registro cronológico» sin editar entradas previas salvo las excepciones acordadas.

2026-04-12 — `.assets-raw` : solo local, sin originales en Git

Contexto: Evitar que PSD, RAW, vídeos u otros brutos acaben en GitHub.

Hecho: `.gitignore` pasa a ignorar `.assets-raw/*` con excepción de `.assets-raw/.gitkeep`. `instrucciones.md` y `README.md` describen que la carpeta es convención de trabajo local y que lo versionado en `/assets` es lo optimizado.

Detalle técnico: Patrón en `.gitignore` : `!.assets-raw/.gitkeep` tras `.assets-raw/*`.

Motivo / criterio: Repositorio ligero y reproducible; los originales viven fuera del remoto (disco, NAS, etc.).

Siguiente paso o deuda: En Fase 3, documentar el flujo concreto `merci-optimizer.py` de `.assets-raw` → `assets/`.

2026-04-12 — Documentación pública sin notas personales al mantenedor

Contexto: Evitar frases tipo “cuando lo tengas claro añade LICENSE” en el README u otros textos versionados para GitHub.

Hecho: `README.md` (Licencia y otras frases) redactado en tono neutro. Nueva regla 7 en `instrucciones.md`: recordatorios al autor fuera del repo; en Git, texto útil para visitantes o colaboradores.

Detalle técnico: N/A.

Motivo / criterio: El remoto es documentación de producto/proyecto, no la libreta personal.

Siguiente paso o deuda: Revisar futuros `docs/` públicos con el mismo criterio.

2026-04-12 — Fase 2: carpeta `public/` como raíz del documento

Contexto: Inicio de la Fase 2 por la estructura antes del primer HTML.

Hecho: Directorio `public/` en el repo con `.gitkeep`; entrada en §3 de `instrucciones.md` y fila en `README.md`. Convención: aquí vive el núcleo estático servido como documento raíz; WP fuera hasta Fase 4.

Detalle técnico: Nombre elegido: `public/` (convención habitual de “document root” en despliegues estáticos).

Motivo / criterio: Separar claramente sitio servido, automatización, conocimiento y brutos locales.

Siguiente paso o deuda: `public/index.html` semántico + JSON-LD + `robots.txt` / `sitemap.xml` en la misma raíz cuando toque.
